PERBANDINGAN KINERJA FRAMEWORK RASA DAN RAG PADA CHATBOT LAYANAN KEMAHASISWAAN

SKRIPSI

Disusun Sebagai Salah Satu Syarat
untuk Memperoleh Gelar Sarjana Komputer
pada Program Studi Informatika

Disusun oleh:
ALFONSO CLEMENT SUTANTIO
24060122130080

DEPARTEMEN INFORMATIKA
FAKULTAS SAINS DAN MATEMATIKA
UNIVERSITAS DIPONEGORO
2026

HALAMAN PERNYATAAN KEASLIAN SKRIPSI

Saya yang bertanda tangan di bawah ini :

Nama : Alfonso Clement Sutantio

NIM : 24060122130080

Judul : Perbandingan Kinerja *Framework* RASA dan RAG pada *Chatbot* Layanan Kemahasiswaan

Dengan ini saya menyatakan bahwa dalam skripsi ini tidak terdapat karya yang pernah diajukan untuk memperoleh gelar kesarjanaan di suatu Perguruan Tinggi, dan sepanjang pengetahuan saya juga tidak terdapat karya atau pendapat yang pernah ditulis atau diterbitkan oleh orang lain, kecuali yang secara tertulis diacu dalam naskah ini dan disebutkan di dalam daftar pustaka.

Semarang, 28 Januari 2026

Alfonso Clement Sutantio

24060122130080

HALAMAN PENGESAHAN

Judul : Perbandingan Kinerja *Framework* RASA dan RAG pada *Chatbot* Layanan Kemahasiswaan

Nama : Alfonso Clement Sutantio

NIM : 24060122130080

Telah diujikan pada sidang tugas akhir dan dinyatakan lulus pada tanggal 28 Januari 2026

Semarang, 28 Januari 2026

Pembimbing 1,	Pembimbing 2,
Dr. Indra Waspada, S.T., M.TI	Sandy Kurniawan, S.Kom., M.Kom
NIP. 97902122008121002	NIP. 199603032024061003

Mengetahui,

Ketua Departemen Informatika	Ketua Panitia Penguji Tugas Akhir
Dr. Aris Sugiharto, S.Si., M.Kom.	Dr. Sutikno, S.T., M.Cs.
NIP. 197108111997021004	NIP. 197905242009121003

KATA PENGANTAR

Segala puji syukur dipanjatkan ke hadirat Tuhan yang Maha Esa atas rahmat dan karunia-Nya, sehingga penulis dapat menyelesaikan penulisan skripsi dengan judul "Perbandingan Kinerja *Framework* RASA dan RAG pada *Chatbot* Layanan Kemahasiswaan" sebagai syarat untuk memperoleh gelar sarjana strata-satu (S1) pada Program Studi Informatika Departemen Informatika Faultas Sains dan Matematika Universitas Diponegoro Semarang.

Dalam penulisan dan penyusunan skripsi ini, penulis mendapatkan banyak bantuan, bimbingan, dan dukungan dari berbagai pihak. Maka dari itu dalam kesempatan ini, penulis menyampaikan ucapan terima kasih kepada:

1. Prof. Dr. Kusworo Adi, S.Si., M.T. selaku Dekan Fakultas Sains dan Matematika Universitas Diponegoro
2. Dr. Aris Sugiharto, S.Si., M.Kom. selaku Ketua Departemen Informatika.
3. Nurdin Bahtiar, S.Si., M.T, selaku Koordinator Tugas Akhir.
4. Dr. Indra Waspada, S.T., M.TI. dan Sandy Kurniawan, S.Kom., M.Kom. selaku dosen pembimbing yang telah bersedia meluangkan waktunya untuk memberikan saran, masukan, dan pemecahan masalah selama penulisan skripsi.
5. Dr. Sutikno, S.T., M.Cs. dan Yunila Dwi Putri Ariyanti, S.Kom., M.Kom. selaku dosen penguji yang telah bersedia meluangkan waktu untuk menguji dan memberikan masukan terkait Tugas Akhir.
6. Pihak Fakultas Sains dan Matematika Universitas Diponegoro yang telah memberikan izin kepada penulis untuk dapat melakukan penelitian.
7. Keluarga dan sahabat yang selalu memberikan dukungan dan doa.

Penulis menyadari bahwa penulisan skripsi ini masih jauh dari kata sempurna. Oleh karena itu, penulis menerima segala kritik, saran, dan masukan yang bersifat membangun. Semoga skripsi yang telah dikerjakan oleh penulis dapat bermanfaat bagi yang membutuhkan.

Semarang, 28 Januari 2026

Penulis

HALAMAN PERNYATAAN PERSETUJUAN PUBLIKASI SKRIPSI UNTUK KEPENTINGAN AKADEMIS

Sebagai civitas akademik Universitas Diponegoro, saya yang bertanda tangan di bawah ini:

Nama	:	Alfonso Clement Sutantio
NIM	:	24060122130080
Program Studi	:	Informatika
Departemen	:	Informatika
Fakultas	:	Sains dan Matematika
Jenis Karya	:	Skripsi

demi pengembangan ilmu pengetahuan, menyetujui untuk memberikan **Hak Bebas Royalti Noneksklusif (Non-exclusive Royalty Free Right)** kepada Universitas Diponegoro atas karya ilmiah saya yang berjudul:

Perbandingan Kinerja Framework RASA dan RAG pada Chatbot Layanan Kemahasiswaan

beserta perangkat yang ada (jika diperlukan). Dengan Hak Bebas Royalti Non-eksklusif ini Universitas Diponegoro berhak menyimpan, mengalihmedia/formatkan, mengelola dalam bentuk pangkalan data (*database*), merawat, dan mempublikasikan skripsi saya tanpa meminta izin dari saya selama tetap mencantumkan nama saya sebagai penulis/pencipta dan sebagai pemilik Hak Cipta.

Demikian pernyataan ini saya buat dengan sebenarnya.

Semarang, 28 Januari 2026

Alfonso Clement Sutantio

24060122130080

ABSTRAK

Kebutuhan informasi akademik dan kemahasiswaan merupakan aspek krusial bagi mahasiswa. Di lingkungan Fakultas Sains dan Matematika Universitas Diponegoro (FSM Undip), mahasiswa harus mendatangi secara langsung bagian layanan akademik di Gedung Acintya Prasada untuk memperoleh informasi terkait. Pendekatan ini menimbulkan sejumlah permasalahan, antara lain inefisiensi waktu akibat antrean, potensi kegagalan dalam menemui staf terkait, serta tingginya repetisi pertanyaan yang diajukan. Oleh karena itu, penelitian ini mengusulkan solusi otomasi layanan melalui perbandingan performa dua *framework chatbot*, yaitu *Retrieval-Augmented Generation* (RAG) dan RASA. Kedua *framework* diimplementasikan menggunakan sumber pengetahuan yang sama, berupa tujuh dokumen Standar Operasional Prosedur (SOP) layanan akademik dan kemahasiswaan. Evaluasi dilakukan menggunakan lima metrik, yaitu *response time*, *coverage*, ROUGE-1, ROUGE-L, dan BERTScore. Hasil penelitian menunjukkan bahwa akurasi jawaban kedua *framework* tidak berbeda secara signifikan. *Framework* RASA memiliki keunggulan utama pada kecepatan respons, dengan performa rata-rata 658 kali lebih cepat dibandingkan RAG. Namun, keunggulan tersebut disertai dengan kebutuhan upaya yang lebih besar dalam menyiapkan sumber pengetahuan, khususnya pada tahap pengembangan dan pelatihan komponen *Natural Language Understanding* (NLU). Keterbatasan ini dapat diatasi melalui pemanfaatan *framework* RAG yang menawarkan kemudahan dalam persiapan serta penambahan sumber pengetahuan secara langsung dari dokumen yang tersedia. Meskipun demikian, *framework* RAG memiliki kelemahan berupa kebutuhan komputasi yang relatif lebih besar.

Kata kunci : Chatbot Akademik, Retrieval Augmented Generation, RASA Framework, Perbandingan Framework

ABSTRACT

The need for academic and student-related information constitutes a crucial aspect for university students. In the Faculty of Science and Mathematics at Diponegoro University (FSM Undip), students are required to directly visit the academic services office located in the Acintya Prasada Building to obtain relevant information. This approach gives rise to several issues, including time inefficiency due to queues, the potential inability to meet the appropriate staff, and a high level of repetitive inquiries. Therefore, this study proposes an automated service solution through a performance comparison of two chatbot frameworks, namely Retrieval-Augmented Generation (RAG) and RASA. Both frameworks were implemented using the same knowledge source, consisting of seven Standard Operating Procedure (SOP) documents related to academic and student services. The evaluation was conducted using five metrics: response time, coverage, ROUGE-1, ROUGE-L, and BERTScore. The results indicate that the answer accuracy of both frameworks does not differ significantly. The RASA framework demonstrates a primary advantage in response speed, with an average performance 658 times faster than RAG; however, this advantage is accompanied by greater effort in preparing the knowledge source, particularly during the development and training of the Natural Language Understanding (NLU) component. This limitation can be addressed by utilizing the RAG framework, which offers ease in preparation and direct integration of knowledge sources from existing documents, although it requires relatively higher computational resources.

Keywords : Academic Chatbot, Retrieval Augmented Generation, RASA Framework, Framework Comparison

DAFTAR ISI

HALAMAN PERNYATAAN KEASLIAN SKRIPSI ii
HALAMAN PENGESAHAN iii
KATA PENGANTAR iv
HALAMAN PERNYATAAN PERSETUJUAN PUBLIKASI SKRIPSI v
ABSTRAK vi
ABSTRACT vii
DAFTAR ISI viii
BAB I PENDAHULUAN 1
1.1. Latar Belakang 1
1.2. Rumusan Masalah 4
1.3. Tujuan Penelitian 4
1.4. Manfaat Penelitian 4
1.5. Ruang Lingkup 4
1.6. Sistematika Penelitian 5
BAB II TINJAUAN PUSTAKA 7
2.1. Penelitian Terkait 7
2.2. *Chatbot Model* 11
2.3. *Natural Language Processing* 12
2.4. *Deep Learning* 13
2.5. *Transformer Model* 13
2.6. *Large Language Model* 15
2.7. *Chunking Process* 15
2.8. *Embedding Process* 17
2.9. *Vector Database* 18
2.10. *Retrieval-Augmented Generation* 19
2.11. RASA *Framework* 21
2.12. *Evaluation Metrices* 22
2.12.1. ROUGE *Metric* 23
2.12.2. BERTScore *Metric* 25
BAB III METODOLOGI PENELITIAN 28
3.1. Perolehan 7 Dokumen SOP 28

3.2. Wawancara dengan Pihak FSM 29
3.3. Pengembangan *Chatbot* 29
3.3.1. Pengembangan *Chatbot* RAG 29
3.3.1.1. Data Preparation 30
3.3.1.2. Data Modeling 32
3.3.1.3. Knowledge Store 32
3.3.1.4. Data Retrieval 33
3.3.1.5. LLM Generation 34
3.3.1.6. Evaluation Process 34
3.3.2. Pengembangan *Chatbot* RASA 35
3.3.2.1. Data Preparation 36
3.3.2.2. Data Training 36
3.3.2.3. Prediction Stage 36
3.3.2.4. Result Generation 37
3.3.2.5. Evaluation Process 37
3.4. Perbandingan Performa Kedua Model 37
BAB IV HASIL DAN PEMBAHASAN 39
4.1. Pengembangan *Chatbot* RAG 39
4.1.1. *Data Preparation* 39
4.1.2. *Data Modeling* 44
4.1.3. *Knowledge Store* 56
4.1.4. *Data Retrieval* 57
4.1.5. LLM *Generation* 61
4.1.6. *Evaluation Process* 65
4.2. Pengembangan *Chatbot* RASA 68
4.2.1. *Data Preparation* 68
4.2.2. *Data Training* 72
4.2.3. *Prediction Stage* 77
4.2.4. *Result Generation* 80
4.2.5. *Evaluation Process* 82
4.3. Perbandingan Performa Kedua Model 85
BAB V PENUTUP 101
5.1. Kesimpulan 101

5.2. Saran 102
DAFTAR PUSTAKA 103
LAMPIRAN 1. Hasil Wawancara dengan Staf Akademik 109
LAMPIRAN 2. Isi Mentah Dokumen Sumber Pengetahuan 112
LAMPIRAN 3. Isi Dokumen Hasil Penyesuaian Manual 115
LAMPIRAN 4. Jawaban dan *Ground Truth* Evaluasi Model RAG 119
LAMPIRAN 5. Daftar Komponen yang Digunakan Pada RASA 124
LAMPIRAN 6. Jawaban dan *Ground Truth* Evaluasi Model RASA 133

DAFTAR TABEL

Tabel 2.1. Daftar Penelitian Terkait 7
Tabel 2.2. Model Populer dan Karakteristiknya (Kim dkk., 2025) 15
Tabel 3.1. Spesifikasi Perangkat Keras 29
Tabel 3.2. Konfigurasi Eksperimen pada Proses *Chunking* 31
Tabel 3.3. Contoh Representasi Vektor Dokumen dan *Query* 32
Tabel 4.1. Hasil Konfigurasi *Chunk* pada Salah Satu Dokumen Sumber 41
Tabel 4.2. Jumlah Pecahan Dokumen Sumber Proses *Chunk* 43
Tabel 4.3. Hasil Pembuatan QA dari Setiap Dokumen SOP 47
Tabel 4.4. Contoh Hasil Proses *Data Retrieval* 59
Tabel 4.5. Hasil Evaluasi Model RAG Terhadap *Frequently Asked Question* 66
Tabel 4.6. Hasil *Lexical Syntatic Featurizer* 78
Tabel 4.7. Hasil Klasifikasi Model DIET 79
Tabel 4.8. Tabel Hasil Prediksi Model 79
Tabel 4.9. Tabel Contoh *Rule* untuk *Intent* “tanya_legalisir” 80
Tabel 4.10. Tabel Contoh *Stories* Proses Legalisir 81
Tabel 4.11. Tabel Hasil Akhir Jawaban *Chatbot* 82
Tabel 4.12. Hasil Evaluasi Model RASA Terhadap *Frequently Asked Question* 83
Tabel 4.13. Perbandingan Waktu Respon RAG pada Pertanyaan Nomor 1 dan 10 86
Tabel 4.14. Perbandingan Waktu Respon RASA pada Pertanyaan Nomor 1 dan 2 88
Tabel 4.15. Perbandingan Waktu Respon RASA dan RAG 89
Tabel 4.16. Perbandingan Kemampuan *Coverage* Kedua Model *Chatbot* 89
Tabel 4.17. Perbandingan *Coverage* RAG pada Pertanyaan 3 dan 4 89
Tabel 4.18. Perbandingan *Coverage* RASA pada Pertanyaan 3 dan 8 90
Tabel 4.19. Perbandingan Skor ROUGE-1 Kedua Model pada Jawaban Benar 92
Tabel 4.20. Jawaban Nomor 7 model RAG dan RASA terhadap *Ground Truth* 93
Tabel 4.21. Perbandingan Skor ROUGE-1 Model pada Pertanyaan Tak Terjawab 93
Tabel 4.22. Perbandingan Skor ROUGE-L Kedua Model pada Jawaban Benar 95
Tabel 4.23. Perbandingan Skor ROUGE-L Model pada Pertanyaan Tak Terjawab 95
Tabel 4.24. Jawaban model RAG dan RASA terhadap *Ground Truth* 97
Tabel 4.25. Perbandingan Skor BERTScore Kedua Model pada Jawaban Benar 98

DAFTAR GAMBAR

Gambar 2.1. The Transformer – Model Architecture (Vaswani dkk., 2017) 14
Gambar 2.2. Contoh Stuktur *Embedding* berbasis Model BERT (Dadure dkk., 2022) 17
Gambar 2.3. Tahapan-Tahapan pada *Framework* RAG (Zhou dkk., 2024)..... 20
Gambar 2.4. Contoh Komputasi pada BERTScore (Zhang dkk., 2020) 27
Gambar 3.1. Alur Tahapan Penelitian 28
Gambar 3.2. *Pipeline* Pengembangan *Chatbot* RAG 30
Gambar 3.3. *Pipeline* Pengembangan *Chatbot* RASA 35
Gambar 4.1. *Preview* Struktur *Knowledge Database* yang dihasilkan..... 57
Gambar 4.2. *Preview* Implementasi NLU *Component* dalam *Code* 69
Gambar 4.3. *Preview* Implementasi Komponen *Rules* dalam *Code* 70
Gambar 4.4. *Preview* Implementasi Komponen *Stories* dalam *Code* 71
Gambar 4.5. *Preview* Implementasi Komponen *Domain* dalam *Code*..... 72
Gambar 4.6. Waktu Respon Model RAG pada Setiap Pertanyaan 85
Gambar 4.7. Waktu Respon Model RASA pada Setiap Pertanyaan 87
Gambar 4.8. Perbandingan Metrik ROUGE-1 Kedua Model 92
Gambar 4.9. Perbandingan Metrik ROUGE-L Kedua Model 95
Gambar 4.10. Perbandingan Metrik BERTScore Kedua Model 96
Gambar 4.11. Visualisasi Akhir Perbandingan Kedua Model 99

DAFTAR SOURCE CODE

Source Code 4.1. Implementasi Proses *Chunking* 40
Source Code 4.2. Implementasi Pembuatan QA untuk *Fine-tuning* 45
Source Code 4.3. Implementasi Proses *Fine-tuning* Model *Embedding* 49
Source Code 4.4. Implementasi Proses *Embedding* 52
Source Code 4.5. Implementasi Pemrosesan *User Query* 58
Source Code 4.6. Implementasi *Prompt Template* 62
Source Code 4.7. Implementasi Bagaimana LLM Menghasilkan Jawaban 62
Source Code 4.8. Implementasi Metrik Evaluasi 65
Source Code 4.9. Pendefinisian Komponen *Pipeline* dan *Policies* dalam *Code* 72

BAB I
PENDAHULUAN

Bab pendahuluan ini membahas mengenai latar belakang masalah, rumusan masalah, tujuan dan manfaat, ruang lingkup, serta sistematika penulisan yang akan digunakan dalam dokumen skripsi ini.

1.1. Latar Belakang

Layanan akademik menjadi salah satu landasan operasional sebuah universitas yang berfungsi sebagai antarmuka utama yang memastikan keberlangsungan dan kualitas pengalaman belajar mengajar (NGO dkk., 2025; Peña-Lang dkk., 2023). Layanan yang responsif dan akurat tidak hanya mendukung efisiensi administratif, tetapi juga secara langsung memengaruhi kepuasan, retensi, dan keberhasilan akademik mahasiswa (Osman dkk., 2024). Dalam lingkungan kampus, kebutuhan mahasiswa akan akses cepat terhadap informasi akademik, seperti prosedur pengajuan proposal kegiatan, permohonan surat rekomendasi beasiswa, atau informasi pembayaran Uang Kuliah Tunggal (UKT) terus meningkat (Joshi dkk., 2024).

Secara konvensional, untuk mengakses informasi-informasi tersebut, mahasiswa harus mendatangi langsung bagian layanan akademik. Pendekatan ini dinilai kurang efektif karena rentan terhadap kesalahan manusia (*human error*) dan tidak efisien dalam hal waktu, mengingat proses yang diperlukan seringkali tidak proporsional dengan kompleksitas pertanyaan yang diajukan. Meskipun sebagian besar universitas kini telah memiliki website informasi, konten layanan kemahasiswaan di dalamnya umumnya bersifat statis. Keterbatasan ini menyulitkan pengguna ketika membutuhkan klarifikasi lebih lanjut atau pertanyaan susulan (Fergencs dan Meier, 2021).

Pendekatan tersebut hingga saat ini masih diterapkan di lingkungan Fakultas Sains dan Matematika Universitas Diponegoro (FSM Undip). Saat ini, mahasiswa FSM Undip yang ingin memperoleh informasi akademik diwajibkan untuk mendatangi secara langsung bagian layanan akademik yang berlokasi di Gedung Acintya Prasada. Prosedur tersebut mengharuskan mahasiswa meluangkan waktu dan tenaga untuk memperoleh jawaban atas pertanyaan terkait akademik. Di sisi lain, banyak pertanyaan yang disampaikan bersifat

berulang dengan topik serupa, sehingga meningkatkan beban kerja staf layanan akademik. Kondisi ini mencerminkan bahwa sistem penyediaan informasi akademik di lingkungan FSM Undip belum berjalan secara efektif dan efisien.

Evolusi pesat dalam perkembangan *Artificial Intelligence* (AI) dan *Natural Language Processing* (NLP) telah mempermudah penerapan *chatbot* di lingkungan akademik, sehingga chatbot dapat dimanfaatkan sebagai solusi yang efektif untuk meningkatkan efisiensi dan efektivitas layanan akademik dan kemahasiswaan (Maheswari dan Nagarajan, 2023; Oliveira dan Matos, 2023; Purba dkk., 2024; Salloum dkk., 2024). Maheswari dan Nagarajan (2023) menjelaskan bahwa *chatbot* di bidang pendidikan mampu memberikan bantuan yang lebih terpersonalisasi serta mengurangi ketergantungan pada staf manusia. Lebih lanjut pengembangan *chatbot* berbasis NLP dalam sistem informasi akademik juga telah berhasil memperoleh akurasi hingga 84,5% dalam menjawab pertanyaan pengguna (Purba dkk., 2024), sementara beberapa *chatbot Frequently Asked Question* (FAQ) universitas memperoleh skor F1 mendekati 99,75% (Salloum dkk., 2024). Dengan demikian, penerapan *chatbot* di lingkungan FSM Undip diharapkan dapat mempermudah mahasiswa dalam memperoleh informasi secara fleksibel kapan pun dan di mana pun (Oliveira dan Matos, 2023), sekaligus mengurangi beban kerja staf terhadap pertanyaan yang bersifat repetitif (Maheswari dan Nagarajan, 2023).

Solusi yang diberikan tidak hanya terbatas pada gambaran umum penerapan *chatbot*, tetapi juga perlu berfokus pada pemilihan *framework* yang memiliki performa terbaik dalam implementasi *chatbot*. Performa ini dapat diukur dari akurasi jawaban dan kemampuan *chatbot* dalam menyesuaikan domain informasi tertentu. Selain itu sumber daya komputasi yang dibutuhkan juga perlu menjadi dasar pertimbangan dalam pemilihan *framework* yang ada (Jain, 2025).

Framework yang digunakan dalam pengembangan chatbot sangat beragam. Pada tahap awal, *chatbot* berbasis *Large Language Model* (LLM) umumnya bergantung pada skrip statis atau pengetahuan parametrik yang tertanam di dalam model. Pendekatan ini sering kali menghasilkan respons yang tidak didukung oleh sumber pengetahuan yang valid. Oleh karena itu, dibutuhkan suatu *framework* yang mampu mengambil informasi relevan secara dinamis dari sumber pengetahuan eksternal. Framework *Retrieval-Augmented Generation* (RAG) muncul sebagai solusi yang efektif karena mampu mengurangi kesalahan berupa

jawaban yang tidak sesuai dengan fakta atau tidak didukung oleh sumber pengetahuan yang valid. Hal ini dicapai dengan menghasilkan respons berdasarkan pengetahuan eksternal yang diambil secara *real-time*, sehingga tidak semata-mata mengandalkan memori parametrik pada LLM (Zhu dkk., 2024). Secara kuantitatif, Zhang dkk. (2024) menunjukkan bahwa penerapan *framework* RAG mampu meningkatkan akurasi respons *chatbot* secara signifikan, dengan tingkat kebenaran naik dari 85,5% menjadi 92,7%, serta menghasilkan skor kesamaan semantik sebesar 92% dalam konteks domain pendidikan dan kebijakan. Meskipun demikian, RAG memerlukan sumber daya komputasi yang tinggi karena bergantung pada representasi *high-dimensional embeddings* untuk menyimpan dan mengelola dokumen relevan (Jeong, 2024).

Sebaliknya, untuk menghindari kebutuhan komputasi yang besar, tersedia *framework* RASA sebagai alternatif. *Framework open-source* ini digunakan secara luas dalam pengembangan *chatbot* dan terdiri dari dua komponen utama, yaitu RASA Core untuk manajemen dialog (*dialogue management*) dan RASA NLU untuk pemahaman bahasa alami (*natural language understanding*) (Elsayed dkk., 2024). RASA menggabungkan logika berbasis aturan dengan pendekatan *machine learning*, memberikan keseimbangan antara transparansi, kontrol, dan kemampuan adaptasi (Luise dkk., 2023; Shenoy dkk., 2019). Dari segi efisiensi, penelitian oleh Kavitha dkk. (2024) menunjukkan bahwa sistem berbasis framework RASA mampu mencapai penghematan energi yang signifikan, dengan rata-rata pengurangan konsumsi daya sebesar 87,42% dan 89,98% dibandingkan dengan framework lainnya.

Berdasarkan hasil studi tersebut, dapat diketahui bahwa *framework* RAG dapat menjadi pilihan yang tepat dalam mengembangkan *chatbot* akademik. Namun karena biaya komputasinya yang cukup berat, *framework* RASA yang memerlukan sedikit biaya komputasi juga perlu diujikan untuk mengembangkan *chatbot* akademik. Oleh karena itu penelitian ini akan berfokus pada pengembangan *framework* RAG dan RASA dalam membangun *chatbot* untuk membantu layanan akademik kemahasiswaan di lingkungan FSM Undip.

1.2. Rumusan Masalah

Berdasarkan latar belakang yang telah dijelaskan, rumusan masalah dalam penelitian ini adalah bagaimana performa *framework* RAG dan *framework* RASA dalam pengembangan *chatbot* akademik di lingkungan Fakultas Sains dan Matematika Universitas Diponegoro.

1.3. Tujuan Penelitian

Tujuan dari penelitian ini adalah membandingkan performa dari *framework* RAG dan RASA dalam pengambangan *chatbot* pada studi kasus dokumen Standar Operasional Prosedur (SOP) FSM Undip.

1.4. Manfaat Penelitian

Manfaat dilaksanakannya penelitian ini adalah sebagai berikut.

1. Mengetahui *framework* yang paling tepat untuk digunakan dalam pengembangan *chatbot* akademik di lingkungan Fakultas Sains dan Matematika Universitas Diponegoro.
2. Menghasilkan sebuah *chatbot* yang dapat membantu mahasiswa Fakultas Sains dan Matematika Universitas Diponegoro untuk memperoleh informasi seputar akademik secara mudah dan cepat.
3. Menjadi sumber referensi untuk penelitian selanjutnya dalam pengembangan *chatbot* di bidang akademik.

1.5. Ruang Lingkup

Ruang lingkup bertujuan untuk memberikan batasan terhadap penelitian agar tetap sesuai dengan tujuan penelitian. Ruang lingkup pada penelitian ini adalah sebagai berikut.

1. Penelitian ini berfokus pada perbandingan performa *framework* dalam pengembangan *chatbot*.
2. *Framework* pengembangan yang akan digunakan untuk pengujian adalah RAG dan RASA.
3. Sumber pengetahuan yang akan digunakan dalam pengembangan *chatbot* berasal dari 7 dokumen layanan akademik mahasiswa yang meliputi : SOP pengisian IRS, SOP permohonan izin aktif kuliah setelah cuti, SOP permohonan izin cuti akademik, SOP

permohonan izin keterlambatan pembayaran UKT, SOP legalisir ijazah dan transkrip, SOP pengajuan beasiswa, dan SOP pengajuan proposal kegiatan organisasi.

4. Proses konversi dokumen dari bentuk *flowchart* ke teks langkah-langkah SOP dilakukan secara manual dan saat ini masih terbatas pada jalur proses yang menghasilkan keluaran akhir. Jalur alternatif, seperti kondisi tidak terpenuhi atau kegagalan pada titik percabangan keputusan, belum diuraikan dalam bentuk teks.
5. Pertanyaan yang dibuat untuk mengevaluasi performa dan akurasi masing-masing *framework* dibuat secara manual berdasarkan data pertanyaan yang sering diajukan.
6. Evaluasi terhadap performa masing-masing framework dilakukan dengan menganalisis waktu respon serta question coverage dari *chatbot* dalam memberikan jawaban kepada user. Selain itu performa *chatbot* juga diukur dari pebandingan respon yang diberikan terhadap *ground truth.* Perbandingan respon dilakukan menggunakan metrik ROUGE dan BERTScore.
7. *Ground truth* yang digunakan dalam mengevaluasi akurasi *framework* didapatkan secara langsung dari sumber dokumen yang digunakan dalam penelitian ini.

1.6. Sistematika Penelitian

Penyusunan laporan tugas akhir dengan judul Perbandingan Kinerja *Framework* RASA dan RAG pada *Chatbot* Layanan Kemahasiswaan menggunakan sistematika penulisan sebagai berikut.

BAB I PENDAHULUAN

Bab ini membahas latar belakang, rumusan masalah, tujuan penelitian, manfaat penelitian, ruang lingkup, dan sistematika penulisan laporan tugas akhir Perbandingan Kinerja *Framework* RASA dan RAG pada *Chatbot* Layanan Kemahasiswaan.

BAB II TINJAUAN PUSTAKA

Bab ini membahas teori yang digunakan sebagai acuan dalam penyusunan laporan skripsi, yaitu Penelitian Terkait, Chatbot, Neural Network, Deep Learning, Transformer Model, Larga Language Model, Chunking,

Embedding, Vector Database, Retrieval Augmented Generation, RASA, Evaluation, ROUGE Metric, BERTScore Metric.

BAB III METODOLOGI PENELITIAN

Bab ini membahas rancangan alur penyelesaian masalah berdasarkan teori-teori yang telah dipaparkan. Tahapan ini mencakup penerapan teori, proses penelitian, serta berbagai aktivitas yang akan dilakukan dalam penelitian termasuk di dalamnya proses pengembangan dan pengujian *chatbot*.

BAB IV HASIL DAN PEMBAHASAN

Bab ini akan membahas hasil dari tahapan-tahapan yang telah dilakukan dalam pengembangan *chatbot*. Hasil dari semua tahapan yang telah dilakukan adalah dua jenis *chatbot* berbasis *framework* RAG dan RASA yang kemudian diujikan dengan beberapa set pertanyaan. Performa diukur dengan melihat *response time*, dan membandingkan respon terhadap *ground truth* yang kemudian dinilai dengan metrik ROUGE dan BERTScore.

BAB V PENUTUP

Bab ini membahas kesimpulan dari bab-bab yang telah dibahas sebelumnya serta saran perbaikan atau pengembangan dari penelitian yang sudah dilakukan.

BAB II
TINJAUAN PUSTAKA

Bab ini memberikan penjelasan mengenai teknik, metode, dan konsep yang digunakan dalam melaksanakan penelitian Perbandingan Kinerja *Framework* RASA dan RAG pada *Chatbot* Layanan Kemahasiswaan.

2.1. Penelitian Terkait

Bagian ini membahas penelitian-penelitian terdahulu yang berfokus pada pengembangan *chatbot* di bidang akademis. Daftar penelitian terkait hal tersebut dapat dilihat pada Tabel 2.1.

Tabel 2.1. Daftar Penelitian Terkait

No	Penulis	*Approach*	Metodologi Pengembangan	Pokok Temuan
1	Maheswari dan Nagarajan (2023)	LSTM *with feature extraction*	a. Pengumpulan data b. *Pre-Processing* data c. Ekstraksi fitur dengan BERT d. *Intent classification* dengan LSTM	a. Chatbot pendidikan ini dirancang untuk otomasi layanan akademik seperti menjawab pertanyaan administratif dan pembelajaran. b. Arsitektur BERT + LSTM diterapkan sebagai inti dari sistem pemahaman bahasa alami (NLU) pada chatbot. c. Chatbot yang diusulkan berhasil mencapai akurasi pemahaman niat (*intent*) sebesar 95.60%, yang jauh lebih tinggi dibandingkan model dasar lainnya.
2	Oliveira dan Matos (2023)	LLM *deployed on streamlit*	a. Perencanaan dan perancangan b. Pemilihan *platform* dan *framework* c. Pengolahan dan pengindeksan dokumen d. Pembangunan *user interface.*	a. Penggunaan Streamlit memungkinkan pembuatan antarmuka *chatbot* yang cepat dan fungsional tanpa memerlukan keahlian pengembangan web yang mendalam. b. *Chatbot* yang dihasilkan memiliki tingkat beban komputasi "sedang", sehingga praktis untuk digunakan secara *real-time* oleh mahasiswa di portal *web*. c. *Chatbot* berhasil menyediakan dukungan instan dan terpersonalisasi untuk pertanyaan umum mengenai administrasi, akademik, dan layanan kampus
3	Zhu dkk. (2024)	RAG	a. *Multi agent system construction* b. Inisialisasi dan persiapan data c. Optimasi Iteratif dan Advesarial	a. Penelitian ini berhasil menerapkan paradigma pelatihan *adversarial* khas GAN (*Generative Adversarial Networks*) ke dalam dunia LLM dan sistem *chatbot* RAG. b. *Chatbot/Generator* RAG yang dihasilkan bukan hanya menjadi lebih baik dalam menjawab pertanyaan, tetapi juga menjadi lebih kebal terhadap informasi menyesatkan yang mungkin ada dalam konteks yang diberikan kepadanya.

No	Penulis	*Approach*	Metodologi Pengembangan	Pokok Temuan
				c. *Generator* ATM (ATM7B-Iter2) secara konsisten mengungguli semua model *baseline* (seperti REAR, Self-RAG, RetRobust, RAAT). Peningkatannya signifikan, contohnya pada dataset *Natural Questions*, ATM mencapai akurasi EM 54.21%, mengalahkan *baseline* terdekat (RAAT dengan 48.06%).
4	Zhang dkk. (2024)	RAG	a. Pengembangan k*nowledge* b*ase* b. Implementasi pipeline RAG	a. *Chatbot Ask* NAEP berhasil dibangun sebagai alat bukti konsep (*proof-of-concept*) yang fungsional untuk NCES. b. Arsitektur RAG terbukti efektif dalam mengatasi keterbatasan LLM standar (seperti halusinasi dan informasi yang tidak mutakhir) dengan "melengkapi" LLM dengan dokumen resmi dan terpercaya dari situs web NAEP. c. Komponen RAG (*Retrieval*) terbukti sangat penting. Dibandingkan dengan GPT-4o tanpa *retrieval*, *Ask* NAEP (dengan RAG) meningkatkan persentase jawaban "lulus" untuk *Correctness* dari 87.3% menjadi 92.7%. Ini setara dengan pengurangan 50% dalam respons yang tidak lulus.
5	Elsayed dkk. (2024)	RASA	a. Pengumpulan data b. *Data corpus compilation* c. Pemrosesan data d. Anotasi data e. Arsitektur model f. *Data Integration & Training*	a. Berhasil membangun dua asisten virtual: Genie (untuk GPRO) dan Abu (untuk LIMS). b. Mendukung interaksi bilingual (Inggris dan Spanyol) dengan deteksi bahasa otomatis. c. Mengintegrasikan basis data eksternal dan API untuk respons dinamis dan *real-time*. d. Dua jenis model yang dibuat memiliki skor evaluasi F1-Score 0.937 (Model Genie), dan F1-Score 0.934 (Model Abu).
6	Luise dkk. (2023)	RASA	a. Pengumpulan data b. Identifikasi *intent* dan *entity* c. Pembuatan korpus data d. Augmentasi data e. Pemilihan *pipeline*	a. Berhasil mengadaptasi ITS berbasis tombol (HBPS) menjadi sistem berbasis percakapan menggunakan Rasa. b. Mengidentifikasi 25 intent dan 5 entitas yang relevan dengan domain penyelesaian masalah aljabar. c. Performa model menunjukkan akurasi tinggi dalam klasifikasi niat (*Intent Classification*) dengan *Weighted* F1-Score sebesar 0.965, dan performa yang sangat kuat dalam pengenalan entitas (*Entity Recognition*) dengan *Weighted* F1-Score sebesar 0.989. d. Dalam hal augmentasi data, penambahan *synonym replacement* sebesar 10% berhasil meningkatkan performa, sementara teknik backtranslation justru menurunkannya, kemungkinan besar karena mengaburkan makna konsep matematika.

Berdasarkan tinjauan terhadap penelitian-penelitian terdahulu, dapat dilihat bahwa pengembangan *chatbot* untuk bidang akademik telah dilakukan dengan berbagai pendekatan, baik yang berbasis model *deep learning* klasik, *framework* modern seperti RAG dan RASA, maupun solusi berbasis *Large Language Model* (LLM) yang terintegrasi.

Sebagai contoh, penelitian oleh Maheswari dan Nagarajan (2023) berhasil mengembangkan *chatbot* pendidikan dengan mengadopsi arsitektur *hybrid* BERT + LSTM. Proses pengembangan chatbot dimulai dengan tahapan pengumpulan data percakapan secara manual yang berisi pasangan *query-response* seputar pertanyaan akademik umum. Data tersebut kemudian melalui tahap *pre-processing* seperti penghapusan tanda baca, karakter khusus, *stopwords*, dan penerapan *stemming* untuk menyederhanakan bentuk kata. Selanjutnya, fitur teks diekstraksi menggunakan model BERT untuk memperoleh representasi kontekstual dua arah, dan hasilnya digunakan sebagai masukan pada model LSTM untuk klasifikasi *intent*. Pendekatan bertingkat ini membuat model dapat memahami konteks kalimat dengan lebih baik dan berhasil mencapai tingkat akurasi *intent* sebesar 95.60%. Temuan ini membuktikan bahwa pendekatan *deep learning* tradisional dengan kombinasi antara teknik ekstraksi fitur berbasis *transformer* dan klasifikasi sekuensial masih sangat relevan untuk sistem pemahaman *natural language* di bidang akademik.

Di sisi lain, pendekatan yang lebih modern dengan memanfaatkan LLM dan *framework Retrieval-Augmented Generation* (RAG) menunjukkan hasil yang sangat menjanjikan. Oliveira dan Matos (2023), dalam penelitiannya mengimplementasikan *chatbot* akademik yang berfungsi sebagai asisten informasi dan dukungan layanan kemahasiswaan di kampus. Proses pengembangannya meliputi perencanaan cakupan fungsi *chatbot*, pemilihan *framework* Streamlit yang diintegrasikan dengan LLM GPT-3.5-turbo, dan memilih penyimpanan *embedding* pada *vector database* Pinecone. Mereka menggunakan *DirectoryLoader* dan *RecursiveCharacterTextSplitter* untuk membagi dokumen menjadi potongan-potongan kecil (*chunk*). Kemudian melakukan proses *embedding* dengan menggunakan model all-MiniLM-L6-v2 dan menyimpannya ke Pinecone untuk dapat digunakan dalam pencarian semantik secara efisien. Tahap akhir berupa pembuatan *user-interface* interaktif dengan Streamlit-chat yang memfasilitasi percakapan *real-time* dua arah. Hasilnya menunjukkan bahwa arsitektur RAG berbasis LLM dapat diterapkan secara praktis dalam konteks akademik dan memberikan *user experience* yang baik.

Sementara itu, penelitian oleh Zhang dkk. (2024) dalam pengembangan *chatbot Chatbot* Ask NAEP menerapkan metodologi RAG *pipeline* yang lengkap. Tahapan dimulai dari proses pengumpulan data melalui *web scraping* sekitar 5000 halaman situs resmi, kemudian dilanjutkan dengan proses ekstraksi elemen penting (judul, teks, tabel, dan ringkasan), hingga melakukan pembuatan *structured knowledge database*. Dokumen hasil *scraping* diubah ke dalam bentuk vektor menggunakan model text-embedding-ada-002 dan disimpan pada *database* ChromaDB. Proses *retriveal* dan *re*-ranking dilakukan dengan menggunakan model *cross*-encoder untuk memastikan hasil paling relevan diperoleh sebelum akhirnya dikirim ke model GPT-3.5 atau GPT-4o. Penelitian menggunakan serangkaian *pipeline* ini terbukti mampu meningkatkan akurasi jawaban dari 87.3% menjadi 92.7% dan mengurangi halusinasi secara signifikan.

Pendekatan berbeda namun masih dalam konteks *framework* RAG juga dilakukan oleh Zhu dkk. (2024) melalui pengembangan sistem multi agen adversarial (ATM). Dalam penelitian ini, dua agen LLM yaitu *attacker* dan generator akan dikontruksi untuk saling berkompetisi. *Attacker* bertugas membuat hasil/dkumen palsu yang menyesatkan, sementara *generator* dilatih agar tetap mampu memberikan jawaban benar. Proses ini melibatkan *fine-tuning* dengan data *Question Answer* (QA) RAG dari dataset Natural Questions dan TriviaQA, serta optimasi iteratif dengan *loss function* MITO (perpaduan antara SFT *loss* dan KL-divergence). Pendekatan *advesarial* ini berhasil membuat model RAG menjadi lebih *robust* terhadap data fabrikasi dan menghasilkan akurasi EM sebesar 54.21% pada *dataset* Natural Questions, mengalahkan *baseline* terdekat (RAAT dengan 48.06%).

Sementara kedua pendekatan di atas berfokus pada aspek retrieval dan pembuatan respon, *framework* RASA menunjukkan keunggulannya dalam menangani percakapan yang terstruktur dan *Natural Language Understanding* (NLU). Elsayed dkk. (2024) berhasil membangun dua asisten virtual (Genie dan Abu) di domain bioinformatika menggunakan RASA. Prosesnya dimulai dengan melakukan pengumpulan data melalui kolaborasi dengan ahli, lalu dilanjutkan dengan *compiling* tiga *database* utama yaitu *usage, support,* dan Bioinformatics Q&A. *Pre-proccessing* terhadap data dilakukan dengan melakukan pembersihan noise dan konversi format. Data yang telah siap kemudian dianotasi secara manual untuk *intent* dan *entity* sebelum dilatih menggunakan *pipeline supervised_embeddings* RASA. *Pipeline supervised_embeddings* RASA mencakup tokenisasi, *featurization,* dan klasifikasi intent. Model yang dihasilkan kemudian diperkuat

dengan komponen kustom yaitu deteksi bahasa dan ekstraksi entitas khusus domain, yang membuat model ini memiliki *F1-Score* sebesar 93.7% untuk model Genie dan 93.4% untuk model Abu.

Demikian pula, Luise dkk. (2023) sukses mengadaptasi *Human-Based Problem solving* (HBPS) menjadi *chatbot* edukatif berbasis RASA. Proses pengembangannya dimulai dengan pengumpulan 4400 pesan otentik antara guru dan siswa. Selanjutnya proses dilanjutkan dengan identifikasi *intent* dan *entity* serta pembuatan korpus pelatihan *multilingual* yang diterjemahkan dan dianotasi secara manual. Untuk meningkatkan keragaman data, dilakukan augmentasi berbasis *synonym replacement* hingga menghasilkan 16081 sampel. Kemudian dilakukan eksperimen konfigurasi *pipeline* NLU, dan hasil terbaik diperoleh dengan penggunaan spaCy en_core_web_lg serta DIETClassifier untuk deteksi *intent* dan pengenalan *entity*. Pendekatan ini berhasil menghasilkan sebuah model *chatbot* dengan *Weighted F1-Score* sebesar 96.5% untuk *intent* dan 98.9% untuk *entity*. Penelitian ini menegaskan kekuatan RASA dalam menangani alur percakapan yang dinamis dan kompleks, serta menyediakan panduan metodologis untuk adaptasi sistem tutoring ke dalam bentuk *natural language interaction*.

Berdasarkan peta penelitian ini, terlihat jelas bahwa RAG dan RASA menunjukkan keunggulan jika dibandingkan dengan pendekatan deep learning klasik. RAG unggul dalam menyediakan jawaban berbasis pengetahuan eksternal yang akurat, sementara RASA tangguh dalam memahami maksud pengguna dan mengelola dialog yang terstruktur. Oleh karena itu, penelitian ini dilakukan untuk mengembangkan dan membandingkan performa kedua *framework* tersebut secara langsung dalam konteks yang spesifik, yaitu membangun chatbot layanan kemahasiswaan untuk lingkungan Fakultas Sains dan Matematika Universitas Diponegoro. Hasil perbandingan ini diharapkan dapat memberikan panduan empiris dalam memilih pendekatan terbaik untuk kebutuhan layanan mahasiswa yang dinamis dan informatif, sekaligus mengevaluasi kelebihan dan tantangan dari masing-masing pendekatan dalam ekosistem layanan akademik yang nyata.

2.2. *Chatbot Model*

Chatbot adalah sebuah perangkat lunak berbasis AI yang didesain untuk dapat berinteraksi dengan manusia secara khusus biasanya melakukan aksi tanya jawab (*question answering*) dalam bentuk teks percakapan melalui *Natural Language Processing* (NLP) dan

juga teknik *machine learning*. Secara konseptual, chatbot menggabungkan beberapa teknologi dasar, seperti Natural Language Understanding (NLU), deep learning, dan arsitektur berbasis transformer, yang memungkinkan sistem untuk memberikan respons secara real-time dan kontekstual pada berbagai domain, termasuk layanan kesehatan, layanan pelanggan, dan pendidikan. *Chatbot* dapat diimplementasikan dalam beberapa jenis pendekatan, diantaranya adalah dengan sistem berbasis aturan (*rule-based*), pengambilan informasi (*retriveal-based*), atau sistem generatif (Rosas dan Rodriguez, 2024). Sebagai contoh, *framework* RASA merupakan salah satu implementasi chatbot berbasis aturan (*rule-based*) yang mengandalkan pola dan *intent* yang telah didefinisikan sebelumnya, sedangkan pendekatan generatif dapat diimplementasikan melalui *framework* RAG yang menghasilkan respons secara dinamis dengan memanfaatkan model bahasa dan sumber pengetahuan eksternal.

Relevansi penggunaan *chatbot* meningkat seiring dengan bertambahnya *platform* digital dan adanya kebutuhan akan sebuah sistem yang mampu menjawab dan membantu manusia dalam berbagai bidang dengan tampilan *user-friendly* dan juga komputasi yang efisien. Sehingga saat ini, hampir seluruh bidang dan perusahaan diseluruh dunia berlomba-lomba menciptakan dan memanfaatkan *chatbot* dengan baik terutama dalam bidang kesehatan, pendidikan, perbankan, dan layanan pelanggan (Szmurlo dan Akhtar, 2024).

2.3. *Natural Language Processing*

Natural Language Processing (NLP) merupakan cabang dari ilmu komputer, kecerdasan buatan, dan linguistik komputasional yang berfokus pada bagaimana sistem komputer dapat mengolah, menganalisis, dan memanipulasi bahasa manusia secara otomatis. Tujuan utama NLP adalah memungkinkan mesin untuk memahami serta menghasilkan bahasa alami dalam bentuk teks maupun ujaran, sehingga dapat digunakan untuk berbagai tugas seperti tokenisasi, analisis sintaksis, ekstraksi informasi, dan penerjemahan bahasa. Dalam perkembangannya, NLP menggabungkan pendekatan linguistik dengan metode statistik dan pembelajaran mesin untuk merepresentasikan bahasa manusia ke dalam bentuk yang dapat diproses oleh komputer. Oleh karena itu, NLP menjadi fondasi utama dalam pengembangan sistem berbasis bahasa, termasuk sistem pencarian informasi, *chatbot*, dan aplikasi kecerdasan buatan lainnya yang berinteraksi langsung dengan pengguna (Jurafsky & Martin, 2026).

Seiring dengan tujuan tersebut, salah satu aspek penting dalam NLP adalah *Natural Language Understanding* (NLU) yang berperan dalam memahami makna, konteks, dan maksud dari bahasa alami yang digunakan oleh manusia. Sekain berfokus pada struktur permukaan kalimat, NLU juga berfokus pada interpretasi semantik, relasi antar kata, serta konteks percakapan secara keseluruhan. Dengan adanya NLU, sistem NLP diharapkan mampu menangkap informasi implisit seperti intensi pengguna, ambiguitas makna, dan hubungan wacana, sehingga respon yang dihasilkan menjadi lebih relevan dan kontekstual. Dalam praktiknya, NLU menjadi komponen kunci dalam aplikasi yang memerlukan pemahaman bahasa tingkat tinggi, seperti sistem tanya jawab, analisis sentimen, dan agen percakapan cerdas, serta berperan sebagai jembatan konseptual menuju pendekatan lanjutan berbasis *deep learning* dalam pemrosesan bahasa alami (Allen, 2016).

2.4. *Deep Learning*

Deep Learning dapat diartikan sebagai sebuah bagian dari *machine learning* yang berfokus pada penggunaan *neural network* untuk dapat memodelkan abstraksi data tingkat tinggi melalui lapisan neuron dalam jumlah banyak. Semua lapisan yang digunakan terdiri dari struktur kompleks dan transformasi non-linear yang membuat sistem dapat belajar dari beragam sumber data. Kemampuan lapisan untuk belajar dari data dicapai melalui penggunaan lapisan non-linier yang berfungsi mengekstraksi dan mentransformasi fitur-fitur input. Dengan mekanisme ini, sistem dapat membangun representasi hierarkis dari data, di mana fitur tingkat tinggi diperoleh dari kombinasi fitur tingkat rendah (Goodfellow dkk., 2016; Hao dkk., 2016).

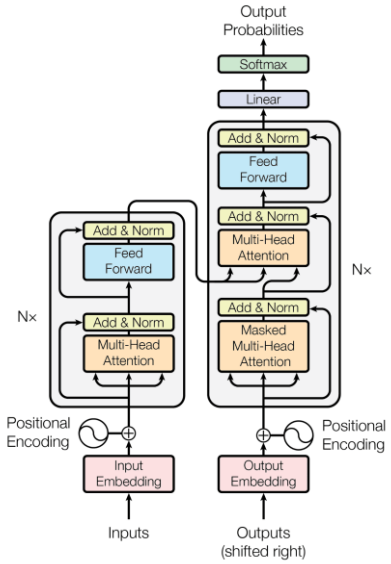
Era baru bagi *deep learning* dalam pemrosesan teks (NLP) ditandai dengan munculnya *transformer model* yang diprakarsai oleh Google pada tahun 2017 dalam sebuah artikel yang berjudul "*Attention is All You Need"* (Vaswani dkk., 2017). Jika sebelumnya *natural language processing* sangat populer dengan model terbaik kala itu yang adalah LSTM, kehadiran *transformer model* mengubah semuanya. Apalagi di kala itu *transformer model* di awal kemunculannya sudah digadang dapat melampaui kemampuan manusia pada umumnya (dalam kontes SQuAD2.0 di *Stanford University*) (Zhang dan Shafiq, 2024).

2.5. *Transformer Model*

Transformer model adalah sebuah model yang terdiri dari serangkaian lapisan *self-attention* yang berulang untuk menemukan hubungan antar token, gabungan informasi

(*TokenMix*), dan *Feed-Forward Network* (FFN) berikutnya untuk menangkap pola-pola kompleks (Lee dkk., 2023). Seperti yang telah sempat dibahas pada bagian sebelumnya, *transformer model* diusulkan oleh *Google Brain Team* yang tujuan awalnya adalah untuk menyederhanakan model RNN atau CNN yang kala itu cukup kompleks meskipun memiliki kinerja yang baik berkat adanya mekanisme *attention*. *Google Team* mengusulkan sebuah arsitektur jaringan sederhana baru yang sepenuhnya berbasis mekanisme *attention* dan menghilangkan *reccurence* serta *convolutions* sepenuhnya (Vaswani dkk., 2017). Jenis mekanisme *attention* yang dimaksudkan disini adalah *self-attention*. *Self-attention* atau biasa juga sering disebut dengan *intra-attention* adalah sebuah mekanisme atensi yang menghubungkan posisi-posisi berbeda dari suatu urutan tunggal untuk menghitung representasi dari urutan tersebut. Pada *transformer model*, sumber atau target data diinteraksikan dengan dirinya sendiri dimulai dari tingkat permukaan hingga seterusnya secara sekuensial hal inilah juga yang menjadi salah satu alasan metode yang digunakan disebut sebagai *self-attention* (Liu & Xu, 2023; Rogers dkk., 2022).

Struktur dasar dari *transformer model* yang ada dapat dilihat pada Gambar 2.1. yang secara jelas memberikan gambaran *self-attention* bertingkat dan lapisan *decoder* dan *encoder* yang terhubung sepenuhnya.



Gambar 2.1. The Transformer – Model Architecture (Vaswani dkk., 2017)

2.6. *Large Language Model*

Large Language Model (LLM) muncul dan terinisiasi berkat adanya NLP dan juga *transformer model* yang kemudian memungkinkan adanya *chatbot*, *virtual assistant*, dan berbagai macam aplikasi lainnya. Model ini seringkali terdiri dari miliaran parameter yang memanfaatkan arsitektur *transformer* dan mekanisme *attention* untuk dapat memproses secara efektif dan menangani dependensi jangka panjang jika dibandingkan dengan pendekatan sebelumnya RNN. Terlihat dari sejak diperkenalkannya Llama, perkembangan arsitektur LLm telah terkonfigurasi secara signifikan, terutama berfokus pada *transformer-based decoder only architecture*. Keberadaan LLM saat ini tentunya didorong juga oleh adanya kemajuan memori *bandwith* tinggi, *high performance accelerator*, arsitektur yang telah dioptimasi dan berbagai macam faktor lainnya yang berhasil memunculkan model dengan miliaran parameter (Kim dkk., 2025). Beberapa contoh LLM yang populer saat ini dan karakteristiknya dapat dilihat pada Tabel 2.2.

Tabel 2.2. Model Populer dan Karakteristiknya (Kim dkk., 2025)

No	*Model*	*Release*	*Provider*	*Parameter Size*	*Open Source*	*Key Features*
1	GPT-4	2023.03	OpenAI	*Undisclosed*	*No*	*Multimodal support, Expanded context window,* RLHF
2	GPT-3	2020.05	OpenAI	175B	*No*	*Few-shot learning, Wide applicability*
3	Llama 3	2024.04	Meta	8B, 70B	*Yes*	*Expanded context window, Multilingual support*
4	Llama 2	2023.07	Meta	7B, 13B, 70B	*Yes*	GQA, SFT, RLHF
5	Gemma 2	2024.06	Google	2B, 9B, 27B	*Yes*	*Logit soft-capping, Alternating local and global attention*
6	Gemma 1	2024.02	Google	2B, 7B	*Yes*	MQA (2B), SFT, RLHF
7	Qwen 2	2024.06	Alibaba Cloud	0.5B, 1.5B, 7B, 57B, 72B	*Yes*	GQA, YaRN*, Multilingual support*
8	Phi-3	2024.04	Microsoft	3.8B, 7B, 14B	*Yes*	Llama 2 *tokenizer,* LongRoPE
9	RoBERTa	2019.07	Facebook	125M, 355M	*Yes*	*Dynamic masking, Eliminate* NSP
10	BERT	2018.10	Google	110M, 340M	*Yes*	*Bidirectional contextual understanding,* MLM, NSP
11	Mistral	2023.12	Mistral AI	7B (2023.09), 8x7B	*Yes*	SWE, SMoE

2.7. *Chunking Process*

Chunking atau yang biasa juga dikenal dengan *phrase chunking* (*shallow parsing*) adalah sebuah proses untuk mensegmentasi kalimat menjadi serangkaian konstituen atau *chunk*. *Chunk* dapat didefinisikan sebagai rangkaian kata-kata yang berdekatan dan

dikelompokkan berdasarkan sifat linguistik. Hal ini menjadikan proses *chunking* menjadi proses fundamental dalam tahapan *preprocessing* teks pada NLP sehingga nantinya teks dapat diketahui arti secara semantik maupun sintaksis (Ehsan dkk., 2022). *Chunking* muncul sebagai alternatif dari *full parsing* dengan alasan utama karena proses *full chunking* dan penyajian hasil teks secara efisien tanpa analisis mendalam sangat sulit untuk dilakukan.

Meskipun teknik *chunking* konvensional sudah cukup efektif, ternyata masih ditemui kesulitan ketika harus berhadapan dengan data yang kompleks, hiearkikal, dan juga data tertentu yang spesifik terhadap domain. Teknik *chunking* konvensional atau yang biasa disebut sebagai *flat chunking* seringkali masih gagal mempertahankan hubungan semantik dan struktur bersarang pada data, yang akhirnya menyebabkan segmen terfragmentasi atau bahkan ambigu secara kontekstual (Yin dkk., 2025). Oleh karena itu, kebutuhan akan adanya teknik lanjutan menjadi urgensi tersendiri. Disinilah teknik *recursive splitting* dapat menjadi salah satu solusi yang dapat dilakukan untuk mengatasi limitasi tersebut dengan menyediakan integrasi semantik yang lebih baik.

Recursive character splitting meningkatkan kemampuan *chunking* dengan membagi teks secara rekursif pada tingkat karakter berdasarkan pemisah (*delimiter*), mengaktifkan *hierarchical chunking* yang mempertahankan struktur bertingkat dan koherensi semantik. Pendekatan secara hierarkis ini memberikan pendekatan yang lebih rinci, relevan secara kontekstual, dan efisien dalam menjawab semua task yang berhubungan dengan sumarisasi dan *question answering* (Yang dkk., 2020). Rumus dasar yang digunakan dalam proses *chunking* adalah sebagai berikut (Mungai dan Huang, 2017).

$$\sum_{k=1}^{n} A_k = BCD^T \quad (2.1)$$

Keterangan :

n : jumlah total matriks yang dijumlahkan

A_k : matriks *slice* atau matriks data (untuk dimensi tertentu) ke-k dalam penjumlahan

B : matriks faktor pertama yang berisi informasi fitur laten dari data

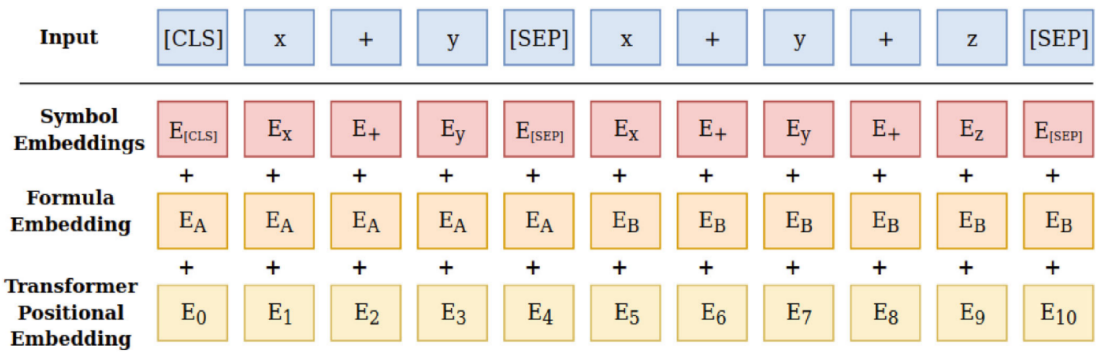
C : matriks inti yang memodulasi interaksi antar matriks B dan D

D^T : *transpose* dari matriks faktor ketiga yang menangkap informasi terkompresi dari data

2.8. *Embedding Process*

Text embedding adalah representasi matematis dari sebuah teks pada ruang vektor berdimensi tinggi yang mana semua elemen mirip diletakkan pada titik-titik yang berdekatan satu sama lain. *Text embedding* telah menjembatani dan mentransformasi bagaimana mesin memproses dan memahami bahasa manusia. Dengan mengkonversi kata, kalimat, dan dokumen menjadi sebuah representasi kontinu dari vector, *embedding* memungkinkan algoritma untuk bisa menangkap hubungan semantik dan sintaksis dari *natural language*. Inilah yang kemudian menjadi dasar utama pada pengembangan sistem NLP modern dan salah satunya *conversational agents* seperti *chatbot* (Giabelli, 2022).

Pendekatan awal seperti *one-hot encoding* dan *bag of words* mengalami permasalahan sparsial dan juga kurangnya kemampuan menganalisis secara semantik. Selanjutnya dari masalah itu muncul representasi lain misalnya *word2vec, glove* yang memberikan kemajuan signifikan, dimana model akhirnya mampu mempelajari kesamaan kata dari sebuah korpus. Sejak itu perkembangan *embedding* terus mengarah pada model yang terkontekstualisasi, dimana makna sebuah kata ditentukan secara dinamis oleh konteksnya dalam kalimat. Model berbasis *transformer model* seperti BERT dan semua model yang dibuat dengannya menetapkan standar baru dengan menangkap polisemi dan hubungan semantik yang lebih halus (Jiang dkk., 2023). Hal ini tentu penting dalam pengembangan *chatbot*. Dalam konteks pengembangan *chatbot*, *embedding* memfasilitasi model untuk melakukan *mapping* permintaan pengguna (*user query*) kepada makna yang dituju, pengambilan informasi yang relevan, dan pembuatan respons yang sesuai secara kontekstual, sehingga memungkinkan interaksi antara pengguna dan sistem berjalan dengan efektif dan lancar.



Gambar 2.2. Contoh Stuktur *Embedding* berbasis Model BERT (Dadure dkk., 2022)

Text embedding berdasar pada perhitungan vektorisasi yang diterapkan pada semua sumber data yang biasanya berada dalam format yang tidak terstuktur. Salah satu perhitungan yang berkaitan erat dengan *embedding* adalah *cosine similarity* yang berfungsi menghitung kedekatan antar dua vektor (Dadure dkk., 2022). Berikut ini disajikan representasi matematis dari *text embedding* dan *cosine similarity*.

1. Definisi Matematis

   $f: X \rightarrow R^d$ .....(2.2)

   Keterangan :

   X : ruang input bertipe diskrit

   R^d : ruang output bertipe vektor kontinu yang isinya adalah bilangan riil

2. Representasi Vector

   $\vec{w} = (w_1, w_2, \ldots, w_d)$ where $w_i \in R$ .....(2.3)

   Keterangan :

   w_i : komponen individu ke-i dari vektor w

   d : dimensi dari ruang *embedding*

3. Cosine Similarity

   $\text{similarity}(\vec{a}, \vec{b}) = \cos(\theta) = \frac{\vec{a} \cdot \vec{b}}{|\vec{a}||\vec{b}|} = \frac{\sum_{i=1}^{n} a_i b_i}{\sqrt{\sum_{i=1}^{n} a_i^2} \sqrt{\sum_{i=1}^{n} b_i^2}}$ .....(2.4)

   Keterangan :

   $\vec{a}, \vec{b}$: dua vektor hasil *embeedding*

   $|\vec{a}||\vec{b}|$: magnitudo dari masing-masing vektor yang kemudian dihitung sebagai

   $$\sqrt{\sum_{i=1}^{n} a_i^2} \sqrt{\sum_{i=1}^{n} b_i^2}$$

2.9. *Vector Database*

Vector database didesain sebagai sebuah sistem yang berguna untuk menyimpan, mengambil, dan mengelola vektor berdimensi tinggi secara efisien (Kang dkk., 2025). Biasanya vektor ini merepresentasikan suatu data yang kompleks dan tidak terstuktur seperti teks, gambar, dan media lainnya. *Vector database* melakukan *encode* pada data ke dalam *high-dimensional embeddings* sehingga memungkinkan pencarian data berbasis semantik

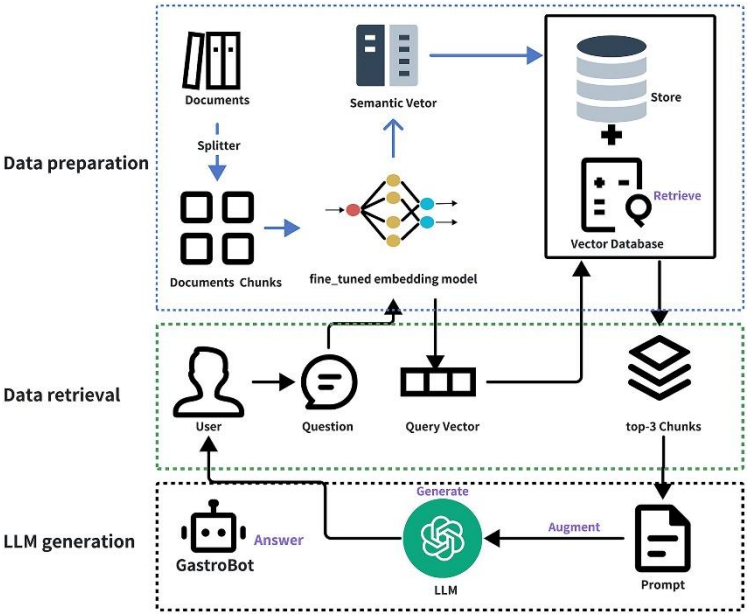
untuk dapat meningkatkan relevansi pencarian dan *user experience*. Akibatnya *vector database* memiliki peran yang penting dalam AI dan *Natural Language Processing* (NLP).

Salah satu contoh dari *vector database* yang dapat digunakan adalah Chroma DB. Chroma DB adalah sebuah *vector database* yang secara efisien menyimpan dan mengambil hasil dari *embedding* dokumen yang tervektorisasi. Chroma DB secara efisien menyimpan dan memungkinkan pencarian data (misalnya, PDF) dalam skala besar. Penggunaan Chroma DB juga dapat diintegrasikan dengan berbagai metode pengambilan informasi, misalnya *multi-query generation* dan *dynamic query-handling capabilities*. Chroma DB memanfaatkan LangChain untuk mengatur interaksi antara model bahasa dan database vektor, memfasilitasi pemrosesan dan penyematan informasi. Dengan sistem penyimpanan tersebut, pengambilan informasi yang relevan dan akurasi kontekstual dari jawaban yang dihasilkan oleh sistem pencarian dapat ditingkatkan (Sonkar dkk., 2025).

2.10. *Retrieval-Augmented Generation*

Retrieval-Augmented Generation atau yang bisa disebut sebagai RAG adalah sebuah framework yang didesain untuk meningkatkan performa dari *Large Language Model* (LLM) dengan mengintegrasikan pengetahuan tambahan (eksternal) ke dalam proses pembuatan hasil (Rothman, 2024). *Large Language Model* yang digunakan pada *chatbot* dilatihkan pada data dalam jumlah besar namun tidak dapat secara langsung memahami konteks atau informasi yang ada dalam data yang berasal dari sumber privat. Hal ini mengakibatkan model yang digunakan pada *chatbot* sering mengalami halusinasi. Oleh karena itu pemberian input/*knowledge* tambahan dengan pendekatan RAG muncul sebagai solusi untuk mengurangi terjadinya halusinasi dalam model yang digunakan. Misalnya saja dalam konteks *customer service*, dengan memberikan data pembelian yang telah dilakukan, pengguna dapat melakukan tanya jawab dengan model yang ada mengenai pembelian pengguna. Tanpa adanya konsep penambahan *knowledge*, pengguna hanya dapat berdiskusi hal lain yang general (Church dkk., 2024).

Beberapa tahapan yang harus dilakukan dalam *framework* RAG terutama dalam konteks pembuatan *chatbot* dapat dilihat pada Gambar 2.3.



Gambar 2.3. Tahapan-Tahapan pada *Framework* RAG (Zhou dkk., 2024)

Tahapan yang harus dilakukan pada *framework* RAG dimulai dari tahapan *data preparation*. Tahapan *data preparation* sendiri dimulai dari tahapan pengumpulan dokumen. Setelah dokumen atau pustaka informasi berhasil dikumpulkan, data yang ada di dalamnya dipecah menjadi unit-unit yang lebih kecil (biasa disebut sebagai *chunk*). Proses mengubah data menjadi unit-unit yang lebih kecil melibatkan fungsi *recursive splitter* sebagai salah satu pendekatan dalam proses *chunking*. Data yang telah dipecah menjadi *chunk* selanjutnya akan diproses menggunakan *embedding model* untuk diubah ke dalam representasi vektor semantik. Namun sebelum itu, *embedding model* yang akan digunakan dilatihkan terlebih dahulu terhadap data yang akan divektorisasi. Biasanya pelatihan ini dilakukan dengan menggunakan pasangan pertanyaan dan jawaban (Q&A) yang dihasilkan secara otomatis, meskipun tidak menutup kemungkinan dibuat secara manual. Selanjutnya model *embedding* yang telah dilatih digunakan untuk melakukan proses vektorisasi pada data yang berhasil ter-*chunk*. Sehingga hasil akhir pada tahapan ini adalah sebuah data representasi vektor dari dokumen-dokumen yang ada. Data representasi vektor ini disimpan ke dalam sebuah *vector database* yang nantinya akan diakses pada tahapan *data retrieval*.

Pada tahapan selanjutnya, model *retrieval* yang dibangun dapat dinyatakan telah siap. Tahapan ini dimulai dari saat *user* melakukan *querying* atau mudahnya mengajukan pertanyaan. Pertanyaan yang diajukan oleh *user* selanjutnya akan diubah menjadi representasi vektor semantik menggunakan *embedding model* yang sama dengan model yang digunakan untuk memproses data *knowledge*. Setelah diperoleh representasi *query* dalam

bentuk vektor, vektor yang didapat akan dibandingkan dengan data vektor yang ada dalam database untuk mencari kesamaan atau kemiripan konteks. Biasanya proses ini dilakukan dengan perhitungan *cosine similarity*. Dari proses ini diperoleh beberapa *retrieved chunk* yang diduga memiliki kesamaan konteks/informasi yang diperlukan oleh *user* dari pertanyaan yang diajukan. *Chunk* sebanyak k akan diambil dan disimpan sementara sebelum nantinya akan digunakan pada tahapan pembuatan hasil (*generation*).

Tahapan yang ketiga yaitu *llm generation* menjadi bagian yang akan memberikan luaran kepada *user*. Tahapannya dimulai ketika proses *retriveal* telah selesai dilakukan dan berhasil diperoleh beberapa *chunk* hasil dari proses tersebut. *Chunk* yang diperoleh akan disematkan pada sebuah susunan *prompt*. *Prompt* inilah yang kemudian akan diberikan kepada model LLM yang telah dipilih untuk kemudian diaugmentasi jawabannya sehingga memungkinkan jawaban yang lebih interaktif atau kaya konteks namun tetap berdasar pada konteks yang terdapat dari k jumlah *chunks* yang disematkan pada *prompt*. Hasil buatan dari LLM inilah yang kemudian diberikan kembali kepada *interface chatbot* yang digunakan untuk selanjutnya ditampilkan kepada pengguna. Tahapan-tahapan inilah yang menjadi proses yang fundamental dalam pembuatan *chatbot* berbasis *framework* RAG (Zhou dkk., 2024).

2.11. RASA *Framework*

RASA adalah sebuah *open-source framework* yang didesain untuk membangun *chatbot* konversational. RASA menyediakan alat untuk mengkostumisasi *Natural Language Understanding* (NLU) dan manajemen dialog sehingga pengembang dapat membuat sebuah *chatbot* yang berorientasi pada tugas dan paham terhadap konteks percakapan. *Framework* ini sangat mudah dikostumisasi dan mampu menerima berbagai algoritma atau model *machine learning* untuk fungsionalitasnya (Kong dan Wang, 2021).

Berikut ini adalah dua komponen yang diperlukan untuk dapat membangun *chatbot* berbasis RASA (Elsayed dkk., 2024).

1. NLU *Training Data*

   RASA menggunakan format YAML sebagai format utama dan terukur untuk mengatur semua jenis data pelatihan yang mencakup 3 hal yaitu :

   a. Komponen NLU untuk melatih *chatbot* dalam mengurai dan memahami *input*

File ini berisi contoh dari *input* yang dimasukkan oleh *user*. Contoh *input* dikategorikan dan diklasifikasikan dengan menggunakan *intent*, yang merepresentasikan tujuan dari pengguna.

b. Alur naratif yang terdiri dari *rules* dan *stories*

Rules mendefinisikan potongan dari konversasi yang harus selalu mengikuti aturan yang ada. Sementara *stories* mendefinisikan representasi dan urutan percakapan konversasi yang lebih kompleks, yang memungkinkan *chatbot* untuk memahami dan beradaptasi dengan interaksi dari *user*.

c. *File domain* yang berfungsi sebagai aturan operasional bagi *chatbot*

File ini berisi semua kemungkinan user intent, bot responses, entities, dan actions yang beresensi memetakan setiap interaksi yang mampu ditangani oleh chatbot. Pengaturan ini memungkinkan chatbot beroperasi sesuai dengan cakupan yaitu dengan memberikan respon dan aksi yang terstuktur dalam konteks konversasi.

2. *Dialogue Management*

Komponen manajemen dialog RASA terdiri dari *dialogue policies*, *action selection*, dan juga *custom actions*. *Custom actions* digunakan untuk membuat *chatbot* dapat membaca berbagai sumber data lainnya yang mungkin tersimpan dalam file XML, basis data, dan bahkan API. *Custom actions* memungkinkan pembuatan respon dinamis untuk menghasilkan respon yang koheren, relevan, dan disesuaikan dengan permintaan pengguna.

2.12. *Evaluation Metrices*

Maraknya penggunaan *chatbot* di berbagai bidang, mulai dari layanan pelanggan, layanan kesehatan, dan bahkan di bidang pendidikan, menjadikan evaluasi kualitasnya sebagai perhatian penting. *Chatbot* diharapkan mampu memberikan informasi yang akurat meskipun tetap harus mempertahankan suasana percakapan yang menarik. Seiring dengan meningkatnya kompleksitas dari *chatbot*, evaluasi sistematis menjadi penting untuk meningkatkan kepuasan pengguna, peningkatan sistem, dan penerapan yang bertanggungjawab (Deutsch dan Roth, 2021).

Automated metrics telah menjadi alat standar untuk mengevaluasi respon dari *chatbot* yang akan menghasilkan tolak ukur yang *scalable* dan mudah untuk dipahami. Contoh metrik yang digunakan adalah ROUGE dan BERTScore, yang berfungsi untuk menilai

akurasi dan relevansi semantik dari respons *chatbot*. ROUGE memberikan gambaran mengenai tingkat kesamaan leksikal (*lexical overlap*), yang bermanfaat dalam menilai kesesuaian konteks respons terhadap sumber referensi. Sementara itu, BERTScore memanfaatkan *embedding* secara kontekstual untuk dapat menangkap kesamaan semantik yang lebih dalam (Kaster dkk., 2021).

2.12.1. ROUGE *Metric*

Recall-Oriented Understudy for Gisting Evaluation atau yang dapat disingkat dengan sebutan ROUGE adalah salah satu contoh dari metrik evaluasi otomatis yang didesain untuk mengevaluasi kualitas dari teks yang dihasilkan oleh mesin dengan membandingkannya ke satu atau lebih teks/jawaban referensi (*ground truth*) . Prinsip utama yang juga sekaligus menjadi daya tarik utama dari metrik ROUGE adalah kesederhanaan, reproduktivitas, dan kemampuannya menjadi *proxy* untuk justifikasi manusia dengan mengukur adanya *overlap* dari n-gram (urutan kata yang berdekatan), *sequences*, atau pasangan kata antara teks hasil dan jawaban referensi . Oleh karenanya metrik ROUGE menjadi salah satu landasan dalam evaluasi sistem berbasis *Natural Language Processing* (NLP) yang biasanya terkait dengan *text summarization*, *machine translation*, dan *text generation* (Zhu dkk., 2023).

Terdapat 2 jenis metrik ROUGE yang paling umum digunakan yaitu ROUGE-N (berbasis *n-grams overlap*), dan ROUGE-L (berbasis *longest common subsequence*). Berikut ini adalah definisi dan formula sederhana dari dua ROUGE *metrics* tersebut.

1. ROUGE-N

   Mengukur *overlap n-grams* diantara *candidate* dan *reference text* (*ground truth*). Rumusnya adalah sebagai berikut.

   a. $Recall = \frac{\text{Jumlah overlapping n-grams}}{\text{Total n-grams pada reference}}$ .....(2.5)

      Keterangan :

      Jumlah *overlapping* n-grams : jumlah n-gram yang muncul di kalimat kandidat dan juga di kalimat referensi.

      Total n-grams pada *reference* : jumlah total n-gram unik atau n-gram pada kalimat referensi.

      Recall pada mengukur sejauh mana n-gram yang terdapat dalam jawaban referensi berhasil dicakup oleh respon *chatbot*. Nilai *recall* yang tinggi menunjukkan bahwa

sebagian besar unsur teks penting dalam jawaban referensi telah muncul dalam respon, sehingga mencerminkan kelengkapan informasi yang dihasilkan.

b. $Precision = \frac{\text{Jumlah overlapping n-grams}}{\text{Total n-grams pada candidate}}$ ..... (2.6)

Keterangan :

Jumlah *overlapping* n-grams : jumlah n-gram yang muncul di kalimat kandidat dan juga di kalimat referensi.

Total n-grams pada *candidate* : jumlah total n-gram pada kalimat kandidat.

Precision mengukur proporsi n-gram dalam respon *chatbot* yang juga terdapat pada jawaban referensi. Nilai *precision* yang tinggi mengindikasikan bahwa respon *chatbot* mengandung sedikit n-gram yang tidak relevan, sehingga mencerminkan tingkat ketepatan dalam pemilihan kata dan frasa.

c. $F1\text{-}score = \frac{2\times(\text{Precision}\times\text{Recall})}{\text{Precision}+\text{Recall}}$ ..... (2.7)

Keterangan :

F1-Score merupakan nilai kombinasi antara *recall* dan precision yang digunakan untuk menilai keseimbangan antara kelengkapan dan ketepatan respon *chatbot* berdasarkan kesamaan n-gram.

2. ROUGE-L

Mengukur *Longest Common Subsequence* (LCS) diantara *candidate* dan *reference text*. Komputasi dan rumus perhitungannya adalah sebagai berikut.

a. Menggunakan pendekatan *dynamic programming* untuk mencari nilai LCS

b. $Recall = \frac{\text{Panjang LCS}}{\text{Panjang Reference}}$ ..... (2.8)

Keterangan :

Panjang LCS : jumlah kata dalam *longet common subsequence* antara kalimat kandidat dan referensi.

Panjang *Reference* : jumlah total kata dalam kalimat referensi.

Recall mengukur banyak urutan kata dalam jawaban referensi yang berhasil direpresentasikan dalam respon *chatbot* melalui subsekuens terpanjang yang sama. Nilai *recall* yang tinggi menunjukkan bahwa struktur informasi utama dalam jawaban referensi telah tercakup dalam respon, meskipun tidak seluruh kata muncul secara identik atau berurutan secara ketat.

c. $Precision = \frac{\text{Panjang LCS}}{\text{Panjang Candidate}}$ .....(2.9)

Keterangan :

Panjang LCS : jumlah kata dalam *longet common subsequence* antara kalimat kandidat dan referensi.

Panjang *Candidate*: jumlah total kata dalam kalimat kandidat

Precision mengukur proporsi urutan kata dalam respon *chatbot* yang sesuai dengan subsekuens terpanjang dari jawaban referensi. Nilai *precision* yang tinggi mencerminkan bahwa respon *chatbot* memiliki keselarasan struktur kalimat yang baik dan minim penyimpangan dari urutan informasi yang diharapkan.

d. *F1-score* merupakan nilai kombinasi antara *recall* dan *precision* yang digunakan untuk menilai keseimbangan antara kelengkapan dan kesesuaian struktur respon *chatbot* terhadap jawaban referensi.

Meskipun demikian, ketergantungan pada *lexical overlap* dan kurangnya kemampuan dari ROUGE *metric* untuk menangkap hubungan semantik menjadi suatu permasalahan jika model ini digunakan untuk mengevaluasi hasil luaran yang lebih abstrak dan kreatif. Akhirnya limitasi ini nantinya berhasil diatasi dengan alternatif metrik lain yang pengembangannya berangkat dari ROUGE *metric*, salah satu contohnya adalah BERTScore yang menggunakan pendekatan *embedding* (Sellam dkk., 2020).

2.12.2. BERTScore *Metric*

BERTScore adalah sebuah metrik evaluasi semantik yang didesain untuk mengukur similiaritas antara *candidate text* (misalnya hasil *chatbot generated response*) dan teks referensi. Berbeda dengan metrik tradisional seperti BLEU atau ROUGE yang bergantung pada kecocokan token atau *overlap n-grams*, BERTScore memanfaatkan *contextual embedding* dari model *transformer* seperti BERT untuk dapat menangkap hubungan semantik yang lebih dalam pada teks. BERTScore bekerja dengan cara merepresentasikan setiap token baik pada *candidate* dan *reference* sebagai suatu *contextual embedding* berdimensi tinggi yang dihasilkan dari model *pre-trained* BERT. Metrik tersebut kemudian menghitung *cosine similarity* dari *embedding* yang dihasilkan, menghasilkan *similarity matrix* yang menggambarkan hubungan semantis pada tingkat token. Berbeda dengan *token matching*, pendekatan ini memungkinkan BERTScore mengenali parafrase, sinonim, dan ekspresi yang sama secara kontekstual, memberikan hasil evaluasi yang lebih baik pada

kesamaan antar teks. Hal ini lah yang menjadikan BERTScore sangat relevan untuk digunakan dalam pengukuran atau pengujian *natural language generation* seperti jawaban yang dihasilkan oleh *chatbot*, dimana variabilitas leksikal dan parafrase merupakan hal yang umum dan *surface-level metric* seringkali gagal mencerminkan kedekatan semantik yang sebenarnya (Zhang dkk., 2020).

Mirip dengan metrik lainnya, perhitungan pada BERTScore juga melibatkan perhitungan *recall*, *precision*, dan *f1-score*. Berikut ini adalah rumus perhitungan yang digunakan dalam BERTScore (T. Zhang dkk., 2020).

1. *Recall* (R_BERT)

$$R_{\text{BERT}} = \frac{1}{|x|}\sum_{x_i \in x} \max_{\widehat{x_j} \in \hat{x}} x_i^{\top}\, \widehat{x_j} \quad \text{.....(2.10)}$$

Keterangan :

x : himpunan token *embeddings* untuk kalimat referensi.

$|x|$: jumlah token dalam kalimat referensi yang digunakan untuk normalisasi.

x_i : vektor *embedding* untuk token ke-i dalam kalimat referensi.

$\hat{x}$: himpunan token *embedding* untuk kalimat kandidat.

$\widehat{x_j}$: vektor *embedding* untuk token ke-j dalam kalimat kandidat.

$x_i^{\top}\widehat{x_j}$: *dot product* antara x_i dan $\widehat{x_j}$. *Dot product* disini setara dengan *cosine similarity*.

$\max_{\widehat{x_j} \in \hat{x}} x_i^{\top}\, \widehat{x_j}$: untuk setiap token x_i dalam referensi, dicari token $\widehat{x_j}$ di kandidat yang paling mirip (*cosine similarity* tertinggi).

R_BERT mengukur representasi semantik dari jawaban referensi berhasil dicakup oleh respon *chatbot*. Nilai ini merefleksikan kemampuan *chatbot* dalam menyampaikan makna dan informasi utama yang terkandung dalam jawaban referensi, meskipun tidak selalu menggunakan kata atau struktur kalimat yang sama.

2. *Precision (*P_BERT)

$$P_{\text{BERT}} = \frac{1}{|\hat{x}|}\sum_{\widehat{x_j} \in \hat{x}} \max_{x_i \in x} x_i^{\top}\, \widehat{x_j} \quad \text{.....(2.11)}$$

Keterangan :

$\hat{x}$: himpunan token *embedding* untuk kalimat kandidat

$|\hat{x}|$: jumlah token dalam kalimat kandidat yang digunakan untuk normalisasi

x : himpunan token *embeddings* untuk kalimat referensi.

x_i : vektor *embedding* untuk token ke-i dalam kalimat referensi.

$\hat{x}_j$: vektor *embedding* untuk token ke-j dalam kalimat kandidat.

$x_i^\top \hat{x}_j$: *dot product* antara x_i dan $\hat{x}_j$. *Dot product* disini setara dengan *cosine similarity*.

$\max_{x_i \in x} x_i^\top \hat{x}_j$: untuk setiap token $\hat{x}_j$ dalam kandidat, dicari token x_i di referensi yang paling mirip.

P_BERT mengukur tingkat relevansi semantik dari respon *chatbot* terhadap jawaban referensi. Nilai *precision* yang tinggi menunjukkan bahwa makna yang disampaikan dalam respon *chatbot* sebagian besar sesuai dengan konteks dan informasi yang diharapkan, serta meminimalkan penyimpangan makna.

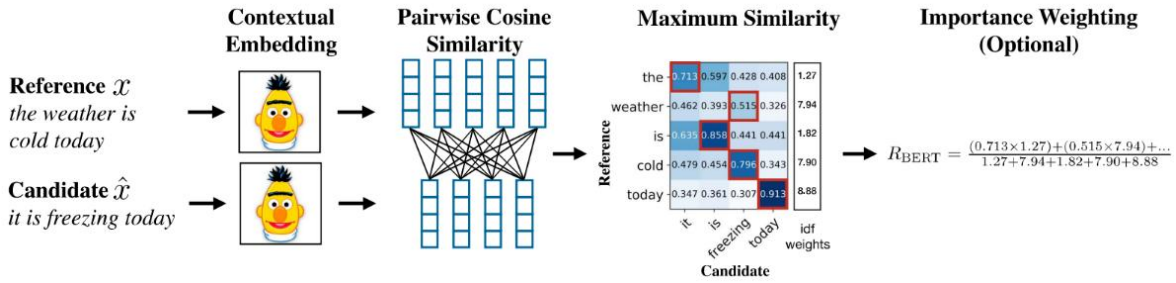
3. *F1-score* (F_BERT)

$$F_{\text{BERT}} = \frac{2 \cdot P_{\text{BERT}} \cdot R_{\text{BERT}}}{P_{\text{BERT}} + R_{\text{BERT}}} \quad \text{.....(2.12)}$$

Keterangan :

F_BERT merupakan nilai kombinasi antara R_BERT dan P_BERT yang digunakan untuk menilai keseimbangan antara kelengkapan dan ketepatan makna dalam respon *chatbot*. Komponen ini memberikan gambaran menyeluruh mengenai kualitas semantik respon yang dihasilkan.

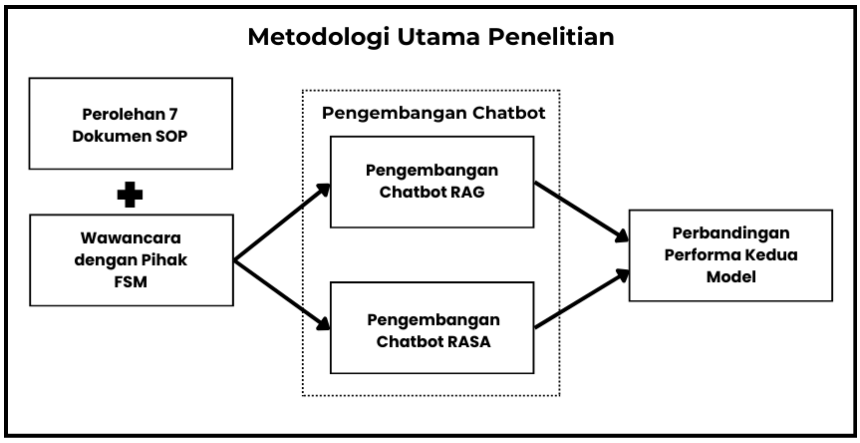
Contoh proses komputasi pada BERTScore (*Recall*) dapat dilihat pada Gambar 2.4.



Gambar 2.4. Contoh Komputasi pada BERTScore (Zhang dkk., 2020)

BAB III
METODOLOGI PENELITIAN

Bab ini berisi penjelasan mengenai tahapan yang akan dilakukan dalam pengembangan dan perbandingan kinerja *chatbot* berbasis *framework* RASA dan RAG pada layanan kemahasiswaan FSM Undip. Tahapan yang dilakukan meliputi pengembangan *chatbot* RAG, pengembangan *chatbot* RASA, dan kemudian melakukan perbandingan performa kedua model yang dihasilkan. Alur dari tahapan yang dilakukan dapat dilihat pada Gambar 3.1.



Gambar 3.1. Alur Tahapan Penelitian

3.1. Perolehan 7 Dokumen SOP

Tahapan pertama yang dilakukan dalam alur penelitian adalah perolehan 7 dokumen SOP. Dokumen diperoleh dan diizinkan untuk digunakan oleh Alan Prahutama, S.Si., M.Si., Ph.D dengan NIP 198804212014041002 sebagai salah satu anggota Tim Penjaminan Mutu Fakultas (TPMF). Ketujuh dokumen tersebut adalah sebagai berikut.

1. SOP pengisian IRS
2. SOP permohonan izin aktif kuliah setelah cuti
3. SOP permohonan izin cuti akademik
4. SOP permohonan izin keterlambatan pembayaran UKT
5. SOP legalisir ijazah dan transkrip
6. SOP pengajuan beasiswa
7. SOP pengajuan proposal kegiatan organisasi mahasiswa

Ketujuh dokumen tersebut akan digunakan sebagai sumber pengetahuan utama untuk membangun kedua model *chatbot*.

3.2. Wawancara dengan Pihak FSM

Tahapan berikutnya adalah wawancara dengan pihak mitra yaitu FSM Undip. Wawancara tersebut bertujuan untuk mengetahui pertanyaan apa saja yang sering diajukan oleh mahasiswa saat mengunjungi bagian akademik dan kemahasiswaan, sekaligus juga memperoleh jawaban asli (*ground truth*) dari masing-masing pertanyaan. Wawancara dilakukan secara *offline* dengan salah satu staf bidang akademik dan kemahasiswaan FSM yaitu Sukarni, S.E. dengan NIP 197504162007012001.

Wawancara telah berhasil dilakukan dan menghasilkan 10 pasang pertanyaan yang paling sering diajukan beserta jawaban aslinya (*ground truth*). Hasil wawancara ini dapat dilihat pada Lampiran 1

3.3. Pengembangan *Chatbot*

Proses pengembangan *chatbot* dibagi menjadi dua tahapan utama yaitu pengembangan *chatbot* berbasis *framework* RAG dan *chatbot* berbasis *framework* RASA. Keduanya dilakukan secara bersamaan/paralel. Pengembangan *chatbot* dilakukan pada perangkat keras dengan spesifikasi komponen yang ditampilkan Pada Tabel 3.1.

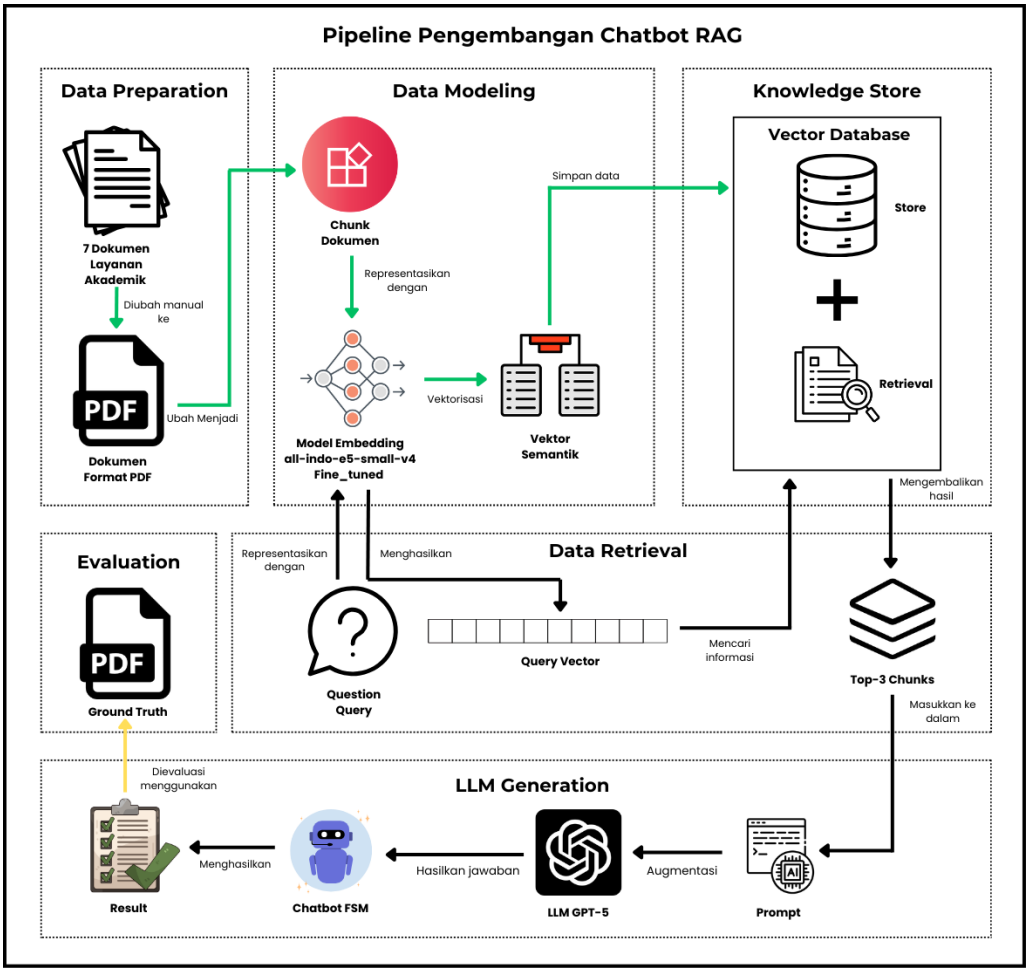
Tabel 3.1. Spesifikasi Perangkat Keras

No	Komponen	Spesifikasi
1	*Processor*	11th Gen Intel(R) Core(TM) i5-11400H @ 2.70GHz 2.69 GHz (6 Cores / 12 Threads)
2	RAM	16,0 GB DDR4
3	GPU *Type*	NVIDIA GeForce RTX 3060 (8 GB VRAM)
4	*Storage*	512 GB SSD NVMe
5	*Screen Resolution*	1920 x 1080 px (Full HD)

Berikutnya akan dijelaskan bagaimana proses pengembangan masing-masing *chatbot*.

3.3.1. Pengembangan *Chatbot* RAG

Bagian ini akan berfokus pada penjelasan bagaimana tahapan yang dilakukan dalam mengembangkan *chatbot* berbasis *framework* RAG. *Pipeline* yang digunakan dalam pengembangan *chatbot* RAG dapat dilihat pada Gambar 3.2.



Gambar 3.2. *Pipeline* Pengembangan *Chatbot* RAG

Dari *pipeline* tersebut, selanjutnya akan dijelaskan bagian-bagian tahapannya sebagai berikut.

3.3.1.1. *Data Preparation*

Tahapan *data preparation* menjadi hal yang pertama yang harus dilakukan dalam alur pengembangan *chatbot* RAG. *Data preparation* adalah tahapan *pre-processing* data sebelum digunakan dalam permodelan data. Data awal yang dimiliki masih berupa *word document*. Namun, pemrosesan pada *framework* RAG membutuhkan dokumen sumber dalam format PDF (secara umum). Oleh karena itu perlu dilakukan pengubahan terlebih dahulu dokumen-dokumen yang didapat ke dalam format PDF. Selain itu, isi dokumen-dokumen tersebut disajikan dalam bentuk *flowchart* atau bagan alur yang merepresentasikan setiap proses dalam layanan akademik dan kemahasiswaan di lingkungan FSM Undip. Sehingga akhirnya disimpulkan bahwa dokumen-dokumen itu tidak dapat secara langsung diubah ke dalam format PDF menggunakan *converter,* namun harus diubah secara manual terlebih dahulu

menjadi bentuk representasi teks alur SOP. Baru setelahnya akan diubah ke dalam format PDF menggunakan *converter word to pdf*.

Setelah dokumen sudah berada dalam format PDF, selanjutnya dokumen tersebut akan dipecah menjadi bagian-bagian kecil yang disebut sebagai *chunk. Chunk* inilah yang nantinya akan digunakan lebih lanjut pada tahapan *data modeling*. Proses mengubah dari dokumen format PDF ke *chunk* ini disebut sebagai proses *chunking*, dan dilakukan menggunakan sebuah fungsi *RecursiveCharacterTextSplitter* yang berasal dari *library* Langchain python. Rumus dasar dari bagaimana sebuah dokumen dapat dipecah menjadi bagian-bagian *chunks* (*chunking*) dapat dilihat pada rumus 2.1. Berikut ini adalah contoh dari proses *chunking* salah satu kalimat di dokumen SOP izin cuti akademik.

Kalimat asli : "Mahasiswa mengunduh, mengisi, dan menandatangani Form Cuti Akademik dengan melampirkan persyaratan."

Jika ditetapkan panjang *chunk* adalah 10 kata maka kalimat akan dipecah menjadi beberapa bagian *chunk* yang disimbolkan dengan A_k. Jumlah maksimal token/kata adalah sebanyak 10, sehingga hasilnya akan tampak seperti ini :

A_1 = "Mahasiswa mengunduh mengisi dan menandatangani"

A_2 = "Form Cuti Akademik dengan melampirkan persyaratan"

Pada proses *chunking*, dalam penelitian ini akan diujikan 4 konfigurasi eksperimen yang bertujuan untuk menemukan nilai terbaik pada dua parameter utama. Dua parameter utama dalam *chunking* adalah *chunk_size* dan *chunk_overlap.* Konfigurasi yang akan diujikan ditunjukkan Pada Tabel 3.2.

Tabel 3.2. Konfigurasi Eksperimen pada Proses *Chunking*

Konfigurasi	***Chunk_size***	***Chunk_overlap***
1	500	50
2	800	80
3	1000	100
4	1700	100

Dilakukannya eksperimen pada dua parameter tersebut bertujuan untuk dapat menemukan hasil *chunking* terbaik yang sesuai dengan masalah yang akan coba diselesaikan pada penelitian ini yaitu dalam konteks dokumen SOP. Setelah proses *chunking* diterapkan, dokumen sudah dapat dikatakan siap untuk digunakan pada tahapan selanjutnya.

3.3.1.2. *Data Modeling*

Tahapan kedua, *data modeling* berfokus pada pembuatan model data yang dapat digunakan dalam proses *information retrieval* yang terdapat di dalam *framework* RAG. Setelah pada tahapan sebelumnya dokumen-dokumen PDF telah dirubah ke dalam bentuk *chunk*, langkah pertama yang dilakukan pada tahapan ini adalah untuk mengubah representasi dari *chunk* dokumen ke dalam sebuah ruang vektor berdimensi tinggi yang mana semua kata/elemen *chunk* yang mirip diletakkan pada titik-titik yang berdekatan satu sama lain. Langkah ini disebut sebagai proses *embedding*. Proses ini bertujuan agar kata, kalimat, dan dokumen dapat ditampilkan dalam representasi kontinu dari vector, yang memungkinkan algoritma pencarian nantinya dapat menangkap hubungan semantik dan sintaksis. Proses *embedding* ini biasanya dilakukan dengan menggunakan model yang telah dilatih khusus untuk *embedding* dan biasanya berupa model *pre-trained*. Akhir dari proses ini adalah bentuk baru atau represenasi *chunk* dalam bentuk vektor semantik. Vektor inilah yang akan disimpan ke dalam basis data vektor untuk digunakan dalam proses *retrieval*.

3.3.1.3. *Knowledge Store*

Knowledge store berfokus pada bagaimana data vektor semantik yang menjadi sumber pengetahuan utama dari *framework* RAG disimpan. Basis data yang digunakan dalam keseluruhan *framework* RAG ditentukan pada tahapan ini. Perlu diketahui bahwa *database* yang digunakan bukanlah jenis *database* biasa melainkan sebuah *vector database*. *Vector database* didesain secara khusus untuk menyimpan, mengambil, dan mengelola vektor berdimensi tinggi secara efisien. Pada bagian atau tahapan ini juga dilakukan pengaturan proses *retrieval* yang akan digunakan dalam keseluruhan *framework*. Proses *retrieval* yang akan digunakan pada *framework* adalah proses *information retrieval* berbasis *cosine similarity*. Perhitungan *cosine similarity* seperti yang telah disebutkan pada rumus 2.4. akan menghasilkan keluaran berupa ukuran seberapa dekat sebuah *query* yang diberikan terhadap semua dokumen dalam representasi vektor semantik yang disimpan dalam *database*. Contoh perhitungannya dapat dilihat pada Tabel 3.3.

Tabel 3.3. Contoh Representasi Vektor Dokumen dan *Query*

No	Indeks	Teks Dokumen atau *Query*	Representasi Vektor Sederhana (Contoh)
1	A_1	Mahasiswa mengunduh mengisi dan menandatangani	[0.6, 0.8, 0.0]

No	Indeks	Teks Dokumen atau *Query*	Representasi Vektor Sederhana (Contoh)
2	A_2	Form Cuti Akademik dengan melampirkan persyaratan	[0.3, 0.9, 0.1]
3	Q	Apa langkah pertama untuk mengajukan cuti akademik?	[0.4, 0.8, 0.2]

1. Nilai *Cosine Similarity* antara Teks A_1 dengan Query

$$\text{similarity}(A_1, Q) = \frac{\overrightarrow{A_1} \cdot \vec{Q}}{|\overrightarrow{A_1}||\vec{Q}|}$$
$$= \frac{(0.6)(0.4) + (0.8)(0.8) + (0.0)(0.2)}{\sqrt{0.6^2 + 0.8^2 + 0.0^2}\sqrt{0.4^2 + 0.8^2 + 0.2^2}}$$
$$= \frac{0.24 + 0.64 + 0}{\sqrt{1.0\sqrt{0.84}}}$$
$$= \frac{0.88}{0.9165}$$
$$= 0.96$$

2. Nilai *Cosine Similarity* antara Teks A_2 dengan Query

$$\text{similarity}(A_2, Q) = \frac{\overrightarrow{A_2} \cdot \vec{Q}}{|\overrightarrow{A_2}||\vec{Q}|}$$
$$= \frac{(0.3)(0.4) + (0.9)(0.8) + (0.1)(0.2)}{\sqrt{0.3^2 + 0.9^2 + 0.1^2}\sqrt{0.4^2 + 0.8^2 + 0.2^2}}$$
$$= \frac{0.12 + 0.72 + 0.02}{\sqrt{0.91\sqrt{0.84}}}$$
$$= \frac{0.86}{0.8747}$$
$$= 0.98$$

Ukuran matematis tersebut kemudian digunakan sebagai dasar dalam menentukan tingkat relevansi sumber pengetahuan terhadap query yang diberikan. Di akhir tahapan ini, semua bagian yang dibutuhkan dalam proses pencarian informasi pada *framework* RAG telah berhasil dibentuk dan informasinya telah berhasil disimpan ke dalam *vector database*.

3.3.1.4. *Data Retrieval*

Tahapan ini adalah tahapan yang akan terjadi saat *user* melakukan interaksi terhadap *chatbot* yang dibangun. Pertanyaan atau pernyataan yang diajukan oleh *user* akan diproses terlebih dahulu menggunakan *model embedding* untuk diubah ke dalam bentuk representasi vektor. Hal ini tentu perlu untuk dilakukan mengingat semua *knowledge* yang ada di dalam

penyimpanan berada dalam bentuk vektor semantik. Setelah berhasil diubah ke dalam bentuk vektor kueri, data ini kemudian diberikan kepada *retriever* untuk diproses. Dengan menggunakan algoritma pencarian berbasis *cosine similarity*, diperoleh skor dari dokumen-dokumen yang ada pada *database*. Dari semua dokumen yang dikembalikan, diambil 3 dokumen terbaik yang nantinya akan dijadikan sebagai sumber informasi dalam memberikan respon kepada *user*. Tiga dokumen tersebut akan digunakan pada tahapan selanjutnya yaitu LLM *Generation*.

3.3.1.5. *LLM Generation*

LLM *generation* menjadi salah satu tahapan yang cukup penting dalam pembuatan *chatbot* berbasis *framework* RAG. Karena prinsip utama sebenarnya dalam *framework* RAG adalah bagaimana cara yang dapat dilakukan untuk memberikan *knowledge* tambahkan bagi LLM sebelum dapat memberikan jawaban atau respon kepada *user*. Pada tahapan sebelumnya telah diketahui bagaimana proses pencarian sumber dokumen yang relevan dengan *query* yang diberikan oleh *user*. Ketiga dokumen sumber yang paling relevan tersebut selanjutnya akan disematkan ke dalam *prompt* sebagai informasi yang dibawa. *Prompt* tentunya juga telah diatur sedemikian rupa sehingga mampu membuat LLM menjawab sesuai dengan beberapa aturan tertentu yang diminta pada *prompt* dan informasi yang ada. Selanjutnya *prompt* akan dikirimkan ke model LLM OpenAI gpt-5 yang dipanggil melalui API untuk dapat mengaugmentasi/memberikan kreativitas pada jawaban yang diberikan berdasarkan informasi yang dibawa pada *prompt*. Hasil jawaban yang dibuat oleh LLM selanjutnya akan diberikan kepada *chatbot* FSM untuk kemudian diberikan kepada *user* sebagai respon atas apa yang dikirimkan *user* melalui *query*.

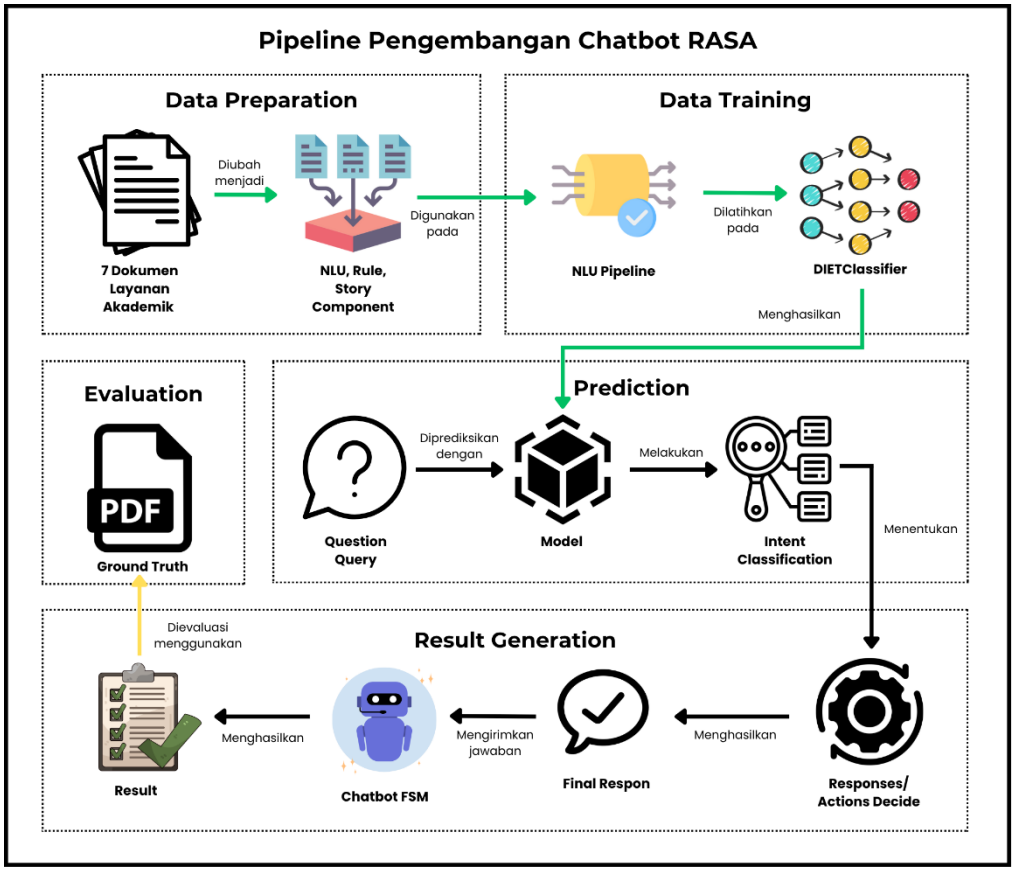
3.3.1.6. *Evaluation Process*

Tahapan terakhir yang sekaligus juga menjadi tahapan yang penting adalah evaluasi terhadap performa model *chatbot* berbasis RAG yang telah dibangun. Proses evaluasi dilakukan dengan mengujikan *chatbot* menggunakan beberapa pertanyaan yang sering diajukan (FAQ). Daftar pertanyaan yang sering diajukan diperoleh dari hasil wawancara dengan *staf* di bidang akademik dan kemahasiswaan yang biasanya secara langsung melayani mahasiswa. Hasil dari pengujian dibandingkan dengan *ground truth* jawaban sebenarnya yang juga diperoleh dari hasil wawancara. Penilaian performa dilakukan dengan menggunakan dua metrik yaitu ROUGE *Metric* dan BERTScore *Metric* yang mampu

melakukan penilaian *token matching* dan hubungan semantik. Kedua metrik tersebut diterapkan pada jawaban masing-masing pertanyaan yang diujikan terhadap jawaban *ground truth*. Hasil pengujian akan disimpan bersama dengan lama respon tiap pertanyaan untuk digunakan nantinya pada tahapan evaluasi dan perbandingan hasil dengan model *chatbot* RASA.

3.3.2. Pengembangan *Chatbot* RASA

Bagian ini akan berfokus pada penjelasan bagaimana tahapan yang dilakukan dalam mengembangkan *chatbot* berbasis *framework* RASA. *Pipeline* yang digunakan dalam pengembangan *chatbot* RASA dapat dilihat pada Gambar 3.3.



Gambar 3.3. *Pipeline* Pengembangan *Chatbot* RASA

Dari *pipeline* tersebut, selanjutnya akan dijelaskan bagian-bagian tahapannya sebagai berikut.

3.3.2.1. *Data Preparation*

Tahapan *data preparation* yang dilakukan pada *pipeline* pengembangan *chatbot* RASA berbeda dengan *pipeline* pengembangan *chatbot* RASA. Hal ini dikarenakan dokumen yang telah diperoleh tidak perlu diubah ke dalam format PDF melainkan langsung saja diekstraksi, dipecah dan dibuat menjadi ketiga komponen utama yang diperlukan dalam pengembangan *model* pada *chatbot* RASA. Tiga komponen tersebut adalah NLU yang berisi serangkaian *intent*, *rules* dan juga *story*. Tiga komponen tersebut berfungsi sebagai sumber *knowledge* yang jika pada *pipeline* pengembangan RAG disimpan dalam bentuk vektor semantik. Proses pengubahan ini dapat dilakukan secara manual maupun menggunakan *tools* otomasi seperti LLM, namun pada penelitian ini akan dibuat secara manual untuk hasil yang presisi. Setelah tahapan ini selesai dilakukan, maka data telah siap untuk digunakan dalam proses pelatihan dan pembuatan model.

3.3.2.2. *Data Training*

Tahapan selanjutnya yaitu *data training* bertujuan untuk dapat melatih model yang telah disediakan oleh *framework* RASA agar dapat mengenali dokumen sumber yang digunakan melalui ketiga komponen yang telah dibuat sebelumnya yaitu NLU, *rule*, dan *story*. Ketiga komponen tersebut akan digunakan pada NLU *Pipeline* yang berisi file *domain, endpoint, config* dan file-file lainnya yang sebenarnya tidak perlu dilakukan banyak penyesuaian, hanya mungkin file *domain* perlu dilakukan penyesuaian sedikit agar nantinya model dapat memberikan jawaban dalam lingkup yang diinginkan. Setelah file-file tersebut sudah tersedia dan selesai disesuaikan, maka file-file tersebut akan digunakan untuk mengkonfigurasi dan melatih model bernama DIETClassifier yang menjadi model utama bawaan dari *framework* RASA. Model DIETClassifier yang sudah dilatih ini kemudian disimpan sebagai model baru yang nantinya akan dipanggil saat proses prediksi dilakukan pada *chatbot*.

3.3.2.3. *Prediction Stage*

Tahapan prediksi berfokus pada alur yang menjelaskan bagaimana *user* dapat mengajukan pertanyaan atau pertanyaan dan kemudian diproses oleh model yang diperoleh dari tahapan sebelumnya. Proses pertama yang dilakukan adalah *user* mengetikkan pertanyaan atau pernyataan yang ingin akan diproses model. Kemudian pertanyaan atau pernyataan tersebut dikirimkan pada model untuk diprediksi. Model kemudian

mengklasifikasikannya dengan *intent* yang sesuai berdasarkan data latih yang diberikan. Hasil dari klasifikasi ini kemudian akan digunakan untuk menentukan respon atau aksi apa yang akan dilakukan dalam menanggapi pertanyaan atau pernyataan dari *user*.

3.3.2.4. *Result Generation*

Tahapan ini menjadi tahapan penting dalam *pipeline framework* RASA. *Intent* yang berhasil diidentifikasi oleh model selanjutnya digunakan untuk menentukan aksi apa yang akan diambil dan respon apa yang akan diberikan, hal ini dilakukan berdasarkan file *rule* yang telah ditetapkan sebelumnya pada tahapan *data preparation*. Proses atau aksi yang dilakukan akan menghasilkan jawaban akhir yang dikirimkan model *chatbot* untuk diteruskan kepada *user* sebagai jawaban atau tanggapan dari *query* pertanyaan atau pernyataan yang diajukan. Berbeda dengan pendekatan model yang menggunakan LLM, jawaban atau respon yang diberikan akan sama persis atau exact dengan pengetahuan yang sebelumnya telah ditanamkan dalam file-file konfigurasi *framework* RASA.

3.3.2.5. *Evaluation Process*

Tahapan evaluasi dilakukan sama dengan apa yang dilakukan pada evaluasi *framework* RAG, dimana model *chatbot* FSM yang telah berhasil dibentuk diujikan dengan beberapa pasang pertanyaan hasil wawancara dengan staf bagian akademik dan kemahasiswaan. Namun yang membuat berbeda pada tahapan evaluasi di *framework* RASA ini adalah sebelum dapat mengevaluasi perlu ditambahkan *custom function* untuk dapat mengukur *response time* dan *confidence* dari jawaban yang dihasilkan *chatbot* (*tracker*). Dari *function tracker* tersebutlah dapat dilihat *response time* dan *confidence* dan mencatat masing-masing jawaban pertanyaan yang diujikan. Jawaban pertanyaan yang dicatat kemudian diukur dengan menggunakan dua metrik yang sama yaitu ROUGE dan BERTScore terhadap *ground truth* dari masing-masing pertanyaan. Pengukuran dicatat dan disimpan untuk digunakan pada tahapan akhir dari keseluruhan metodologi penelitian yaitu evaluasi dan perbandingan hasil dari kedua model *chatbot* yang dibangun.

3.4. Perbandingan Performa Kedua Model

Tahapan terakhir dari alur penelitian adalah melakukan evaluasi dan perbandingan hasil dari kedua pengujian yang sebelumnya telah dilakukan pada masing-masing *chatbot*. Fokus pada bagian ini bukanlah lagi mengujikan pertanyaan kepada model *chatbot* RAG

dan RASA, namun melakukan analisis yang lebih dalam dari hasil evaluasi yang telah dilakukan sebelumnya. Metrik evaluasi yang akan dianalisis meliputi *response time*, *coverage*, skor ROUGE-1, skor ROUGE-L, dan skor BERT-Score. Analisis terhadap metrik-metrik tersebut akan digunakan untuk membuat kesimpulan pendekatan *framework* mana yang lebih baik dalam menyelesaikan permasalahan layanan akademik dan kemahasiswaan di lingkungan FSM Undip. Dengan berakhirnya tahapan ini, maka seluruh rangkaian alur penelitian yang dibuat telah berhasil dilakukan dan diselesaikan.

BAB IV
HASIL DAN PEMBAHASAN

Bab ini berisi penjelasan mengenai tahapan penelitian yang telah dilakukan dalam melakukan perbandingan Kinerja *Framework* RASA dan RAG pada *Chatbot* Layanan Kemahasiswaan. Tahapan tersebut meliputi pengembangan *chatbot* RAG, pengembangan *chatbot* RASA, dan perbandingan performa kedua model.

4.1. Pengembangan *Chatbot* RAG

Tahapan pengembangan *chatbot* RAG dilakukan melalui beberapa tahapan sebagai berikut mulai dari tahapan persiapan data hingga pengujian dan evaluasi terhadap model *chatbot* yang berhasil dibangun.

4.1.1. *Data Preparation*

Langkah pertama yang dilakukan dari tahapan *data preparation* adalah melakukan perubahan terhadap dokumen yang diperoleh dari hasil wawancara ke format *pdf*. Format dokumen yang diperoleh dari supervisor bidang akademik dan kemahasiswaan masih dalam bentuk *word document* dan isi dari dokumen tersebut juga tidak dapat langsung digunakan pada sistem RAG. Isi dari dokumen yang diberikan adalah berupa *flowchart* alur SOP layanan kemahasiswaan. Isi mentah dari dokumen-dokumen ini dapat dilihat pada Lampiran 2.

Dokumen-dokumen tersebut harus ditulis ulang secara manual dalam bentuk urutan langkah-langkah SOP. Langkah-langkah yang akan dituliskan diperoleh dari pemahaman terhadap *flowchart* yang ada pada dokumen utama. Tiap langkah-langkah yang dituliskan nantinya juga disertai dengan keterangan tambahan yang sebelumnya juga disebutkan di dalam dokumen awal, misalnya seperti waktu pengerjaan, prasyarat dokumen, dokumen luaran dan keterangan lainnya seperti link unduh dokumen prasyarat. Penulisan ualng dilakukan oleh peneliti dalam format *word document* dan kemudian dikonversi kembali ke dalam bentuk *pdf* menggunakan fungsi *export* (*create pdf*) yang disediakan dalam aplikasi *microsoft word*. Hasil dari penyesuaian isi dokumen-dokumen yang akan digunakan sebagai sumber pengetahuan dalam pengembangan *chatbot* RAG dapat dilihat pada Lampiran 3.

Selanjutnya setiap dokumen yang telah disesuaikan isinya dan disimpan dalam format *pdf* perlu dipecah ke dalam bagian-bagian yang lebih kecil atau disebut sebagai *chunk*. Pengubahan ke dalam bentuk *chunk* dapat dilakukan dengan menggunakan bantuan *library* `langchain.text_splitter` yang di dalamnya terdapat fungsi `RecursiveCharacterTextSplitter`. Implementasi kode python yang digunakan untuk melakukan proses ini ditunjukkan pada *Source Code* 4.1.

Source Code 4.1. Implementasi Proses *Chunking*

```
import os
from langchain_community.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter

DATA_PATH = "data"
OUTPUT_PATH = "chunks.json"

def split_document(file_path):
    loader = PyPDFLoader(file_path)
    documents = loader.load()
    splitter = RecursiveCharacterTextSplitter(
        chunk_size=1700,
        chunk_overlap=100,
        length_function=len,
        is_separator_regex=False,
    )
    chunks = splitter.split_documents(documents)

    print(f"\nFile: {os.path.basename(file_path)} → {len(chunks)} chunks")
    chunk_data = []
    for i, chunk in enumerate(chunks, start=1):
        print(f"\n--- Chunk {i}/{len(chunks)} ---")
        print(chunk.page_content)
        chunk_data.append({
            "chunk_id": i,
            "content": chunk.page_content
        })

    return chunk_data

def main():
    all_chunks = {}

    for fname in os.listdir(DATA_PATH):
        if fname.endswith(".pdf"):
            doc_path = os.path.join(DATA_PATH, fname)
            chunks = split_document(doc_path)
            all_chunks[fname] = chunks

if __name__ == "__main__":
    main()
```

Source code 4.1 adalah implementasi proses *chunking* menggunakan *library* yang disediakan oleh python. Proses *chunking* dapat dilihat pada fungsi `split_document`. Fungsi tersebut akan memproses dokumen yang dimuat dengan menggunakan bantuan fungsi `PyPDFLoader` dari lokasi file yang diterima sebagai parameter. Selanjutnya proses inti *chunking* akan dilakukan menggunakan fungsi `RecursiveCharacterTextSplitter` yang

telah dipannggil. Di dalam fungsi tersebut telah diberikan beberapa parameter yang akan mengatur bagaimana proses *chunking* dilakukan pada dokumen sumber. Parameter pertama yaitu `chunk_size` menentukan seberapa panjang maksimum ukuran *chunk* yang akan digunakan untuk membagi setiap dokumen sumber. Jika teks melebihi panjang `chunk_size` maka akan dijadikan bagian (*chunk*) baru. Biasanya proses pemilihan ukuran *chunk* yang akan digunakan dilakukan bersamaan dengan penentuan parameter berikutnya yaitu `chunk_overlap` yang mengatur jumlah karakter yang saling tumpang tindih antara dua *chunk* berurutan. Parameter `chunk_overlap` sangat berguna untuk menjaga konteks antar *chunk* dan memastikan informasi antar *chunk* konsisten. Sebelum menentukan nilai yang tepat dari kedua parameter untuk kasus penelitian ini, telah dicoba konfigurasi parameter `chunk_size` dan `chunk_overlap` yang sebelumnya telah disebutkan pada Tabel 3.1. Hasil dari masing-masing konfigrasi yang telah dilakukan dan contoh hasilnya terhadap dokumen SOP pengajuan beasiswa dapat dilihat pada Tabel 4.1.

Tabel 4.1. Hasil Konfigurasi *Chunk* pada Salah Satu Dokumen Sumber

No	*Chunk size*	Teks Hasil
1	500	**Chunk 1 :** Standar Operasional Prosedur (SOP) Surat Pengajuan Rekomendasi Beasiswa Dokumen ini berisi langkah -langkah yang dapat diikuti untuk mengajukan dan mendapatkan rekomendasi beasiswa dengan total maksimal waktu pemrosesan m aksimal 3 (tiga) hari, 45 menit waktu kerja. 1. Mahasiswa mendownload dan mengisi formulir/Surat Rekomendasi Pengajuan Beasiswa ke Subbag Akademik dan Kemahasiswaan (BAK) Fakultas. Waktu: ±30 menit
		Chunk 2 : Waktu: ±30 menit Dokumen yang dibutuhkan: Surat Rekomendasi Beasiswa dan KHS yang dilegalisir Link unduh form: https://drive.google.com/file/d/18f_nbPXHbElgRNoplDFQVNYGGDkFxBTT/view?usp=sharing Output: Surat Rekomendasi Beasiswa dan KHS yang dilegalisir 2. BAK Fakultas meneliti dan memverifikasi Surat Rekomendasi Pengajuan Beasiswa. Waktu: ±1 hari 3. BAK Fakultas memberikan paraf dan nomor surat pada Surat Rekomendasi Pengajuan Beasiswa tersebut. Waktu: ±10 menit
		Chunk 3 : Beasiswa tersebut. Waktu: ±10 menit 4. Wakil Dekan Akademik dan Kemahasiswaan memberikan tanda tangan pada Surat Rekomendasi Pengajuan Beasiswa tersebut. Waktu: ±2 hari 5. Mahasiswa mengambil surat tersebut di BAK Fakultas dan mencatat bukti pengambilan.

No	*Chunk size*	Teks Hasil
		Waktu: ±5 menit
2	800	**Chunk 1 :** Standar Operasional Prosedur (SOP) Surat Pengajuan Rekomendasi Beasiswa Dokumen ini berisi langkah -langkah yang dapat diikuti untuk mengajukan dan mendapatkan rekomendasi beasiswa dengan total maksimal waktu pemrosesan m aksimal 3 (tiga) hari, 45 menit waktu kerja. 1. Mahasiswa mendownload dan mengisi formulir/Surat Rekomendasi Pengajuan Beasiswa ke Subbag Akademik dan Kemahasiswaan (BAK) Fakultas. Waktu: ±30 menit Dokumen yang dibutuhkan: Surat Rekomendasi Beasiswa dan KHS yang dilegalisir Link unduh form: https://drive.google.com/file/d/18f_nbPXHbElgRNoplDFQVNYGGDkFxBTT/view?usp=sharing Output: Surat Rekomendasi Beasiswa dan KHS yang dilegalisir 2. BAK Fakultas meneliti dan memverifikasi Surat Rekomendasi Pengajuan Beasiswa. Waktu: ±1 hari
		Chunk 2 : Waktu: ±1 hari 3. BAK Fakultas memberikan paraf dan nomor surat pada Surat Rekomendasi Pengajuan Beasiswa tersebut. Waktu: ±10 menit 4. Wakil Dekan Akademik dan Kemahasiswaan memberikan tanda tangan pada Surat Rekomendasi Pengajuan Beasiswa tersebut. Waktu: ±2 hari 5. Mahasiswa mengambil surat tersebut di BAK Fakultas dan mencatat bukti pengambilan. Waktu: ±5 menit
3	1000	**Chunk 1 :** Standar Operasional Prosedur (SOP) Surat Pengajuan Rekomendasi Beasiswa Dokumen ini berisi langkah -langkah yang dapat diikuti untuk mengajukan dan mendapatkan rekomendasi beasiswa dengan total maksimal waktu pemrosesan m aksimal 3 (tiga) hari, 45 menit waktu kerja. 1. Mahasiswa mendownload dan mengisi formulir/Surat Rekomendasi Pengajuan Beasiswa ke Subbag Akademik dan Kemahasiswaan (BAK) Fakultas. Waktu: ±30 menit Dokumen yang dibutuhkan: Surat Rekomendasi Beasiswa dan KHS yang dilegalisir Link unduh form: https://drive.google.com/file/d/18f_nbPXHbElgRNoplDFQVNYGGDkFxBTT/view?usp=sharing Output: Surat Rekomendasi Beasiswa dan KHS yang dilegalisir 2. BAK Fakultas meneliti dan memverifikasi Surat Rekomendasi Pengajuan Beasiswa. Waktu: ±1 hari 3. BAK Fakultas memberikan paraf dan nomor surat pada Surat Rekomendasi Pengajuan Beasiswa tersebut. Waktu: ±10 menit 4. Wakil Dekan Akademik dan Kemahasiswaan memberikan tanda tangan pada Surat
		Chunk 2 : Waktu: ±10 menit 4. Wakil Dekan Akademik dan Kemahasiswaan memberikan tanda tangan pada Surat Rekomendasi Pengajuan Beasiswa tersebut. Waktu: ±2 hari 5. Mahasiswa mengambil surat tersebut di BAK Fakultas dan mencatat bukti pengambilan.

No	*Chunk size*	Teks Hasil
		Waktu: ±5 menit
4	1700	**Chunk 1 :** Standar Operasional Prosedur (SOP) Surat Pengajuan Rekomendasi Beasiswa Dokumen ini berisi langkah -langkah yang dapat diikuti untuk mengajukan dan mendapatkan rekomendasi beasiswa dengan total maksimal waktu pemrosesan m aksimal 3 (tiga) hari, 45 menit waktu kerja. 1. Mahasiswa mendownload dan mengisi formulir/Surat Rekomendasi Pengajuan Beasiswa ke Subbag Akademik dan Kemahasiswaan (BAK) Fakultas. Waktu: ±30 menit Dokumen yang dibutuhkan: Surat Rekomendasi Beasiswa dan KHS yang dilegalisir Link unduh form: https://drive.google.com/file/d/18f_nbPXHbElgRNoplDFQVNYGGDkFxBTT/view?usp=sharing Output: Surat Rekomendasi Beasiswa dan KHS yang dilegalisir 2. BAK Fakultas meneliti dan memverifikasi Surat Rekomendasi Pengajuan Beasiswa. Waktu: ±1 hari 3. BAK Fakultas memberikan paraf dan nomor surat pada Surat Rekomendasi Pengajuan Beasiswa tersebut. Waktu: ±10 menit 4. Wakil Dekan Akademik dan Kemahasiswaan memberikan tanda tangan pada Surat Rekomendasi Pengajuan Beasiswa tersebut. Waktu: ±2 hari 5. Mahasiswa mengambil surat tersebut di BAK Fakultas dan mencatat bukti pengambilan. Waktu: ±5 menit

Dari konfigurasi parameter `chunk_size` dan `chunk_overlap` yang telah diterapkan yaitu secara berturut-turut (500 dan 50), (800 dan 80), (1000 dan 100), dan (1700 dan 100), diperoleh hasil bahwa dokumen sumber terpecah menjadi sejumlah bagian seperti yang tertulis pada Tabel 4.2.

Tabel 4.2. Jumlah Pecahan Dokumen Sumber Proses *Chunk*

No	*Chunk Size*	Pecahan Dokumen
1	500	Dokumen terpecah menjadi 3 bagian.
2	800	Dokumen terpecah menjadi 2 bagian.
3	1000	Dokumen terpecah menjadi 2 bagian.
4	1700	Dokumen tetap utuh.

Keputusan untuk menentukan besaran `chunk_size` dan `chunk_overlap` didasarkan pada tipe dokumen yang seluruhnya adalah dokumen Standar Operasional Prosedur (SOP) yang sudah seharusnya diambil secara utuh dan tidak dapat dipisah-pisah secara langsung dalama ukuran kecil. Ukuran *chunk* yang kecil akan menyebabkan langkah-langkah prosedur yang seharusnya dikembalikan atau diberikan secara lengkap kepada pengguna bisa saja tidak dikembalikan secara utuh (terpotong saat tahapan tertentu). Misalnya pada hasil *chunking* dokumen dengan konfigurasi `chunk_size` sebesar 1000 dan `chunk_overlap`

sebesar 100, dapat dilihat pada Tabel 4.2. bahwa proses *chunking* memotong dokumen menjadi dua bagian. Bagian pertama hanya berhenti sampai pada tahapan ke-4 yang seharusnya proses pengajuan beasiswa memiliki 5 tahapan. Tahapan kelima yang terkandung dalam *chunk* kedua bisa saja tidak dikembalikan oleh pencarian nantinya saat *user* menanyakan bagaimana tahapan pengajuan beasiswa. Oleh karena itu penting untuk menjaga kesatuan langkah-langkah yang biasanya ditemui pada dokumen prosedural. Inilah yang menjadi alasan akhirnya dipilih besaran `chunk_size` sebesar 1700 dan `chunk_overlap` sebesar 100 yang mampu menyimpan seluruh tahapan SOP dalam masing-masing dokumen secara utuh.

Selanjutnya parameter `length_function` dalam fungsi `RecursiveCharacterTextSplitter` digunakan untuk menghitung panjang teks yang akan dipotong. Dalam penelitian ini, digunakan fungsi `len()` bawaan dari python untuk menghitung jumlah karakter dalam teks. Parameter terakhir, yaitu `is_separator_regex` digunakan untuk mengatur apakah akan digunakan sebuah pola regex untuk memisahkan antar *chunk* atau tidak. Jika nilai parameter ini diatur sebagai "False" maka *chunk* hanya akan benar-benar dibagi berdasarkan `chunk_size` yang telah ditentukan. Dengan mengatur semua parameter tersebut dan menjalankan fungsi untuk masing-masing dokumen, maka setiap dokumen telah berubah bentuk menjadi *chunk* dan siap digunakan pada tahapan selanjutnya.

4.1.2. *Data Modeling*

Inti kegiatan utama pada tahapan *data modeling* adalah untuk merubah representasi dari *chunk* yang telah dihasilkan pada tahapan *data preparation* menjadi bentuk vektor matematis yang nantinya dapat digunakan dalam proses *information retrieval*. Proses ini disebut sebagai proses *embedding* dimana setiap kata direpresentasikan ke dalam ruang vektor berdimensi tinggi yang kemudian ditampilkan atau disajikan ke dalam dimensi vektor yang lebih rendah. Pada penelitian ini digunakan sebuah model *pre-trained* yang diperoleh dari *platform* Hugging Face. *Platform* Hugging Face menyediakan berbagai jenis model *machine learning* dan *deep learning* yang dapat digunakan secara *open-source*. Dari sekian banyak model yang dapat digunakan untuk melakukan proses *embedding*, peneliti memilih model `all-indo-e5-small-v4` yang dibuat oleh *author* `LazarusNLP` sebagai salah satu *specialist author* dibidang *text embedding* yang sebelumnya juga telah melakukan berbagai

penelitian terkait model IndoBERT untuk dapat belajar multi bahasa dan multi budaya. Model `all-indo-e5-small-v4` adalah sebuah model *sentence-transformers* yang dapat digunakan untuk merepresentasikan paragraf dan kalimat ke dalam vektor 384 dimensi. Model tersebut dilatih menggunakan 3 *dataset* yaitu `mMARCO, miracl, dan tydiqa-goldp`. Ketiga dataset tersebut merupakan *dataset* multi bahasa dengan tydiqa-goldp berupa *dataset Question-Answer* (QA).

Tentu model tersebut sebenarnya dapat langsung digunakan untuk mengubah setiap *chunk* yang telah dihasilkan ke dalam bentuk dimensi vektor. Namun hal tersebut tidak efektif karena domain data yang digunakan untuk melatih model `all-indo-e5-small-v4` tentu berbeda dengan domain data yang akan digunakan dalam penelitian ini yaitu domain layanan akademik FSM Undip. Perbedaan domain ini jika dibiarkan akan membuat representasi vektor dari teks yang nanti dihasilkan dengan menggunakan model tidak dapat menangkap secara baik hubungan semantik dari domain yang digunakan. Maka dari itu sebelum dapat menggunakan model `all-indo-e5-small-v4` untuk merubah representasi *chunk* dari dokumen sumber, model dilatihkan terlebih dahulu dengan pasangan QA yang dihasilkan dari dokumen sumber yang akan digunakan. Pelatihan ini biasa juga disebut dengan *domain adaptive training*. Pasangan QA dibuat secara otomatis dengan menggunakan model lokal `llama2` yang disediakan secara gratis dengan mengunduhnya melalui Ollama di perangkat lokal (Windows). Otomasi pembuatan pasangan QA diimplementasikan dalam code python, yang ditunjukkan pada *Source Code* 4.2.

Source Code 4.2. Implementasi Pembuatan QA untuk *Fine-tuning*

```
import os, json
from langchain_community.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_ollama import OllamaLLM

DATA_PATH = "data"
DATASET_PATH = "datasets"

llm = OllamaLLM(model="llama2")

def split_document(file_path):
    loader = PyPDFLoader(file_path)
    documents = loader.load()
    splitter = RecursiveCharacterTextSplitter(
        chunk_size=1700,
        chunk_overlap=100,
        length_function=len,
        is_separator_regex=False,
    )
    chunks = splitter.split_documents(documents)

    print(f"\nFile: {os.path.basename(file_path)} → {len(chunks)} chunks")
```

```
    for i, chunk in enumerate(chunks, start=1):
        print(f"\n--- Chunk {i}/{len(chunks)} ---")
        print(chunk.page_content)

    return chunks

def generate_qa_for_doc(doc_path, out_file):
    chunks = split_document(doc_path)
    print(f"{os.path.basename(doc_path)} → {len(chunks)} chunks")

    qa_pairs = []
    for i, chunk in enumerate(chunks):
        prompt = f"""
        Buatlah SATU (1) pasang pertanyaan dan jawaban dalam bahasa Indonesia
        berdasarkan teks berikut.

        Hanya kembalikan hasil dengan format persis seperti ini:

        Q: <pertanyaan>
        A: <jawaban>

        Jangan tambahkan penjelasan lain, kalimat pengantar, atau teks tambahan.

        Teks:
        \"\"\"{chunk.page_content}\"\"\"
        """

        response_id = llm.invoke(prompt).strip()

        qa_pairs.append({
            "id":                                  f"{chunk.metadata.get('source',
os.path.basename(doc_path))}:{chunk.metadata.get('page', 0)}:{i}",
            "qa": response_id,
            "source": chunk.metadata.get("source", os.path.basename(doc_path))
        })

        print(f"Processed {i+1}/{len(chunks)} chunks")

    with open(out_file, "w", encoding="utf-8") as f:
        json.dump(qa_pairs, f, ensure_ascii=False, indent=2)
    print(f"QA pairs saved: {out_file}")

def main():
    os.makedirs(DATASET_PATH, exist_ok=True)

    for fname in os.listdir(DATA_PATH):
        if fname.endswith(".pdf"):
            doc_path = os.path.join(DATA_PATH, fname)
            dataset_file              =              os.path.join(DATASET_PATH,
f"train_{os.path.splitext(fname)[0]}.json")

            if not os.path.exists(dataset_file):
                print(f"Dokumen baru ditemukan: {fname}")
                generate_qa_for_doc(doc_path, dataset_file)
            else:
                print(f"Dataset sudah ada untuk {fname}, skip.")

if __name__ == "__main__":
    main()
```

Keseluruhan *source code* tersebut digunakan untuk membuat pasangan pertanyaan dan jawaban dari dokumen sumber yang diberikan. Fungsi `split_document` yang digunakan pada rangkaian algoritma ini adalah fungsi yang sama yang telah digunakan untuk

melakukan proses *chunking*. Setelah setiap dokumen sumber dipecah menjadi bentuk *chunk*, setiap *chunk* dikirimkan bersamaan dengan *prompt* ke model LLM `llama2` untuk dapat menghasilkan pasangan pertanyaan dan jawaban. *Prompt* diperlukan agar model llm paham apa perintah yang diinginkan yang dalam kasus ini adalah untuk membuat pasangan pertanyaan dan jawaban dari sumber *chunk* yang diberikan dengan menyematkan variabel `chunk.page_content` pada *prompt* yang dikirimkan. *Prompt* juga telah dimodifikasi dengan menyebutkan format jawaban yang diinginkan beserta dengan kalimat perintah larangan agar hasilnya tepat sesuai dengan apa yang diharapkan. Respon dari LLM kemudian disimpan dalam file berformat *json* untuk nantinya digunakan dalam proses *tuning* model *embedding*. Pasangan pertanyaan dan jawaban yang dihasilkan dari setiap dokumen SOP yang digunakan dapat dilihat pada Tabel 4.3.

Tabel 4.3. Hasil Pembuatan QA dari Setiap Dokumen SOP

No	Sumber Dokumen SOP	Hasil QA

1	SOP pengisian IRS	[
		{
		"id": "data\\SOP_Pengisian_IRS.pdf:0:0",
		"qa": "Q: Bagaimana cara mengisi Isian Rencana Studi (IRS) dengan total waktu pemrosesan maksimal 3 (tiga) hari kerja?\nA: Mahasiswa membayar biaya pendidikan pada bank yang bekerja sama dengan Undip, melakukan her-registrasi online melalui SIAP, melakukan pengisian, perbaikan, atau mencetak IRS sementara secara online melalui SIAP, menemui Pembimbing Akademik untuk berkonsultasi mengenai mata kuliah yang akan diambil, dan apabila mata kuliah dan jumlah SKS sudah sesuai, Pembimbing Akademik melakukan persetujuan secara online pada SIAP.",
		"source": "data\\SOP_Pengisian_IRS.pdf"
		}
		]

2	SOP permohonan izin aktif kuliah setelah cuti	[
		{
		"id": "data\\SOP_Permohonan_Izin_Aktif_Setelah_Cuti.pdf:0:0",
		"qa": "Q: Bagaimana cara memperoleh izin aktif setelah cuti?\nA: Mahasiswa membawa surat Izin Cuti Akademik melapor kepada Ketua Program Studi dan Subbag Akademik dan Kemahasiswaan, serta mengikuti registrasi online pada SSO masing-masing mahasiswa untuk terdaftar sebagai Peserta Kuliah / Mahasiswa Aktif di Fakuktas.",
		"source": "data\\SOP_Permohonan_Izin_Aktif_Setelah_Cuti.pdf"
		}
		]

3	SOP permohonan izin cuti akademik	[{ "id": "data\\SOP_Izin_Cuti_Akademik.pdf:0:0", "qa": "Q: Bermasalahologoan dalam proses permohonan izin cuti akademik?\nA: Mahasiswa dapat menghadapi masalah dalam proses permohonan izin cuti akademik, antara lain:\n\n1. Difficulty in downloading and filling out the application form due to technical issues or lack of access to the required documents.\n2. Delay in obtaining approval and signature from the program director due to busy schedules or conflicting priorities.\n3. Difficulty

No	Sumber Dokumen SOP	Hasil QA
		in submitting the completed application form to the dean's office due to misplacing or forgetting the document.\n4. Long processing time by the dean's office in reviewing and approving the application, resulting in delayed issuance of the permit.\n5. Insufficient information or documentation provided by the student, leading to rejection of the application.", "source": "data\\SOP_Izin_Cuti_Akademik.pdf" }]

4	SOP permohonan izin keterlambatan pembayaran UKT	[
		{
		"id": "data\\SOP_Permohonan_Izin_Keterlambatan_Pembayaran_UKT.pdf:0:0",
		"qa": "Q: Bermacam-macam langkah yang dapat diikuti untuk memperoleh izin keterlambatan pembayaran UKT?\nA: 1. Unduh, mengisi, dan menandatangani Form Permohonan Keterlambatan Pembayaran UKT; 2. Minta tanda tangan dosen wali serta persetujuan dan tanda tangan Ketua Program Studi; 3. Menyerahkan Form Permohonan Keterlambatan UKT yang ditandatangani dan disetujui kepada Supervisor Sumber Daya; 4. Supervisor Sumber Daya menerima, memeriksa, dan meneliti kelengkapan persyaratan form; 5. Wakil Dekan Sumber Daya memproses Surat Permohonan Izin Keterlambatan Pembayaran UKT; 6. Wakil Dekan Sumber Daya menandatangani Surat Permohonan Izin Keterlambatan Pembayaran UKT; 7. Subbag Sumber Daya memproses Surat Permohonan Keterlambatan Pembayaran UKT; 8. Mahasiswa membawa Surat Permohonan Keterlambatan Pembayaran UKT kepada Wakil Rektor II melalui Manajer Akademik; 9. Wakil Rektor II mendisposisikan ke Direktorat Keuangan untuk memproses pembayaran UKT; dan 10. Bendahara Penerimaan UKT membuka sistem pembayaran mahasiswa.",
		"source": "data\\SOP_Permohonan_Izin_Keterlambatan_Pembayaran_UKT.pdf"
		}
		]

5	SOP legalisir ijazah dan transkrip	[
		{
		"id": "data\\SOP_Legalisir_Ijazah_Dan_Transkrip.pdf:0:0",
		"qa": "Q: Bagaimana cara melakukan legalisir ijazah dan transkrip dengan mengikuti langkah-langkah yang diberikan dalam teks?\nA: Berikut adalah langkah-langkah yang dapat diikuti untuk melakukan legalisir ijazah dan transkrip:\n\n1. Alumni menyerahkan foto copy ijazah/transkrip nilai/sertifikat akreditasi program studi beserta surat aslinya kepada petugas Subbag Akademik dan Kemahasiswaan. Waktu pemrosesan: ±5 menit.\n2. Petugas Subbag Akademik dan Kemahasiswaan memeriksa keabsahan, kesesuaian foto copy ijazah/transkrip nilai/sertifikat akreditasi program studi dengan surat aslinya dan memberikan cap. Waktu pemrosesan: ±10 menit.\n3. Supervisor Akademik dan Kemahasiswaan memeriksa dan memberi paraf di samping kanan nama Dekan. Waktu pemrosesan: ±15 menit.\n4. Dekan/Wakil Dekan Akademik dan Kemahasiswaan menandatangani pada \"Pengesahan Telah diperiksa Kebenarannya dan Sesuai dengan Aslinya\". Waktu pemrosesan: ±1 hari.\n5. Petugas Subbag Akademik dan Kemahasiswaan mengambil kembali Legalisir yang sudah ditanda tangani Dekan / Wakil Dekan Akademik dan Kemahasiswaan dan memberikan stempel Fakultas Sains dan Matematika. Waktu pemrosesan: ±5 menit.\n6. Alumni mengambil Legalisir di Subbag Akademik dan Kemahasiswaan dengan membawa aslinya dan menulis di buku pengambilan dan di tanda tangani ybs. Waktu pemrosesan: ±15 menit.",
		"source": "data\\SOP_Legalisir_Ijazah_Dan_Transkrip.pdf"
		}
		]

No	Sumber Dokumen SOP	Hasil QA

6	SOP pengajuan rekomendasi beasiswa	[
		{
		"id": "data\\SOP_Pengajuan_Rekomendasi_Beasiswa.pdf:0:0",
		"qa": "Q: Baku apa yang dibutuhkan untuk mengajukan beasiswa melalui SOP Surat Pengajuan Rekomendasi Beasiswa?\nA: Mahasiswa mendownload dan mengisi formulir/Surat Rekomendasi Pengajuan Beasiswa ke Subbag Akademik dan Kemahasiswaan (BAK) Fakultas, dengan waktu pemrosesan maksimal 3 hari dan 45 menit. Dokumen yang dibutuhkan adalah Surat Rekomendasi Beasiswa dan KHS yang dilegalisir, dan hasil pemrosesan akan diberikan dalam bentuk Surat Rekomendasi Beasiswa dan KHS yang dilegalisir.",
		"source": "data\\SOP_Pengajuan_Rekomendasi_Beasiswa.pdf"
		}
		]

7	SOP pengajuan proposal kegiatan organisasi mahasiswa	[
		{
		"id": "data\\SOP_Pengajuan_Proposal_Kegiatan_Organisasi_Mahasiswa.pdf:0:0",
		"qa": "Q: Baku apa langkah-langkah yang dapat diikuti untuk melakukan pengajuan proposal kegiatan organisasi mahasiswa?\nA: Mahasiswa menyerahkan proposal ke petugas kemahasiswaan dan Supervisor Akademik untuk registrasi dan alokasi ruang,Supervisor Akademik dan Kemahasiswaan meneliti dan memverifikasi proposal kegiatan yang diajukan agar sesuai peraturan dan memberikan paraf, Wakil Dekan Akademik dan Kemahasiswaan mengevaluasi substansi proposal kegiatan, pengesahan Wakil Dekan Akademik dan Kemahasiswaan, mahasiswa mengambil proposal yang telah disetujui.",
		"source": "data\\SOP_Pengajuan_Proposal_Kegiatan_Organisasi_Mahasiswa.pdf"
		}
		]

Semua pasangan pertanyaan dan jawaban tersebut kemudian digunakan untuk melatih model *embedding*. Sehingga nantinya setelah pelatihan, model akan lebih dapat memahami makna/konteks setiap kata/kalimat dari sumber pengetahuan yang digunakan. Proses pelatihan model *embedding* diimplementasikan dalam code python, yang ditunjukkan pada *Source Code* 4.3.

Source Code 4.3. Implementasi Proses *Fine-tuning* Model *Embedding*

```
import os
import json
import shutil
import datetime
from sentence_transformers import SentenceTransformer, InputExample, losses
from torch.utils.data import DataLoader

DATASET_DIR = "datasets"
TRAINED_RECORD = os.path.join(DATASET_DIR, "trained.json")
TRAINED_LOG = os.path.join(DATASET_DIR, "trained.log")
BASE_MODEL = "LazarusNLP/all-indo-e5-small-v4"
FINAL_MODEL_DIR = "./indo_finetuned_embedding"
TEMP_MODEL_DIR = "./indo_finetuned_embedding_new"

def load_trained_files():
    """Load daftar file yang sudah pernah dipakai untuk training."""
```

```
    if os.path.exists(TRAINED_RECORD):
        with open(TRAINED_RECORD, "r", encoding="utf-8") as f:
            return set(json.load(f))
    return set()

def save_trained_files(trained_files):
    """Simpan daftar file yang sudah pernah dipakai untuk training."""
    with open(TRAINED_RECORD, "w", encoding="utf-8") as f:
        json.dump(list(trained_files), f, ensure_ascii=False, indent=2)

def append_log(dataset_name):
    """Tulis log history training dengan timestamp."""
    ts = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    with open(TRAINED_LOG, "a", encoding="utf-8") as f:
        f.write(f"[{ts}] Trained on {dataset_name}\n")

def load_qa_pairs(file_path):
    """Load QA pairs dari file JSON."""
    with open(file_path, "r", encoding="utf-8") as f:
        qa_pairs = json.load(f)

    train_examples = []
    for item in qa_pairs:
        qa_text = item["qa"]
        if "Q:" in qa_text and "A:" in qa_text:
            try:
                question = qa_text.split("Q:")[1].split("A:")[0].strip()
                answer = qa_text.split("A:")[1].strip()
                if question and answer:
                    train_examples.append(InputExample(texts=[question,
answer], label=1.0))
            except Exception:
                continue
    return train_examples

def main():
    all_datasets = [
        os.path.join(DATASET_DIR, f)
        for f in os.listdir(DATASET_DIR)
        if f.startswith("train_") and f.endswith(".json")
    ]

    trained_files = load_trained_files()

    new_datasets = [f for f in all_datasets if os.path.basename(f) not in
trained_files]

    if not new_datasets:
        print("Tidak ada dataset baru untuk dilatih. Model sudah up-to-date.")
        return

    print(f"Ditemukan {len(new_datasets)} dataset baru untuk training:" )
    for f in new_datasets:
        print("   -", os.path.basename(f))

    if os.path.exists(FINAL_MODEL_DIR):
        print(f"Load existing fine-tuned model dari {FINAL_MODEL_DIR}")
        model = SentenceTransformer(FINAL_MODEL_DIR)
    else:
        print(f"Load base model {BASE_MODEL}")
        model = SentenceTransformer(BASE_MODEL)

    for dataset_file in new_datasets:
        dataset_name = os.path.basename(dataset_file)
        print(f"\nTraining dengan {dataset_name}")

        train_examples = load_qa_pairs(dataset_file)
```

```
        print(f"   → {len(train_examples)} QA pairs")

        if not train_examples:
            print("Tidak ada data valid, skip.")
            trained_files.add(dataset_name)
            continue

        train_dataloader     =     DataLoader(train_examples,     shuffle=True,
batch_size=16)
        train_loss = losses.MultipleNegativesRankingLoss(model)

        num_epochs = 10
        warmup_steps = int(len(train_dataloader) * num_epochs * 0.1)

        model.fit(
            train_objectives=[(train_dataloader, train_loss)],
            epochs=num_epochs,
            warmup_steps=warmup_steps,
            output_path=TEMP_MODEL_DIR
        )

        if os.path.exists(FINAL_MODEL_DIR):
            shutil.rmtree(FINAL_MODEL_DIR)
        shutil.move(TEMP_MODEL_DIR, FINAL_MODEL_DIR)

        print(f"Training selesai untuk {dataset_name}, model updated!")

        trained_files.add(dataset_name)
        save_trained_files(trained_files)

        append_log(dataset_name)

    print("\nSemua   dataset   baru   sudah   selesai   dilatih   dan   dicatat   di
trained.json")


if __name__ == "__main__":
    main()
```

Dengan menggunakan *code* tersebut, setiap pasangan QA yang sebelumnya telah dibuat dilatihkan terhadap model *embedding*. Di dalam *code* terdapat fungsi utama `main` yang nantinya juga membutuhkan beberapa fungsi tambahan yaitu `load_trained_files`, `save_trained_files`, `append_log`, dan `load_qa_pairs`. Pertama-tama muat terlebih dahulu data seluruh file QA yang akan digunakan untuk melatih model, yaitu file dengan awalan nama "train_" dan ekstensinya *json*. Selanjutnya, gunakan fungsi `load_trained_files` untuk juga memuat data file mana saja yang sudah pernah digunakan untuk melatih model *embedding*. Setelah data file *training* dan juga data file yang sudah pernah digunakan untuk melatih berhasil didapat, selanjutnya cari apakah dari data file *training* tersebut ada file yang belum digunakan untuk melatih model *embedding*. Biasanya saat awal pertama kali akan membuat keseluruhan rangkaian RAG maka isi dari file "trained.json" juga akan kosong yang berarti model belum pernah dilatih dengan menggunakan data *training*. Setelah didapat file *training* mana saja yang akan digunakan

untuk melatih model (belum pernah dilatihkan), selanjutnya muat model *embedding* yang akan digunakan.

Jika ini adalah pertama kali model dilatih, maka *code* akan memuat dan mengambil model langsung dari *platform* Hugging Face, namun jika tidak maka model akan dimuat dari folder yang ada di dalam proyek. Selanjutnya untuk setiap file *training* baru, muat pasangan pertanyaan dan jawaban dari masing-masing file menggunakan fungsi `load_qa_pairs`. Fungsi `load_qa_pairs` akan memparsing isian dari masing-masing file QA untuk diambil pertanyaan dan jawabannya kemudian dilatihkan pada model sebanyak 10 *epoch* untuk setiap `train_example` yang dihasilkan dari pasangan QA tiap file. Setelah pelatihan berhasil dilakukan terhadap semua file maka selanjutnya nama setiap file yang telah berhasil dilatihkan pada model akan ditambahkan dan disimpan di dalam file "trained.json" menggunakan fungsi `save_trained_files`. Selain itu pelatihan yang berhasil dilakukan juga akan dicatat dalam log dengan menggunakan fungsi `append_log` yang juga akan mencatat waktu dan tanggal pelatihan yang memungkinkan nantinya dilakukan pelacakan jika terjadi kesalahan pelatihan. Model yang telah dilatih disimpan pada direktori tempat menyimpan model dan menghapus model yang telah ada jika sebelumnya telah melatih model. Sehingga di akhir setelah keseluruhan *code* ini dijalankan, model telah menjalani *domain adaptive training* terhadap sumber pengetahuan yang akan digunakan nantinya.

Langkah selanjutnya adalah melakukan proses *embedding* terhadap *chunk* dari dokumen-dokumen SOP menggunakan model *embedding* yang telah dilakukan *fine tuning* terhadapnya. Proses *embedding* dilakukan dengan menggunakan implementasi *code* python, yang ditunjukkan pada *Source Code* 4.4.

Source Code 4.4. Implementasi Proses *Embedding*

```
import argparse
import os
import shutil
import json
import subprocess
from langchain_community.document_loaders import PyPDFDirectoryLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain.schema.document import Document
from langchain_chroma import Chroma
from langchain_huggingface import HuggingFaceEmbeddings

CHROMA_PATH = "chroma"
DATA_PATH = "data"
DATASET_PATH = "datasets"
TRAINED_JSON = os.path.join(DATASET_PATH, "trained.json")
EMBEDDING_MODEL_PATH = "./indo_finetuned_embedding"
```

```
def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--reset",    action="store_true",    help="Reset    the
database.")
    args = parser.parse_args()
    if args.reset:
        print("Clearing Database")
        clear_database()

    documents = load_documents()
    chunks = split_documents(documents)
    add_to_chroma(chunks)


def load_documents():
    document_loader = PyPDFDirectoryLoader(DATA_PATH)
    return document_loader.load()


def split_documents(documents: list[Document]):
    text_splitter = RecursiveCharacterTextSplitter(
        chunk_size=1700,
        chunk_overlap=100,
        length_function=len,
        is_separator_regex=False,
    )
    return text_splitter.split_documents(documents)


def add_to_chroma(chunks: list[Document]):
    embedding_function = HuggingFaceEmbeddings(model_name=EMBEDDING_MODEL_PATH)

    db = Chroma(
        persist_directory=CHROMA_PATH,
        embedding_function=embedding_function,
        collection_metadata={"hnsw:space": "cosine"}
    )

    chunks_with_ids = calculate_chunk_ids(chunks)

    existing_items = db.get(include=[])
    existing_ids = set(existing_items["ids"])
    print(f"Number of existing documents in DB: {len(existing_ids)}")

    docs_grouped = {}
    for chunk in chunks_with_ids:
        src = os.path.basename(chunk.metadata.get("source", ""))
        docs_grouped.setdefault(src, []).append(chunk)

    if os.path.exists(TRAINED_JSON):
        with open(TRAINED_JSON, "r", encoding="utf-8") as f:
            trained_files = set(json.load(f))
    else:
        trained_files = set()

    for doc_name, doc_chunks in docs_grouped.items():
        print(f"\nProcessing {doc_name} ...")
        new_chunks  =  [c  for  c  in  doc_chunks  if  c.metadata["id"]  not  in
existing_ids]

        if not new_chunks:
            print(f"{doc_name} already in DB, skipping...")
            continue

        train_file = f"train_{os.path.splitext(doc_name)[0]}.json"
```

```
        if train_file not in trained_files:
            print(f"{doc_name} not yet trained → Generating QA & Fine-tuning...")
            subprocess.run(["python",  "generate_qa.py",  "--doc",  doc_name],
check=True)
            subprocess.run(["python", "finetune_embeddings.py"], check=True)
            embedding_function                                                  =
HuggingFaceEmbeddings(model_name=EMBEDDING_MODEL_PATH)
            db = Chroma(
                persist_directory=CHROMA_PATH,
                embedding_function=embedding_function
            )
        else:
            print(f"{doc_name} already trained, skipping training...")

        print(f"Adding {len(new_chunks)} chunks from {doc_name} to DB")
        new_chunk_ids = [chunk.metadata["id"] for chunk in new_chunks]
        db.add_documents(new_chunks, ids=new_chunk_ids)


def calculate_chunk_ids(chunks):
    last_page_id = None
    current_chunk_index = 0

    for chunk in chunks:
        source = chunk.metadata.get("source")
        page = chunk.metadata.get("page")
        current_page_id = f"{source}:{page}"

        if current_page_id == last_page_id:
            current_chunk_index += 1
        else:
            current_chunk_index = 0

        chunk_id = f"{current_page_id}:{current_chunk_index}"
        last_page_id = current_page_id

        chunk.metadata["id"] = chunk_id

    return chunks


def clear_database():
    if os.path.exists(CHROMA_PATH):
        shutil.rmtree(CHROMA_PATH)


if __name__ == "__main__":
    main()
```

Proses pembuatan representasi vektor dari *chunk* yang ada dilakukan dengan melakukan pemanggilan fungsi `main`. Di dalam fungsi `main` terdapat sebuah algoritma yang nantinya akan digunakan untuk melakukan *reset* pada isian database jika pada pemanggilan fungsi `main` diberikan sebuah argumen "--reset". Selanjutnya akan dipanggil fungsi `load_documents` yang sama dengan yang pernah digunakan sebelumnya, dimana fungsi ini akan menggunakan fungsi `PyPDFDirectoryLoader` bawaan *library* untuk memuat data kemudian disimpan dalam variabel `document`. Setelah data berhasil dimuat, dilakukan proses *chunking* menggunakan fungsi `split_documents` dengan konfigurasi yang sama

dengan fungsi `split_documents` yang telah dibahas sebelumnya yaitu membagi dokumen menjadi *chunk* dengan panjang maksimal 1700 karakter dan *overlap* sebanyak 100 karakter. Hasil dari proses *chunking* disimpan dalam sebuah variabel bernama `chunks`. Bagian terakhir pada fungsi `add_to_chroma` memanggil fungsi inti dari proses *embedding* ini yang juga akan sekaligus menyimpan hasilnya ke dalam vektor database.

Fungsi `add_to_chroma` menerima parameter berupa *chunk* yang bertipe *list* dari *document*. Di bagian awal fungsi ini, panggil fungsi yang akan digunakan untuk *embedding* dengan memanggilnya menggunakan fungsi `HuggingFaceEmbeddings` dan parameternya isikan dengan lokasi model yang telah di *fine-tuning* sebelumnya. Lanjutkan dengan menginisiasi *database* menggunakan Chroma yang nantinya akan disimpan pada folder "Chroma". Database Chroma dikonfigurasi untuk menyimpan data dalam bentuk vektor menggunakan *embedding function* yang telah disimpan sebelumnya. Untuk parameter `collection_metadata` diisiikan dengan key dan value ""hnsw:space": "cosine"" yang berarti penyimpanan dan pencarian di dalam *database* Chroma nantinya akan menggunakan basis jarak kosinus di index *Hierarchical Navigable Small World* (HNSW). HNSW adalah algoritma populer dan efisien untuk melakukan pencarian *nearest neighbor search*. HNSW dipilih karena biasanya dalam penyimpanan data vektor berdimensi cukup tinggi seperti yang dihasilkan dari fungsi *embedding*, pencarian untuk membandingkan vektor *query* dengan setiap vektor data (*brute force*) akan memakan waktu terlalu lama. HNSW mengatasi hal ini dengan membangun sebuah graf *multi-layer*.

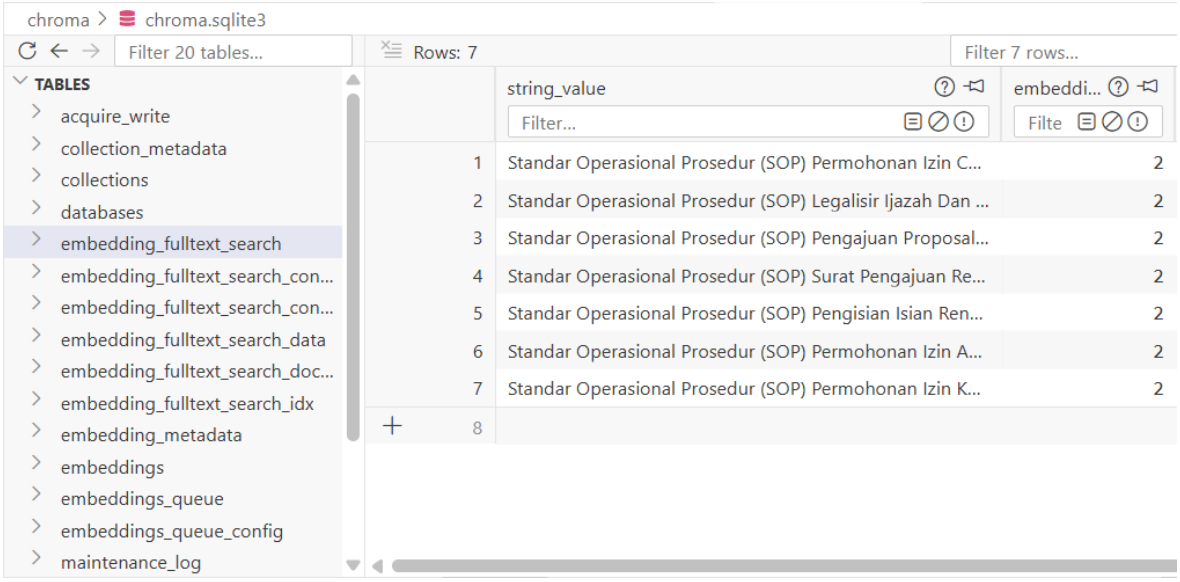
Tahap selanjutnya adalah memanggil *helper function* `calculate_chunk_ids` yang akan digunakan untuk menghasilkan metadata "id" untuk setiap *chunk*. Fungsi akan memberikan id untuk setiap *chunk* yang ada dengan "id" berupa "<source>:<page>:<chunk_index>" dimana chunk_index menandakan urutan chunk pada halaman yang sama. *Source* dan *page* diperoleh dari metadata yang sebelumnya telah dibawa oleh setiap *chunk*. Contoh dari hasil dari "id" *chunk* adalah "data\SOP_Pengajuan_Proposal_Kegiatan_Organisasi_Mahasiswa.pdf". Selanjutnya ambil daftar *knowledge* yang sudah tersimpan dalam *database* menggunakan fungsi get dan simpan id di variabel `existing_ids`. Setelah semua data yang ada dalam *database* telah tersimpan dalam variabel, selanjutnya lakukan pengelompokan terhadap *chunk vector* yang ada berdasarkan sumber file. Data sumber file diperoleh dari metadata *source* yang dibawa oleh masing-masing *chunk vector*. Selanjutnya muat file "trained.json" yang menyimpan data

nama-nama file yang sudah digunakan untuk melatih model *embedding*. File ini dimuat dan disimpan dalam variabel `trained_file` karena nantinya akan digunakan untuk memeriksa apakah sebuah file QA sudah digunakan untuk melakukan *fine-tuning*.

Tahapan selanjutnya adalah melakukan *loop* pada setiap data *chunk* yang ada. Lalu periksa apakah "id" *chunk* tersebut sudah ada di data "id" dari dalam *database*. Jika ada maka berarti *chunk* tersebut akan dilewati dan berarti sudah tersimpan di dalam *database*. Namun jika ternyata *chunk* tersebut belum ada di dalam database, maka gunakan *sub-process* dan jalankan *file* "generate_qa.py" untuk menghasilkan data pasangan pertanyaan dan jawaban untuk pelatihan lalu selanjutnya buat *sub-process* lagi untuk menjalankan *file* "finetune_embeddings.py". Tidak berhenti sampai disini, sesaat setelah model *embedding* mengalami pembaharuan maka muat ulang model *embedding* dengan menggunakan pemanggilan fungsi yang sama saat model dimuat di awal fungsi. Setelah itu lakukan pembuatan "id" untuk *chunk* yang baru dan tambahkan data *chunk* bersama metadatanya ke dalam *database* Chroma yang digunakan. Database yang digunakan untuk penyimpanan *knowledge* pada penelitian ini adalah sqlite3 dengan tipe data Chroma sebagai penyimpanan data vektor semantik. Hingga tahapan ini, data vektor semantik dari semua sumber pengetahuan yang digunakan sudah disimpan dalam *database* dan siap digunakan dalam pencarian.

4.1.3. *Knowledge Store*

Penyimpanan sumber pengetahuan yang akan digunakan pada *chatbot* dilakukan dengan menggunakan *database* Chroma yang tersimpan di dalam ekstensi sqlite3 sebagai database lokal. Dengan menggunakan fungsi yang telah dibahas pada tahapan sebelumnya, fungsi pencarian dalam *database* diatur untuk menggunakan perhitungan *cosine similarity*. Dengan tersimpannya semua sumber pengetahuan yang dibutuhkan pada Chroma maka data pengetahuan sudah siap digunakan sebagai sumber pengetahuan eksternal untuk menjawab pertanyaan pengguna. Adapun *preview database* yang dihasilkan dapat dilihat pada Gambar 4.16.



Gambar 4.1. *Preview* Struktur *Knowledge Database* yang dihasilkan

Pada tahapan selanjutnya, untuk setiap proses pencarian informasi maka proses pencarian informasi yang relevan akan dilakukan terhadap *vector database* dengan berdasar pada rumus *cosine similarity* antara *query* yang diberikan dan sumber pengetahuan yang ada di dalam *database*.

4.1.4. *Data Retrieval*

Tahap selanjutnya adalah *data retrieval* yang menjadi tahapan inti dalam *pipeline* RAG. Dimana pada tahapan ini akan dilakukan pencarian informasi yang relevan dengan *query* yang diberikan oleh user. Perlu diingat kembali bahwa inti dari *pipeline* RAG adalah bagaimana cara yang dapat dilakukan untuk memberikan model LLM sebuah informasi yang relevan terhadap pertanyaan atau pernyataan dari pengguna dari sumber pengetahuan eksternal yang sering disimpan ke dalam *vector database*. Selanjutnya setelah ini akan dibahas secara runtut tahapan dari awal bagaimana *user* memberikan *query* dan kemudian diproses oleh serangkaian tahapan dalam *pipeline* RAG untuk mendapatkan informasi yang relevan dari sumber informasi.

Tahapan dimulai ketika *user* berinteraksi melalui *user interface chatbot* dengan mengetikkan pertanyaan atau pernyataan. Hal tersebut kemudian memicu bagian *back-end* dari sistem untuk memanggil sebuah fungsi seperti yang ditunjukkan pada *Source Code* 4.5.

Source Code 4.5. Implementasi Pemrosesan *User Query*

```
import argparse
from langchain_chroma import Chroma
from langchain_huggingface import HuggingFaceEmbeddings
from sklearn.metrics.pairwise import cosine_similarity

CHROMA_PATH = "chroma"
EMBEDDING_MODEL_PATH = "./indo_finetuned_embedding"

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("query_text", type=str, help="The query text.")
    args = parser.parse_args()
    query_text = args.query_text
    query_rag(query_text)


def query_rag(query_text: str):
    embedding_function = HuggingFaceEmbeddings(model_name=EMBEDDING_MODEL_PATH)
    db                 =                  Chroma(persist_directory=CHROMA_PATH,
embedding_function=embedding_function)

    query_embedding = embedding_function.embed_query(query_text)

    docs = db.similarity_search(query_text, k=3)

    results = []
    for doc in docs:
        doc_embedding = embedding_function.embed_query(doc.page_content)
        cosine_sim = cosine_similarity([query_embedding], [doc_embedding])[0][0]
        results.append((doc, cosine_sim))

    results.sort(key=lambda x: x[1], reverse=True)

    best_doc, best_score = results[0]
    threshold = 0.3

    if best_score < threshold:
        answer = "Maaf, saya tidak menemukan jawaban pada dokumen yang tersedia."
        coverage = 0
        sources = []
        response_time = 0.0
        print("\nJawaban:\n", answer)
        save_to_csv(query_text, answer, sources, response_time, coverage)
        return answer

    context_text = "\n\n---\n\n".join([doc.page_content for doc, _ in results])
```

Fungsi `main` akan dipanggil tepat sesaat setelah interaksi *user* dengan *interface chatbot* selesai dilakukan. Dari masukan yang diberikan oleh *user, query* yang diminta *user* akan diproses sedemikian rupa melewati semua proses yang sebelumnya juga diterapkan pada sumber pengetahuan. Dimulai dari tahapan pertama data masukan user yang sudah disimpan di dalam variabel `query_text` akan dimasukkan ke dalam fungsi `query_rag` untuk diproses. Tahap pertama yang dilakukan terhadap teks *query* tersebut adalah melakukan proses *embedding* menggunakan model yang sama dengan yang digunakan untuk merubah representasi sumber pengetahuan di fase pengembangan *knowledge store*. Pemanggilan fungsi *embedding* dilakukan dengan menggunakan fungsi `HuggingFaceEmbeddings` yang

akan mencari model pada lokasi yang telah ditentukan dalam variabel `EMBEDDING_MODEL_PATH`. Tahapan selanjutnya adalah memuat *vector database* yang ada dengan menggunakan fungsi pemanggilan Chroma dan memberikan lokasi *database* yang tepat pada variabel `CHROMA_PATH`. Setelah berhasil dimuat selanjutnya lakukan pencarian informasi paling relevan terhadap *user query* di dalam *database*. Pencarian dilakukan dengan menggunakan fungsi `similarity_search`. Fungsi tersebut memiliki dua buah parameter yaitu k yang diatur dengan besaran 3 dan juga teks yang akan dicari kesamaannya yaitu teks di dalam variabel `query_text`. Selanjutnya dari hasil fungsi tersebut akan diperoleh 3 dokumen yang memiliki skor kedekatan jarak terendah terhadap `query_text`.

Lebih lanjut dilakukan perhitungan *cosine similarity score* dari masing-masing dokumen yang diperoleh. Perhitungan dilakukan dengan menggunakan fungsi `cosine_similarity`. Dokumen/informasi hasil kemudian diurutkan dari yang paling tinggi skornya, setelah masing-masing dokumen/informasi dihitung jarak kesamaan kosinusnya terhadap *query*.Dan jika dokumen terbaikyang diperoleh memiliki nilai *cosine similarity* di bawah ambang batas (*threshold*) sebesar 0.30 maka *chatbot* akan memberikan pesan gagal yang dalam penelitian ini disampaikan dengan kalimat "Maaf, saya tidak menemukan jawaban pada dokumen yang tersedia.". Hal ini penting untuk mengurangi atau memangkas kemungkinan *chatbot* memberikan jawaban yang tidak sesuai (halu). Namun bagaimana jika skor tidak berada di bawah ambang batas? Maka informasi yang diperoleh dari *database* akan lanjut disimpan ke dalam variabel `context_text` untuk nantinya digunakan pada tahapan selanjutnya yaitu pada pembuatan jawaban oleh LLM. Contoh dokumen/informasi yang dikembalikan dari beberapa *user query* yang diujikan dapat dilihat pada Tabel 4.4.

Tabel 4.4. Contoh Hasil Proses *Data Retrieval*

No	*Query*	Dokumen yang Dikembalikan
1	Gimana caranya dapat surat rekomendasi beasiswa ?	Dokumen #1: Cosine Similarity: 0.6436 Sumber: data\SOP_Pengajuan_Rekomendasi_Beasiswa.pdf:0:0 Konten: Standar Operasional Prosedur (SOP) Surat Pengajuan Rekomendasi Beasiswa Dokumen ini berisi langkah -langkah yang dapat diikuti untuk mengajukan dan mendapatkan rekomendasi beasiswa dengan total mak... Panjang konten: 1129 karakter Dokumen #2: Cosine Similarity: 0.3776 Sumber: data\SOP_Permohonan_Izin_Keterlambatan_Pembayaran_UKT.pdf:0:0 Konten: Standar Operasional Prosedur (SOP) Permohonan Izin Keterlambatan Pembayaran UKT Dokumen ini berisi langkah -langkah yang dapat diikuti untuk memperoleh izin

No	*Query*	Dokumen yang Dikembalikan
		keterlambatan pembayaran UKT dengan tot... Panjang konten: 1414 karakter Dokumen #3: Cosine Similarity: 0.3542 Sumber: data\SOP_Pengisian_IRS.pdf:0:0 Konten: Standar Operasional Prosedur (SOP) Pengisian Isian Rencana Studi (IRS) Dokumen ini berisi langkah-langkah yang dapat diikuti untuk dapat mengisi Isian Rencana Studi (IRS) dengan total maksimal wakt... Panjang konten: 1188 karakter
2	Bagaimana langkah-langkah untuk dapat memperoleh izin aktif kembali setelah cuti ?	Dokumen #1: Cosine Similarity: 0.6577 Sumber: data\SOP_Permohonan_Izin_Aktif_Setelah_Cuti.pdf:0:0 Konten: Standar Operasional Prosedur (SOP) Permohonan Izin Aktif Setelah Cuti Dokumen ini berisi langkah -langkah yang dapat diikuti untuk dapat memperoleh izin aktif setelah cuti. 1. Mahasiswa membawa su... Panjang konten: 596 karakter Dokumen #2: Cosine Similarity: 0.5127 Sumber: data\SOP_Izin_Cuti_Akademik.pdf:0:0 Konten: Standar Operasional Prosedur (SOP) Permohonan Izin Cuti Akademik Dokumen ini berisi langkah -langkah yang dapat diikuti untuk memperoleh izin cuti akademik dengan total maksimal waktu pemrosesan m... Panjang konten: 1572 karakter Dokumen #3: Cosine Similarity: 0.3901 Sumber: data\SOP_Permohonan_Izin_Keterlambatan_Pembayaran_UKT.pdf:0:0 Konten: Standar Operasional Prosedur (SOP) Permohonan Izin Keterlambatan Pembayaran UKT Dokumen ini berisi langkah -langkah yang dapat diikuti untuk memperoleh izin keterlambatan pembayaran UKT dengan tot... Panjang konten: 1414 karakter
3	Bagaimana cara untuk melegalisir ijazah ?	Dokumen #1: Cosine Similarity: 0.4522 Sumber: data\SOP_Legalisir_Ijazah_Dan_Transkrip.pdf:0:0 Konten: Standar Operasional Prosedur (SOP) Legalisir Ijazah Dan Transkrip Dokumen ini berisi langkah -langkah yang dapat diikuti untuk dapat melalukan legalisir ijazah dan transkrip dengan total maksimal w... Panjang konten: 1639 karakter Dokumen #2: Cosine Similarity: 0.3049 Sumber: data\SOP_Pengisian_IRS.pdf:0:0 Konten: Standar Operasional Prosedur (SOP) Pengisian Isian Rencana Studi (IRS)

No	*Query*	Dokumen yang Dikembalikan
		Dokumen ini berisi langkah-langkah yang dapat diikuti untuk dapat mengisi Isian Rencana Studi (IRS) dengan total maksimal wakt... Panjang konten: 1188 karakter Dokumen #3: Cosine Similarity: 0.2954 Sumber: data\SOP_Pengajuan_Rekomendasi_Beasiswa.pdf:0:0 Konten: Standar Operasional Prosedur (SOP) Surat Pengajuan Rekomendasi Beasiswa Dokumen ini berisi langkah -langkah yang dapat diikuti untuk mengajukan dan mendapatkan rekomendasi beasiswa dengan total mak... Panjang konten: 1129 karakter
4	Dimana saya dapat mengetahui ruang kelas saya ?	Dokumen #1: Cosine Similarity: 0.1676 Sumber: data\SOP_Permohonan_Izin_Keterlambatan_Pembayaran_UKT.pdf:0:0 Konten: Standar Operasional Prosedur (SOP) Permohonan Izin Keterlambatan Pembayaran UKT Dokumen ini berisi langkah -langkah yang dapat diikuti untuk memperoleh izin keterlambatan pembayaran UKT dengan tot... Panjang konten: 1414 karakter Dokumen #2: Cosine Similarity: 0.1675 Sumber: data\SOP_Pengisian_IRS.pdf:0:0 Konten: Standar Operasional Prosedur (SOP) Pengisian Isian Rencana Studi (IRS) Dokumen ini berisi langkah-langkah yang dapat diikuti untuk dapat mengisi Isian Rencana Studi (IRS) dengan total maksimal wakt... Panjang konten: 1188 karakter Dokumen #3: Cosine Similarity: 0.1613 Sumber: data\SOP_Legalisir_Ijazah_Dan_Transkrip.pdf:0:0 Konten: Standar Operasional Prosedur (SOP) Legalisir Ijazah Dan Transkrip Dokumen ini berisi langkah -langkah yang dapat diikuti untuk dapat melalukan legalisir ijazah dan transkrip dengan total maksimal w... Panjang konten: 1639 karakter

4.1.5. LLM *Generation*

Proses LLM *generation* adalah proses dimana 3 *chunk* teratas yang diperoleh dari tahapan *data retrieval* dapat dimanfaatkan oleh LLM untuk membantu menjawab pertanyaan dari *user*. Proses dimulai dengan memasukkan tiga *chunk* informasi/dokumen ke dalam format *prompt* yang telah dibuat. Dalam penelitian ini *prompt* yang digunakan ditunjukkan oleh Source Code 4.6.

Source Code 4.6. Implementasi *Prompt Template*

```
PROMPT_TEMPLATE = """
Anda adalah asisten layanan mahasiswa Fakultas Sains dan Matematika Universitas
Diponegoro
yang membantu menjawab pertanyaan berdasarkan dokumen resmi kampus.
Jawablah dengan bahasa Indonesia yang jelas, singkat, dan sopan.
Jika jawaban tidak ada dalam konteks, katakan dengan jujur
bahwa informasi tersebut tidak tersedia. JANGAN gunakan bahasa inggris dalam
memberi respon.

Konteks:
{context}

---

Pertanyaan: {question}

Jawaban sebagai asisten layanan mahasiswa:
"""
```

Prompt diatur agar memiliki persona sistem sebagai asisten layanan mahasiswa FSM Undip serta mengharuskan LLM nantinya memberikan jawaban dalam bahasa Indonesia. Dalam *prompt* juga telah dilakukan pencegahan halusinasi dengan menuliskan bahwa jika tidak terdapat jawaban dalam konteks maka model harus menjawab apa adanya (jika tidak ada maka katakan tidak ada). Hal tersebut dibuat sebagai *second layer anticipation* terhadap kemungkinan halusinasi yang dihasilkan. Lalu dapat dilihat juga bahwa di dalam *prompt* terdapat dua variabel yang dapat diisikan yaitu *context* dan juga *question*. *Context* akan diisikan dengan 3 *chunk* yang telah didapatkan dari tahapan sebelumnya, sementara *question* akan diisikan dengan pertanyaan yang diajukan oleh *user*. Proses bagaimana semuanya itu dapat dilakukan akan ditunjukkan pada *Source Code* 4.7.

Source Code 4.7. Implementasi Bagaimana LLM Menghasilkan Jawaban

```
import argparse
import os
import time
import csv
from dotenv import load_dotenv
from langchain_chroma import Chroma
from langchain.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from langchain_huggingface import HuggingFaceEmbeddings
from sklearn.metrics.pairwise import cosine_similarity

load_dotenv()

CHROMA_PATH = "chroma"
EMBEDDING_MODEL_PATH = "./indo_finetuned_embedding"
CSV_PATH = "results.csv"


PROMPT_TEMPLATE = """
Anda adalah asisten layanan mahasiswa Fakultas Sains dan Matematika Universitas
Diponegoro
yang membantu menjawab pertanyaan berdasarkan dokumen resmi kampus.
```

```
Jawablah dengan bahasa Indonesia yang jelas, singkat, dan sopan.
Jika jawaban tidak ada dalam konteks, katakan dengan jujur
bahwa informasi tersebut tidak tersedia. JANGAN gunakan bahasa inggris dalam
memberi respon.

Konteks:
{context}

---

Pertanyaan: {question}

Jawaban sebagai asisten layanan mahasiswa:
"""


def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("query_text", type=str, help="The query text.")
    args = parser.parse_args()
    query_text = args.query_text
    query_rag(query_text)


def query_rag(query_text: str):
    embedding_function = HuggingFaceEmbeddings(model_name=EMBEDDING_MODEL_PATH)
    query_embedding = embedding_function.embed_query(query_text)
    db                 =                 Chroma(persist_directory=CHROMA_PATH,
embedding_function=embedding_function)

    docs = db.similarity_search(query_text, k=3)

    results = []
    for doc in docs:
        doc_embedding = embedding_function.embed_query(doc.page_content)
        cosine_sim = cosine_similarity([query_embedding], [doc_embedding])[0][0]
        results.append((doc, cosine_sim))

    results.sort(key=lambda x: x[1], reverse=True)

    best_doc, best_score = results[0]
    threshold = 0.3

    if best_score < threshold:
        answer = "Maaf, saya tidak menemukan jawaban pada dokumen yang tersedia."
        coverage = 0
        sources = []
        response_time = 0.0
        print("\nJawaban:\n", answer)
        save_to_csv(query_text, answer, sources, response_time, coverage)
        return answer

    context_text = "\n\n---\n\n".join([doc.page_content for doc, _ in results])

    prompt_template = ChatPromptTemplate.from_template(PROMPT_TEMPLATE)
    prompt = prompt_template.format(context=context_text, question=query_text)

    model = ChatOpenAI(
        model="gpt-5",
        api_key=os.environ["GITHUB_TOKEN"],
        base_url="https://models.inference.ai.azure.com"
    )

    start_time = time.time()
    response = model.invoke(prompt)
    end_time = time.time()
    response_time = end_time - start_time
```

```
    response_text = response.content
    coverage = 1

    sources = [doc.metadata.get("id", None) for doc, _ in results]

    print(response_text)
    print(f"\nAnswered in : {response_time:.2f} sec")

    save_to_csv(query_text, response_text, sources, response_time, coverage)

    return response_text


def save_to_csv(question: str, answer: str, sources: list, response_time: float,
coverage: int):
    """Simpan hasil query ke file CSV"""
    file_exists = os.path.isfile(CSV_PATH)

    with open(CSV_PATH, mode="a", newline="", encoding="utf-8") as f:
        writer = csv.writer(f)

        if not file_exists:
            writer.writerow(["Pertanyaan", "Jawaban", "Sumber", "Response Time
(sec)", "Coverage"])

        writer.writerow([question,       answer,       ";       ".join(sources),
f"{response_time:.2f}", coverage])


if __name__ == "__main__":
    main()
```

Sebagian tahapan yang ada pada fungsi `main` adalah tahapan pada bagian *data retrieval* dan telah berhasil dijalankan hingga menghasilkan 3 *chunk* paling relevan dari sumber pengetahuan yang dimiliiki. Tahapan dilanjutkan dengan menginisialisasi template *prompt* yang sebelumnya telah dibuat dan menyimpannya ke dalam variabel `prompt_template` yang kemudian digunakan untuk membuat objek `prompt` yang dibuat dengan variabel `prompt_template` dan didalamnya diberikan 2 parameter yaitu `context` yang diisikan dengan variabel `context_text` yang diperoleh dari tahapan *data retrieval* dan `question` yang diisikan dengan variabel `query_text` yang adalah masukan dari *user*. Selanjutnya lakukan inisialisasi model LLM yang akan digunakan. Inisiasi model dilakukan dengan menggunakan fungsi `ChatOpenAI` yang kemudian disesuaikan variabel modelnya dengan jenis model yaitu gpt-5. Model tersebut diperoleh dari laman https://ai.azure.com/catalog.

Setelah model berhasil diinisialisasi selanjutnya tetapkan beberapa variabel yang akan berguna dalam pengecekan performa. Variabel tersebut adalah `start_time` yang dipanggil tepat sesaat setelah model diinisialisasi dan `end_time` yang dipanggil sesaat respon sudah dibuat oleh model. Kedua variabel tersebut digunakan dalam perhitungan `response_time` untuk mengetahui berapa lama waktu yang dibutuhkan untuk model. Selain itu variabel

`coverage` juga akan diatur dengan nilai 1 karena model berhasil memberikan jawaban. Pada tahap *development* jawaban juga akan disimpan ke dalam file "result.csv" menggunakan fungsi `save_to_csv` yang akan disimpan bersama *response time* dan dokumen yang menjadi sumber dari jawaban tersebut. File ini yang nantinya akan digunakan pada tahapan evaluasi.

4.1.6. *Evaluation Process*

Pada tahapan ini, model *chatbot* yang sudah dihasilkan akan diujikan dengan set pertanyaan yang diperoleh dari proses wawancara dengan staf bidang akademik dan kemahasiswaan FSM. Daftar pertanyaan yang diperoleh dari tahapan wawancara dapat dilihat pada Lampiran 1. Pertanyaan tersebut kemudian diujikan satu persatu melalui tahapan LLM *generation*. Setiap jawaban yang dihasilkan oleh *chatbot* dari pertanyaan yang ada akan disimpan dalam file "result.csv". File tersebut berisi pertanyaan, jawaban yang dihasilkan oleh *chatbot*, sumber perolehan jawaban, waktu yang dibutuhkan *chatbot* untuk menjawab dan apakah *chatbot* mampu menjawab pertanyaan. File tersebut masih perlu diolah sekali lagi sebelum benar-benar dapat digunakan pada tahapan evaluasi. Perlu diingat bahwa evaluasi pada penelitian ini menggunakan dua buah metrik evaluasi yaitu ROUGE dan BERTScore. Kedua metrik tersebut membutuhkan jawaban asli dari pertanyaan (*ground truth*), sehingga pada tahapan ini file "result.csv" harus diubah sedikit dengan menggunakan aplikasi Microsoft Excel untuk dapat menambahkan satu kolom baru yaitu *Groundtruth*. *Groundtruth* dari setiap pertanyaan diambil dari hasil wawancara pada kolom "Jawaban". Setelah menambahkan kolom tersebut pada file "result.csv", simpan file dengan nama "responses_rag.csv" dengan encoding `utf-8-sig` dan separator ";". Setelah file "responses_rag.csv" berhasil didapatkan, lakukan pengujian menggunakan kedua metrik yaitu ROUGE dan BERTScore yang ditunjukkan dengan implementasi kode pada *Source Code* 4.8.

Source Code 4.8. Implementasi Metrik Evaluasi

```
import pandas as pd
from rouge_score import rouge_scorer
from bert_score import score

df = pd.read_csv("responses_rag.csv", encoding="utf-8-sig", sep=";")
references = df['Groundtruth'].tolist()
candidates = df['Jawaban'].tolist()

scorer = rouge_scorer.RougeScorer(['rouge1', 'rougeL'], use_stemmer=True)
rouge1_scores, rougeL_scores = [], []
for ref, cand in zip(references, candidates):
    scores = scorer.score(ref, cand)
    rouge1_scores.append(scores['rouge1'].fmeasure)
```

```
    rougeL_scores.append(scores['rougeL'].fmeasure)
df['ROUGE-1'] = rouge1_scores
df['ROUGE-L'] = rougeL_scores

P, R, F1 = score(candidates, references, lang="id", verbose=True)
df['BERTScore'] = F1.tolist()

print(df[['Pertanyaan','ROUGE-1','ROUGE-L','BERTScore']].head())
df.to_csv("evaluasi_chatbot.csv", index=False)
```

Dari kode tersebut dapat dilihat bahwa tahapan pertama yang harus dilakukan adalah memuat isi dari file "responses_rag.csv" kemudian menyimpannya dalam variabel `df`. Setiap baris jawaban dari *chatbot* disimpan pada variabel `candidates` dan *ground truth*-nya disimpan pada variabel `references`. Selanjutnya hitung nilai dari metrik ROUGE dengan menggunakan fungsi `ROUGEScorer` dan dengan mengaktifkan parameter *stemmer*. Untuk setiap hasil jawaban (`candidates`) dan *ground truth* (`references`), lakukan perhitungan ROUGE-1 dan ROUGE-L dan simpan sebagai kolom ke dalam *dataframe*. Berikutnya lakukan perhitungan untuk `R_BERT`, `P_BERT`, dan `F_BERT` (F1) untuk masing-masing pasangan `candidates` dan `references` lalu simpan hasil `F_BERT` sebagai kolom dalam *dataframe*. Terakhir, simpan keseluruhan *dataframe* ke dalam format *csv*. Hasil dari evaluasi menggunakan kedua metrik tersebut dapat dilihat pada Tabel 4.5, sementara perbandingan hasil jawaban dan *ground truth*-nya dapat dilihat pada Lampiran 4.

Tabel 4.5. Hasil Evaluasi Model RAG Terhadap *Frequently Asked Question*

No	Pertanyaan	Response Time (Sec)	Coverage	ROUGE -1	ROUGE -L	BERT Score
1	Bagaimana proses pengajuan cuti akademik ?	16.4	1	0.8161	0.7816	0.8896
2	Berapa lama waktu yang dibutuhkan untuk dapat melakukan legalisir ijazah dan atau transkrip ?	10.53	1	0.3125	0.25	0.7355
3	Berapa jumlah maksimal lembar legalisir yang dapat mintakan untuk ijazah dan atau transkrip ?	12.65	1	0.3889	0.3333	0.7257
4	Persyaratan apa yang diperlukan apabila pengambilan legalisir ijazah dan atau transkrip diwakilkan oleh pihak lain ?	18.26	1	0.1212	0.0606	0.6731
5	Bagaimana prosedur dan apa syarat yang diperlukan untuk dapat melakukan pengajuan legalisir ijazah dan atau transkrip ?	30.42	1	0.6379	0.5183	0.7934
6	Apa syarat mengajukan proposal kegiatan yang terkait dengan orientasi mahasiswa baru ?	17.38	1	0.0842	0.0842	0.6571
7	Bagaimana prosedur yang dapat dilakukan untuk mendapatkan rekomendasi beasiswa ?	21.3	1	0.5497	0.4678	0.7625

No	Pertanyaan	Response Time (Sec)	Coverage	ROUGE -1	ROUGE -L	BERT Score
8	Bagaimana jika mata kuliah yang ingin diambil belum muncul di IRS ?	36.49	1	0.0290	0.0290	0.6281
9	Bagaimana jika saya terlambat melakukan pembayaran UKT dan pengisian IRS?	58.78	1	0.5625	0.4375	0.7978
10	Saya sempat cuti, lalu bagaimana prosedurnya untuk saya dapat aktif kembali dalam akademik ?	51.34	1	0.2178	0.2178	0.7215

Dari tabel hasil evaluasi dapat diketahui performa model berdasarkan ketiga metrik evaluasi yang digunakan, yaitu `ROUGE-1`, `ROUGE-L`, dan `BERTScore`. Berdasarkan pengujian terhadap 10 set pertanyaan, model memperoleh rata-rata nilai `ROUGE-1` sebesar 0.37 dan `ROUGE-L` sebesar 0.31. Nilai `ROUGE-1` menunjukkan bahwa terdapat kesamaan sebesar 37% pada tingkat *unigram* antara jawaban yang dihasilkan oleh model dengan *ground truth*. Hal ini mengindikasikan bahwa model telah mampu memilih kata-kata yang relevan dengan konteks pertanyaan, meskipun masih terdapat perbedaan dalam pemilihan atau penempatan kata yang menyebabkan tingkat kesamaan leksikal belum optimal.

Sementara itu, `ROUGE-L` dengan nilai rata-rata 0.31 menggambarkan tingkat kesesuaian urutan kata LCS (*Longest Common Subsequence*) antara jawaban model dengan *ground truth*. Nilai ini menunjukkan bahwa struktur dan alur kalimat yang dihasilkan oleh *chatbot* sudah cukup konsisten dengan jawaban referensi, namun belum sepenuhnya identik. Rendahnya nilai `ROUGE-1` dan `ROUGE-L` ini disebabkan oleh perbedaan isi antara dokumen sumber pengetahuan yang digunakan dalam sistem RAG dengan jawaban *ground truth* hasil wawancara, di mana beberapa informasi yang menjadi kunci jawaban belum sepenuhnya tercakup dalam dokumen yang dimuat ke dalam basis pengetahuan model. Dengan demikian, meskipun model telah melakukan pencarian konteks yang relevan, hasil akhir belum sepenuhnya sesuai dengan jawaban sebenarnya karena keterbatasan konten sumber.

Pada metrik BERTScore, model memperoleh rata-rata nilai sebesar 0.73, yang menunjukkan bahwa jawaban yang dihasilkan model memiliki tingkat kemiripan makna yang cukup tinggi secara semantik dengan ground truth. Metrik ini memperhitungkan representasi konteks kata dalam ruang vektor semantik menggunakan model bahasa pra-latih, sehingga nilai yang relatif tinggi mengindikasikan bahwa model telah memahami makna pertanyaan dan mampu memberikan respons yang relevan secara konseptual, meskipun terdapat perbedaan dalam redaksi atau gaya penyampaian.

Dari sisi waktu respon, model membutuhkan waktu rata-rata 27.3 detik untuk menghasilkan jawaban terhadap setiap pertanyaan yang diajukan. Waktu ini mencakup proses pengambilan konteks (*retrieval*), penyusunan *prompt*, serta proses penghasilan jawaban oleh model LLM.

Berdasarkan hasil evaluasi *coverage*, seluruh variabel pengujian menunjukkan nilai satu (1), yang berarti model telah berupaya menjawab seluruh pertanyaan yang diberikan. Namun demikian, dalam beberapa kasus, *chatbot* tidak memberikan jawaban riil karena informasi yang dibutuhkan tidak tersedia dalam sumber dokumen. Pada kondisi tersebut, chatbot hanya mengembalikan pesan bahwa ia tidak dapat menjawab pertanyaan dan menyarankan pengguna untuk langsung menghubungi subbagian akademik dan kemahasiswaan. Hal ini kembali menunjukkan bahwa keterbatasan cakupan dokumen sumber berpengaruh terhadap keluaran model, meskipun pipeline RAG secara teknis telah berjalan dengan baik.

Secara keseluruhan, hasil evaluasi menunjukkan bahwa model LLM GPT-5 dengan pipeline RAG telah mampu dengan cukup baik menghasilkan jawaban yang relevan terhadap pertanyaan pengguna, terutama dalam aspek pemahaman konteks dan makna jawaban. Dengan demikian, model ini dapat digunakan sebagai chatbot pendukung layanan akademik dan kemahasiswaan di FSM Undip. Hasil evaluasi ini juga akan menjadi dasar pembanding pada tahap komparasi model yang dibahas pada bagian selanjutnya dari penelitian.

4.2. Pengembangan *Chatbot* RASA

Pengembangan *chatbot* RASA terdiri dari beberapa tahapan yang dimulai dari tahapan persiapan data hingga evaluasi terhadap model *chatbot* RASA yang dihasilkan.

4.2.1. *Data Preparation*

Tahap pertama yang harus dilakukan dalam keseluruhan rangkaian pengembangan *chatbot* RASA adalah mempersiapkan ketujuh dokumen SOP yang akan digunakan sebagai sumber pengetahuan oleh *chatbot*. Proses dalam tahapan ini menjelaskan bagaimana dokumen pdf yang berfungsi sebagai sumber pengetahuan mentah dirubah ke dalam komponen yang diperlukan model RASA yaitu NLU, *rules*, *stories*, dan *domain*.

Komponen pertama yaitu NLU berisi daftar `intent` yang dibuat secara manual dengan mengidentifikasi topik atau tema pertanyaan apa saja yang mungkin ditanyakan oleh *user* kepada *chatbot* berdasarkan dokumen sumber pengetahuan. Setelah berhasil mengidentifikasi setiap `intent` yang mungkin diterima. Selanjutnya berikan contoh pertanyaan yang mungkin akan diajukan sesuai dengan domain bahasa yang digunakan. Pada penelitian kali ini, akan digunakan bahasa Indonesia sebagai domain bahasa utama. Oleh karenanya setiap contoh pertanyaan yang diberikan harus juga diberikan dalam bahasa Indonesia agar model dapat mengenal dan mempelajari dengan baik contoh pertanyaan yang akan diajukan oleh *user*. Setiap pertanyaan tersebut kemudian disimpan dalam variabel `examples` yang dimiliki oleh setiap `intent`. Selain mengidentifikasi dan mendefinisikan `intent` sesuai dengan domain sumber pengetahuan yang akan digunakan, dalam RASA dapat juga didefinisikan `intent` umum seperti sapaan pembuka, penutup, ucapan terima kasih, dan *fallback* yang biasanya ditulis menjadi sebuah `intent` dengan nama "nlu_fallback". Selain itu pada file NLU ("nlu.yml") dapat juga didefinisikan `synonym` untuk dapat memperjelas makna dari suatu kata atau mendefinisikan kata yang memiliki arti sama/mirip. Di dalam setiap `synonym` juga terdapat `examples` yang akan menyimpan semua kata dengan arti yang sama dengan kata aslinya. Daftar semua `intent` dan `synonym` yang didefinisikan dapat dilihat pada Lampiran 5 bagain 1. Sementara *Preview* implementasinya dalam *code* dapat dilihat pada gambar 4.2.

```yaml
- intent: tanya_jadwal
  examples: |
    - jam berapa buka layanan?
    - jadwal operasional akademik
    - kapan buka layanan?
    - jam kerja subbag akademik
    - hari apa saja buka?
    - jadwal pelayanan

- synonym: cuti
  examples: |
    - cuti akademik
    - izin cuti
    - cuti semester

- synonym: legalisir
  examples: |
    - legalisir ijazah
    - legalisir transkrip
    - pengesahan dokumen

- synonym: proposal
  examples: |
    - proposal kegiatan
    - proposal organisasi
    - pengajuan acara

- synonym: irs
  examples: |
    - isian rencana studi
    - krs
    - rencana studi
    - kartu rencana studi
```

Gambar 4.2. *Preview* Implementasi NLU *Component* dalam *Code*

Setelah selesai mendefinisikan setiap komponen NLU yang dibutuhkan, langkah selanjutnya yang harus dilakukan pada tahap *data preparation* adalah mendefinisikan `rules` yang disimpan didalam file "rules.yml". Isi komponen `rules` adalah aksi apa yang akan dilakukan dari setiap `intent` yang telah didefinisikan sebelumnya pada komponen NLU. Aksi yang akan dilakukan disimpan di dalam variabel `action`. Aksi yang akan dilakukan nanti disimpan dalam komponen domain. Keseluruhan komponen `rules` yang digunakan pada pengembangan *chatbot* ini dapat dilihat pada Lampiran 5 bagian 2. Sementara *preview* implementasi dalam codenya dapat dilihat pada gambar 4.3.

```yaml
- rule: Ucapkan terima kasih
  steps:
  - intent: terima_kasih
  - action: utter_terima_kasih

- rule: Respon pertanyaan cuti akademik
  steps:
  - intent: tanya_cuti_akademik
  - action: utter_jawab_cuti_akademik

- rule: Respon pertanyaan legalisir
  steps:
  - intent: tanya_legalisir
  - action: utter_jawab_legalisir

- rule: Respon pertanyaan proposal organisasi
  steps:
  - intent: tanya_proposal_organisasi
  - action: utter_jawab_proposal_organisasi

- rule: Respon pertanyaan rekomendasi beasiswa
  steps:
  - intent: tanya_rekomendasi_beasiswa
  - action: utter_jawab_rekomendasi_beasiswa

- rule: Respon pertanyaan IRS
  steps:
  - intent: tanya_irs
  - action: utter_jawab_irs

- rule: Respon pertanyaan izin aktif setelah cuti
  steps:
  - intent: tanya_izin_aktif_setelah_cuti
  - action: utter_jawab_izin_aktif_kuliah_setelah_cuti
```

Gambar 4.3. *Preview* Implementasi Komponen *Rules* dalam *Code*

Setelah mendefinisikan `rules` yang berlaku pada *chatbot* RASA, selanjutnya perlu dibuat contoh alur kasus yang akan terjadi pada percakapan nyata dengan *user*. Contoh alur ini disimpan ke dalam file "stories.yml" sebagai komponen `stories`. `Stories` akan memberikan panduan kepada model *chatbot* untuk dapat belajar bagaimana pola yang biasa terjadi dalam percakapan *user*. Daftar *stories* yang digunakan pada pengembangan *chatbot*

ini dapat dilihat pada Lampiran 5 bagian 3. Dan berikut ini pada gambar 4.4 adalah *preview* bagaimana implementasinya dalam *code*.

```
stories:
- story: Cuti akademik process
  steps:
  - intent: tanya_cuti_akademik
  - action: utter_jawab_cuti_akademik
  - intent: tanya_link_form
  - action: utter_jawab_link_form

- story: Legalisir process
  steps:
  - intent: tanya_legalisir
  - action: utter_jawab_legalisir
  - intent: tanya_waktu_proses
  - action: utter_jawab_waktu_proses

- story: Proposal organisasi process
  steps:
  - intent: tanya_proposal_organisasi
  - action: utter_jawab_proposal_organisasi

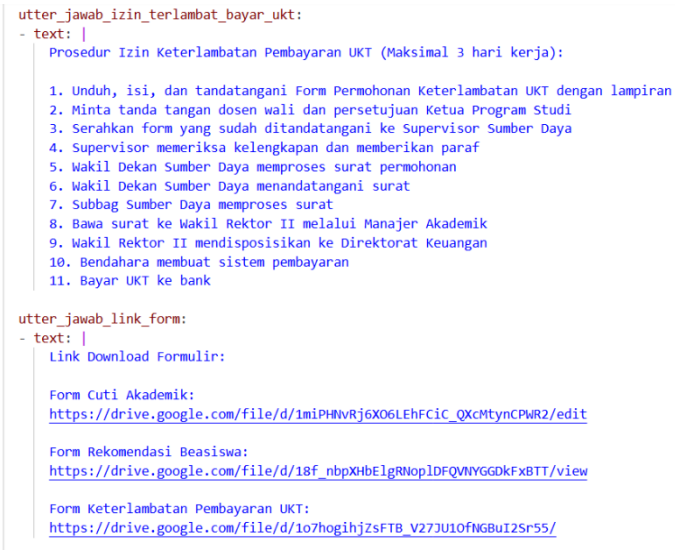
- story: Beasiswa process
  steps:
  - intent: tanya_rekomendasi_beasiswa
  - action: utter_jawab_rekomendasi_beasiswa
  - intent: tanya_link_form
  - action: utter_jawab_link_form

- story: IRS process
  steps:
  - intent: tanya_irs
  - action: utter_jawab_irs

- story: Aktif setelah cuti process
  steps:
  - intent: tanya_izin_aktif_setelah_cuti
  - action: utter_jawab_izin_aktif_kuliah_setelah_cuti
```

Gambar 4.4. *Preview* Implementasi Komponen *Stories* dalam *Code*

Setelah ketiga komponen tersebut diimplementasikan dengan baik pada *code*, selanjutnya buat komponen `domain` yang di dalamnya didefinisikan `intent` beserta `responses` yang akan mengatur bagaimana respon yang akan diberikan oleh model sesuai dengan `intent` yang berkaitan. Selain itu didalam file "domain.yml" juga didefinisikan `entities` yang akan mengatur potongan informasi spesifik yang bisa diekstrak dari tiap masukan dari *user*. Penggunaan `entities` ini digunakan bersamaan dengan `slots` yang juga didefinisikan sebagai tempat penyimpanan sementara dalam sesi percakapan. `Slots` akan menyimpan nilai dari `entity` layanan yang dikenali dalam `entities`. Terakhir lakukan pengaturan `session_config` yang bertujuan mengatur selama apa sesi akan disimpan dan apakah `slots` yang sudah terisi pada sesi sebelumnya akan dibawa ke sesi yang baru. Isi dari komponen `domain` secara lengkap dapat dilihat pada Lampiran 5 bagian 4, dan *preview* implementasi pada *code*-nya dapat dilihat pada Gambar 4.5.



Gambar 4.5. *Preview* Implementasi Komponen *Domain* dalam *Code*

Setelah semua komponen tersebut dibuat dan diimplementasikan maka semua sumber pengetahuan yang akan dibutuhkan dalam pelatihan *chatbot* berbasis RASA sudah terpenuhi dan dapat digunakan untuk melatih model nantinya pada tahapan *data training*.

4.2.2. *Data Training*

Tahapan *data training* dalam proses pengembangan *chatbot* RASA adalah sebuah tahapan dimana komponen-komponen yang telah disiapkan sebelumnya seperti `nlu`, `rules`, `stories` , dan pendefinisian `domain` akan digunakan untuk melatih model yang menerapkan prinsip NLU (klasifikasi `intent` + ekstraksi `entity` + *response retrieval*) dan manajamen dialog (menentukan aksi selanjutnya). Proses *training* akan dijalankan mengikuti alur yang didefinisikan pada variabel `pipeline` yang berisi bagian NLU. Selain itu hasil dari bagaimana model dialog yang akan digunakan juga akan didefinisikan pada variabel `policies`. Hasil dari proses pelatihan ini adalah sebuah model yang disimpan dalam format `.tar.gz` pada direktori "models". Pendefinisian dari komponen `pipeline` dan `policies` dalam file "config.yml" ditunjukkan pada *Source Code* 4.9.

Source Code 4.9. Pendefinisian Komponen *Pipeline* dan *Policies* dalam *Code*

```
language: id
assistant_id: 20250929-171821-brutal-git

pipeline:
- name: WhitespaceTokenizer
- name: RegexFeaturizer
- name: LexicalSyntacticFeaturizer
- name: CountVectorsFeaturizer
```

```
- name: CountVectorsFeaturizer
  analyzer: char_wb
  min_ngram: 1
  max_ngram: 4
- name: DIETClassifier
  epochs: 100
  constrain_similarities: true
- name: EntitySynonymMapper
- name: ResponseSelector
  epochs: 100
- name: FallbackClassifier
  threshold: 0.7
  ambiguity_threshold: 0.1

policies:
- name: MemoizationPolicy
- name: RulePolicy
- name: UnexpecTEDIntentPolicy
  max_history: 5
  epochs: 100
- name: TEDPolicy
  max_history: 5
  epochs: 100
  constrain_similarities: true
```

Di dalam *code* tersebut, dibagian awal didefinisikan bahwa bahasa yang akan digunakan dalam setiap tahapan/*pipeline* adalah bahasa Indonesia, misalnya dalam proses tokenisasi yang membutuhkan pendefinisian bahasa yang akan digunakan secara spesifik. Tidak lupa juga di bagian awal ini dibuat sebuah variabel `assistant_id` yang akan berguna untuk identifikasi/*versioning* proses *training*.

Selanjutnya, definisikan satu persatu komponen per komponen *pipeline* yang akan digunakan. Dalam proses pengembangan ini akan digunakan *pipeline* dengan urutan `WhitespaceTokenizer`, `RegexFeaturizer`, `LexicalSyntacticFeaturizer`, `CountVectorsFeaturizer`, `CountVectorsFeaturizer (char_wb ngrams)`, `DIETClassifier`, `EntitySynonymMapper`, `ResponseSelector`, `FallbackClassifier` sesuai yang tertulis dalam *code*. Berikut ini adalah penjelasan untuk masing-masing komponen.

1. Komponen `WhitespaceTokenizer`

   Adalah sebuah komponen yang berfungsi untuk memecah teks input dengan metode pemisahannya berdasarkan spasi.
2. Komponen `RegexFeaturizer`

   Berfungsi untuk menemukan token-token yang sesuai dengan ekspresi reguler yang didefinisikan pada *training data* misalnya saja nomor telepon FSM atau data lainnya

yang memiliki ekspresi reguler yang kemudian dilakukan *encoding* menjadi sebuah fitur.

3. Komponen `LexicalSyntacticFeaturizer`

   Mirip dengan `RegexFeaturizer` namun perbedaannya adalah `LexicalSyntacticFeaturizer` menghasilkan fitur berdasarkan komponen leksikal dan sintaksis pada level token (kata). Komponen leksikal yang dimaksud biasanya adalah seperti apakah kata tersebut hanya terdiri dari huruf (is_alpha) atau hanya terdiri dari digit (is_digit) dan apakah terdapat awalan atau akhiran dari kata tersebut. Hal ini bertujuan agar representasi token bisa lebih diperkaya dengan adanya petunjuk morfologis kata. Petunjuk morfologis dapat membantu membedakan kata kerja dari kata benda atau menunjukkan jenis *action* yang relevan untuk memahami *intent* yang dituju oleh *user*.

4. Komponen `CountVectorsFeaturizer`

   Komponen ini berfungsi untuk membuat token menjadi representasi *bag of words* atau fitur dalam bentuk vektor. Fitur ini akan berfungsi sebagai fitur tambahan untuk model DIET yang akan digunakan.

5. Komponen `CountVectorsFeaturizer (char_wb ngrams)`

   Komponen ini berfungsi untuk menghasilkan n-gram karakter (N karakter yang berdekatan) dengan batasan kata yang didefinisikan pada `char_wb` (*within word boundaries*). Nantinya frekuensi kemunculan dari setiap n-gram karakter yang ditemukan dalam teks akan dihitung dan dijadikan sebagai vektor fitur. Hal ini akan sangat penting jika berhadapan dengan jumlah data yang terbatas dan atau data yang digunakan mengandung banyak *noise*. Pada proses pengembangan ini digunakan parameter `min_gram` sebesar 1 dan `max_gram` sebesar 4 yang berarti jumlah minimum karakter yang akan dihitung adalah 1 dan maksimalnya adalah 4.

6. Komponen `DIETClassifier`

   DIET (*Dual Intent and Entity Transformer*) adalah sebuah model *transformer* ringan yang dirancang untuk secara bersamaan mempelajari dua hal dalam NLU yaitu *intent classification* dan *entity extraction*. Parameter yang digunakan pada komponen ini adalah `epochs` yang diisi dengan nilai 100 dan `constrain_similarities` yang diatur dengan nilai "true". Hal ini berarti model DIET akan dilatih dengan jumlah perulangan pelatihan sebanyak 100 kali. `Constrain_similarities` yang diatur sebagai "true" berarti pelatihan model menggunakan teknik regularisasi, dimana hal ini akan

membantu model untuk memperjelas pemisahan antar kategori dan meningkatkan generalisasi data yang belum pernah digunakan.

7. Komponen `EntitySynonymMapper`

   Komponen ini menggunakan sinonim yang telah didefinisikan dalam data pelatihan untuk memetakan atau mengganti berbagai bentuk penulisan dari sebuah entitas ke nilai kanonis. Misalnya saja kata "S1" akan dipetakan ke nilai kanonis "Sarjana".

8. Komponen `ResponseSelector`

   Komponen ini digunakan untuk melatih model memetakan masukan dari *user* ke respon yang telah disediakan dengan nilai parameter `epochs` sebesar 100. Hal ini sangat berguna dalam kasus yang melibatkan FAQ atau *retrieval intents* untuk mendapatkan *canned response* (respon siap pakai) tanpe perlu menggunakan model generatif.

9. Komponen `FallbackClassifier`

   Komponen ini berguna untuk mempelajari apakah suatu skor *confidence* yang dihasilkan dari *input* yang ada dikategorikan sebagai rendah atau diluar cakupan. Hal ini berguna dalam memicu *fallbacks*. Pada proses pengembangan ini digunakan batas ambang bawah `threshold` sebesar 0.7 yang berarti bahwa jika nilai *confidence* yang dihasilkan oleh model berada di bawah itu, maka *fallbacks* akan terpicu. Pada komponen ini juga diatur sebuah parameter `ambiguity_threshold` sebesar 0.1 yang artinya jika dua buah `intent` teratas memiliki perbedaan nilai *confidence* kurang dari 1% maka akan diperlakukan sebagai ambigu.

Selain *pipeline* pelatihan, dalam *code* tersebut juga didefinisikan bagaimana manajemen dialog yang akan digunakan. Berikut ini adalah penjelasan dari `policies` dan komponen di dalamnya.

1. Komponen `MemoizationPolicy`

   Jika percakapan yang dilakukan oleh *user* saat ini berjalan sama dengan contoh *story* yang ada pada file "stories.yml", maka lakukan aksi yang sama persis dengan yang ada pada *story* tersebut. Hal ini berguna untuk membantu memastikan bahwa setiap percakapan yang sama dan ada dalam alur *story*, model akan memberikan yang konsisten.

2. Komponen `RulePolicy`

   `RulePolicy` berfungsi untuk memastikan bahwa semua *rules* yang didefinisikan pada komponen `Rules` digunakan pada manajemen dialog. Jadi komponen ini berfungsi sebagai pemicu komponen `Rules` untuk digunakan pada model *chatbot*.

3. UnexpecTEDIntentPolicy

   Input dari *user* yang tidak sesuai dengan alur dialog atau `intent` yang ada akan diarahkan ke *fallback* menggunakan pendekatan neural. Parameter yang digunakan adalah `max_history` dengan nilai 5 yang artinya model akan melihat 5 event terakhir untuk dapat mengatakan apakah *input* dari *user* tidak sesuai dengan alur dialog yang ada. Sementara itu pada proses pelatihan komponen ini juga digunakan dengan iterasi sebanyak 100.

4. TEDPolicy

   Adalah sebuah sebuah aturan berbasis *transformer* yang akan mempelajari *embedding* untuk setiap kondisi dialog, aksi dan kemudian memprediksi aksi berikutnya. Komponen ini mengeneralisasi kondisi yang mirip dan bisa mengatasi hubungan yang cukup panjang tidak seperti *memoization*. Pada komponen ini juga digunakan parameter `max_history` sebesar 5, `epoch` sebesar 100, dan `constrain_similarities` di-*set* sebagai "true". TED digunakan agar model lebih tergeneralisasi dan dapat menangani konversasi yang mungkin belum ada pada pendefinisian alur yang dilakukan saat proses *training*.

Dengan adanya semua komponen tersebut baik pada `pipeline` maupun `policies` maka proses *training* dapat dilakukan. Proses ini dimulai dengan *pre-processing* dan *feature extraction*, dimana data yang ada akan ditransformasikan ke dalam format numerik yang dapat dipahami oleh model. `Tokenizer` akan memecah setiap data yang ada ke dalam bentuk token-token, dan kemudian `featurizer` akan mengubah token tersebut menjadi suatu fitur baru seperti jumlah kata, n-gram karakter, dan juga keterkaitan semantik. Barulah setelah ini proses *training* dimulai. Komponen utama yaitu DIET akan menggunakan fitur yang dihasilkan oleh `featurizer` untuk dilatih dalam menjawab dua task sekaligus yaitu *intent classification* dan *entity extraction*, dengan terus meminimalisasi kombinasi *loss function* yang digunakan. Dalam waktu bersamaan, `responseselector` juga dilatih dalam memasangkan *input* dari pengguna dengan respon yang tersedia. Selain itu `fallbackclassifier` juga dilatih untuk belajar mendeteksi *user input* yang memiliki

tingkat *confidence* rendah. Setelah proses pelatihan NLU selesai, proses pelatihan `policies` dilakukan. Pelatihan ini akan ,enggunakan alur *stories* yang ada dan juga *rules* yang sudah didefinisikan beserta dengan pelacak kondisi percakapan saat ini (yang berisi *conversation history*, *slots*, dan *active forms*). Kebijakan seperti TEDPolicy akan belajar memprediksi tindakan terbaik berikutnya. TEDPolicy akan melakukan ini dengan mengoptimalkan *cross-entropy loss* menggunakan arsitektur berbasis *transfomer* untuk menciptakan representasi konteks dialog. Dengan demikian, maka semua komponen telah terlatih dan disimpan sebagai sebuah file model dalam format `.tar.gz`. File model ini berisi model NLU, model *policy*, dan metadata yang dibutuhkan agar model dapat digunakan pada server RASA.

4.2.3. *Prediction Stage*

Tahapan prediksi berisi penjelasan mengenai seluruh proses yang terjadi saat *user* mengirimkan pesan ke *chatbot* RASA. Pada tahapan ini RASA akan memuat komponen dan model yang telah dilatih sebelumnya untuk kemudian digunakan dalam menginterpretasikan masukan dari *user* dan menentukan langkah selanjutnya (jawaban yang akan diberikan). Tahapan dimulai dengan proses tokenisasi masukan yang diberikan oleh *user*. Misalnya *user* mengetikkan pertanyaan “bagaimana cara legalisir ijazah ?”. Masukan dari *user* tersebut akan diproses dengan melalui beberapa tahapan yang sebelumnya juga telah didefinisikan dalam file “config.yml” yaitu sebagai berikut.

1. *Tokenization*

   Tokenisasi berbasis *white space* dilakukan terhadap input dan hasilnya sebagai berikut. [“bagaimana”, “cara”, “legalisir”, “ijazah”, “?”]
2. *Regex Featurizer*

   Regex *featurizer* akan melakukan pengecekan pola regex dasar terhadap masing-masing token yang diperoleh dari tahapan tokenisasi. Contoh hasilnya adalah sebagai berikut : [“number : 0”, “academic_year_code : 0”, “email : 0”, “time_date : 0”]

   Dimana pada pengecekan ini tidak ditemukan adanya pola regex khusus dalam kalimat *input user*.
3. *Lexical Syntatic Featurizer*

   Setiap token yang ada akan dianalisa dan diubah menjadi beberapa fitur sintetik seperti : is_uppercase, is_titlecase, is_digit, prefixes, suffixes, is_punctuation. Hasil dari tahap ini dapat dilihat pada Tabel 4.6.

Tabel 4.6. Hasil *Lexical Syntatic Featurizer*

No	Token	Fitur
1	bagaimana	is_uppercase=0, is_tittlecase=0, is_digit=0, prefix=" ", suffix=" ", is_punctuation=0
2	cara	is_uppercase=0, is_tittlecase=0, is_digit=0, prefix=" ", suffix=" ", is_punctuation=0
3	legalisir	is_uppercase=0, is_tittlecase=0, is_digit=0, prefix=" ", suffix=" ", is_punctuation=0
4	ijazah	is_uppercase=0, is_tittlecase=0, is_digit=0, prefix=" ", suffix=" ", is_punctuation=0
5	?	is_uppercase=0, is_tittlecase=0, is_digit=0, prefix=" ", suffix=" ", is_punctuation=1

4. *Count Vectors Featurizer*

   Misalnya keseluruhan *vocabulary* yang dimiliki dalam pengembangan *chatbot* RASA adalah sebagai berikut.

   ["cara", "legalisir", "ijazah", "bagaimana", "cuti", "aktif", "beasiswa", ...]

   Maka proses ini akan menghasilkan sebuah vektor dasar sebagai berikut.

   Vektor *input* = [1, 1, 1, 1, 0, 0, 0, ...]

   Dengan nilai yang ada pada vektor tersebut adalah frekuensi kemunculan untuk setiap kata di dalam vocabulary yaitu

   cara: 1

   legalisir: 1

   ijazah: 1

   bagaimana: 1

   cuti: 0

   beasiswa: 0

   ...

   Vektor ini berikutnya akan diubah ke vektor berdimensi tinggi.

5. *Count Vectors Featurizer (1-4 grams)*

   Hasil pemecahan tiap token kata menjadi *character* dengan jumlah 1-4 karakter adalah misalnya sebagai berikut.

   legalisir : l, e, g, a, l, i, s, i, r, le, eg, ga, al, li, is, si, ir, leg, ega, gal, ali, lis, isi, sir, lega, egal, gali, alis, lisi, isir

   ijazah : i, j, a, z, a, h, ij, ja, az, za, ah, ija, jaz, aza, zah, ijaz, jaza, azah

   Hasil tersebut nantinya akan menjadi sebuah *sparse vector* yang menggambarkan urutan antar karakter.

6. DIET *Classifier*

   Model klasifikasi DIET akan menggunakan hasil dari proses-proses sebelumnya seperti : fitur *sparse* pada *level* kata, fitur *n-gram* karakter, fitur lexical, fitur regex, dan token embeddings (dari proses *training*). Dari semua fitur tersebut, *transformer layer* dari model DIET akan memproses secara berurutan dan kemudian membuat sebuah representasi vektor *embedding* yang menggambarkan keseluruhan *input* dari *user*. DIET akan menghitung *similarity* dengan semua *intent*. Hasil dari proses ini adalah berupa daftar *intent* dan seberapa *confidence* model DIET terhadap prediksinya. Contoh hasil keluaran yang diperoleh dari proses ini dapat dilihat pada Tabel 4.7.

Tabel 4.7. Hasil Klasifikasi Model DIET

No	*Intent*	*Confidence*
1	tanya_legalisir	0.9986
2	tanya_irs	0.0713
3	tanya_rekomendasi_beasiswa	0.0326
4	tanya_proposal_organisasi	0.0289
5	tanya_cuti_akademik	0.0235

   Dengan demikian model DIET akan mengembalikan *intent* dengan *confident* prediksi terbaik yaitu “tanya_legalisir”.

7. *Entity Synonym Mapper*

   Tidak ada *entity* yang dicantumkan dalam *intent* “tanya_legalisir” sehingga tahapan ini tidak menghasilkan keluaran apapun.

8. *Response Selector*

   Karena hanya terdapat satu pilihan jawaban yang diberikan untuk *intent* “tanya_legalisir”, *response selector* tidak benar-benar berfungsi pada kasus saat ini.

9. *Fallback Classifier*

   Karena hasi dari klasifikasi DIET menunjukkan terdapat sebuah *intent* terbaik yaitu “tanya_legalisir” dengan tingkat *confidence* 0.9986 maka *fallback* tidak akan dipanggil dalam kasus ini.

Hasil akhir dari keseluruhan proses dalam tahapan *prediction* ini adalah sebagai berikut.

Tabel 4.8. Tabel Hasil Prediksi Model


```
{
 "intent": {
  "name": "tanya_legalisir",
  "confidence": 0.9986
 },
 "entities": [],
```

```
  "text": "bagaimana cara legalisir ijazah ?"
}
```

4.2.4. *Result Generation*

Tahapan *result generation* membahas bagaimana *chatbot* RASA menghasilkan respon ke *user* berdasarkan hasil prediksi *intent* dari model. Proses ini dilakukan oleh komponen RASA yang bernama *dialogue management system* (*core*) yang akan memilih dan menentukan aksi mana yang akan dilakukan selanjutnya berdasar hasil prediksi *intent*, *conversation state*, dan berdasarkan file konfigurasi yang telah digunakan dalam tahap pelatihan. Hasil dari proses *prediction* akan digunakan sebagai masukan utama dari tahapan ini. Setelah masukan tersebut diterima oleh *dialogue management system*, RASA core akan menentukan langkah-langkah yang mungkin untuk dilakukan berdasarkan beberapa *policy* yang telah didefinisikan pada file "config.yml" yaitu *MemoizationPolicy*, *RulePolicy*, *UnexpecTEDIntentPolicy*, dan *TEDPolicy*. Setiap *policy* akan menghasilkan skor probabilitas dan RASA akan memilih aksi dengan skor tertinggi. Pembahasan masing-masing *policy* adalah sebagai berikut.

1. *RulePolicy*

   RulePolicy menjadi prioritas utama/*policy* utama yang akan dicek terhadap *intent* yang dihasilkan dari tahapan *result generation*. *RulePolicy* bergantung dengan isian yang ada dalam file "rules.yml". Sehingga dalam contoh yang digunakan pada pembahasan pengembangan ini yaitu *intent* "tanya_legalisir" akan berasosiasi dengan *rule* seperti ini :

   Tabel 4.9. Tabel Contoh *Rule* untuk *Intent* "tanya_legalisir"

   ```
   - rule: Respon pertanyaan legalisir
     steps:
     - intent: tanya_legalisir
     - action: utter_jawab_legalisir
   ```

   Rule tersebut berarti bahwa setiap kali *dialogue management* memperoleh hasil *intent* "tanya_legalisir", maka *chatbot* RASA harus melakukan aksi atau memberi jawaban kepada *user* sesuai dengan yang tertulis dalam "utter_jawab_legalisir". Jika *Rule* dijalankan, maka proses *result generation* akan berhenti hingga tahapan ini saja.

2. *MemoizationPolicy*

   MemoizationPolicy memeriksa apakah runtutan percakapan yang telah terjadi dengan *user* dalam satu kondisi itu sama persis dengan yang didefinisikan dalam file

"stories.yml". Jika *user* mengikuti atau melakukan *path* yang sama persis dengan yang telah didefinisikan, maka *memoizationpolicy* akan memprediksi langkah atau aksi selanjutnya sesuai dengan temuannya pada *story*. Misalnya setelah *user* menanyakan tentang bagaimana cara legalisir, biasanya *user* juga akan menanyakan berapa lama waktu yang diperlukan untuk memprosesnya. Berikut ini adalah *stories* yang sama dengan contoh alur tersebut.

Tabel 4.10. Tabel Contoh *Stories* Proses Legalisir

```
- story: Legalisir process
  steps:
  - intent: tanya_legalisir
  - action: utter_jawab_legalisir
  - intent: tanya_waktu_proses
  - action: utter_jawab_waktu_proses
```

Dengan adanya *story* tersebut, *chatbot* RASA dapat langsung saja menggunakan aksi yang sudah didefine dalam *story* yaitu "utter_jawab_waktu_proses" tanpa harus mencari *rule* yang sesuai dalam komponen *rules*.

3. *TEDPolicy*

   Jika dua *policy* sebelumnya tidak memberikan hasil, maka *TEDpolicy* yang berupa sebuah model *transformer* akan memprediksi langkah yang harus dilakukan selanjutnya berdasarkan konteks konversasi dan 5 percakapan terakhir dengan *user*. Jadi misalnya ketika *user* mengajukan pertanyaan yang sama namun hanya berbeda penggunaan frasa atau kata, contoh pertanyaan *user* diubah menjadi "Gimana sih cara gue bisa melegalisir ijzah ?", maka *TEDpolicy* masih dapat memberikan hasil aksi "utter_jawab_legalisir".

4. *UnexpecTEDIntentPolicy*

   Biasanya *unexpecTEDintentpolicy* akan mengarahkan *intent* yang tidak sesuai dalam konteks dialog saat ini ke *fallback*.

Setelah setiap *policy* memberikan hasil probabilitasnya maka RASA akan memilih nilai yang paling tinggi. Dari hasil pemilihan tersebut, terdapat tiga kemungkinan aksi yang akan dilakukan atau respon yang diberikan oleh *chatbot* RASA yaitu :

1. *Utterance Action*

   Pada contoh pertanyaan "bagaimana cara legalisir ijazah ?", aksi "utter_jawab_legalisir" akan dikembalikan sebagai hasil kepada *user*. Karena *intent* "tanya_legalisir" ternyata didefinisikan dalam *rule* "Respon pertanyaan legalisir"

dengan aksinya adalah "utter_jawab_legalisir". RASA akan menggambil *template* respon yang ada kemudian memilih jawaban yang sesuai jika memang terdapat banyak variasi jawaban lalu mengirimkannya pada *user*.

2. *Custom Action*

   Aksi kustom tidak didefinisikan dalam penelitian ini karena semua jawaban yang akan diberikan kepada *user* yang menggunakan *chatbot* sudah terlampir dalam *template* data pelatihan komponen NLU yang digunakan. Namun biasanya aksi *custom* yang dilakukan adalah untuk mengeksekusi kode *python*.

3. *Fallback Action*

   Jika ternyata *confidence* yang sebelumnya di dapatkan dari tahapan *prediction* tidak melebihi angka 0.7, terdapat ambiguitas, dan juga *unexpecTEDintentpolicy* mendeteksi adanya ketidakkonsistenan maka *fallback action* akan memanggil action "utter_diluar_scope" yang sudah didefinisikan pada file "domain.yml".

Setelah semua proses dalam tahapan ini dilakukan maka *user* sudah memperoleh jawaban dari pertanyaannya ke *chatbot*. Pada contoh pertanyaan "bagaimana cara legalisir ijazah ?", jawaban *chatbot* adalah sebagai berikut.

Tabel 4.11. Tabel Hasil Akhir Jawaban *Chatbot*

Prosedur Legalisir Ijazah dan Transkrip (Maksimal 3 hari kerja): 1. Serahkan fotokopi ijazah/transkrip/sertifikat beserta aslinya ke Subbag Akademik 2. Petugas memeriksa keabsahan dan memberikan cap 3. Supervisor memeriksa dan memberi paraf 4. Dekan/Wakil Dekan menandatangani 5. Petugas memberikan stempel fakultas 6. Ambil legalisir dengan membawa aslinya dan tanda tangan di buku pengambilan

4.2.5. *Evaluation Process*

Tahapan terakhir dalam pengembangan *chatbot* RASA yang juga telah dilakukan pada pengembangan *chatbot* RAG yaitu evaluasi, penting untuk dilakukan. Sehingga nantinya kedua model *chatbot* yang dibangun dapat dibandingkan performanya dengan melihat parameter metrik yang sama. Proses evaluasi dilakukan dengan mengujikan set pertanyaan yang diperoleh dari hasil wawancara dengan staf bidang akademik dan kemahasiswaan FSM. Daftar pertanyaan tersebut disajikan dalam lampiran 1 dan juga telah diujikan pada tahap evaluasi dalam pengembangan *chatbot* RAG. Setiap pertanyaan yang ada akan diujicobakan satu persatu untuk dijawab oleh *chatbot* RASA dengan melalui tahapan *prediction* dan *result generation*. Jawaban yang dihasilkan untuk setiap pertanyaan disimpan

dalam sebuah file yaitu "response_rasa.csv". File tersebut akan berisi pertanyaan yang diajukan oleh *user*, *intent* yang terdeteksi oleh model, skor *confidence*, respon yang diberikan oleh *chatbot* RASA, total waktu respon, dan *coverage*. Sama seperti file yang dihasilkan oleh *chatbot* RAG, file ini harus diproses terlebih dahulu dengan menambahkan kolom *Groundtruth* sebelum dapat digunakan untuk mengevaluasi performa model. File jawaban yang telah ditambahkan *ground truth* dari setiap pertanyaan yang ada kemudian disimpan menggunakan encoding `utf-8-sig` dan separator ";" dengan nama file "responses_rag.csv". Setelah berhasil disimpan, lakukan pengujian metrik ROUGE-1, ROUGE-L dan BERTScore dengan implementasi *code* pengujian yang sama yang digunakan pada pengujian performa *chatbot* RAG. Implementasi *code*-nya dapat dilihat kembali pada *Source Code* 4.8. Hasil evaluasi *chatbot* RASA dalam menjawab set pertanyaan yang ada dan dengan tolak ukur metrik ROUGE serta BERTScore dapat dilihat pada Tabel 4.12, sementara perbandingan jawaban *chatbot* dengan *ground truth*-nya dapat dilihat pada Lampiran 6.

Tabel 4.12. Hasil Evaluasi Model RASA Terhadap *Frequently Asked Question*

No	Pertanyaan	Response Time (ms)	Coverage	ROUGE -1	ROUGE -L	BERT Score
1	Bagaimana proses pengajuan cuti akademik ?	249.12	1	0.5164	0.4836	0.7809
2	Berapa lama waktu yang dibutuhkan untuk dapat melakukan legalisir ijazah dan atau transkrip ?	15.01	1	0.1587	0.1587	0.6800
3	Berapa jumlah maksimal lembar legalisir yang dapat mintakan untuk ijazah dan atau transkrip ?	10.12	1	0.1212	0.0909	0.6562
4	Persyaratan apa yang diperlukan apabila pengambilan legalisir ijazah dan atau transkrip diwakilkan oleh pihak lain ?	11.89	1	0.0278	0.0278	0.6351
5	Bagaimana prosedur dan apa syarat yang diperlukan untuk dapat melakukan pengajuan legalisir ijazah dan atau transkrip ?	31.27	1	0.4865	0.4324	0.7454
6	Apa syarat mengajukan proposal kegiatan yang terkait dengan orientasi mahasiswa baru ?	13.54	1	0.0952	0.0635	0.6432
7	Bagaimana prosedur yang dapat dilakukan untuk mendapatkan rekomendasi beasiswa ?	12.07	1	0.6102	0.5593	0.7842
8	Bagaimana jika mata kuliah yang ingin diambil belum muncul di IRS ?	36.69	0	0.16	0.16	0.6871
9	Bagaimana jika saya terlambat melakukan pembayaran UKT dan pengisian IRS?	11.26	1	0.5029	0.4737	0.7628

No	Pertanyaan	Response Time (ms)	Coverage	ROUGE -1	ROUGE -L	BERT Score
10	Saya sempat cuti, lalu bagaimana prosedurnya untuk saya dapat aktif kembali dalam akademik ?	19.85	0	0	0	0.6499

Dari tabel hasil evaluasi tersebut dapat dicermati beberapa hal. Ternyata dari hasil pengujian 10 pertanyaan hasil perolehan wawancara dengan staf akademik FSM Undip, model *chatbot* RASA mampu menjawab 8 dari 10 pertanyaan yang ada. Namun meskipun demikian jika dicermati lebih lanjut pada Lampiran ke 6 dimana jawaban hasil pemberian *chatbot* dituliskan, maka dapat dimengerti bahwa meskipun dikategorikan sebagai terjawab (*coverage* bernilai 1), tidak semua jawaban yang diberikan sesuai dengan jawaban asli. Hal ini dibuktikan dengan skor perolehan ketiga *metrik* yaitu `ROUGE-1`, `ROUGE-L`, dan `BERTScore`.

Dari metrik `ROUGE-1` dapat diketahui bahwa model *chatbot* RASA menunjukkan rata-rata kesamaan tingkat unigram antara jawaban yang diberikan dengan *ground truth* sebesar 27%. Pada jawaban yang memang benar dijawab, misalnya pada pertanyaan 1 dan 7, model berhasil memberikan skor kesamaan unigram sekitar 50%-60%. Hal ini menjadi bukti bahwa model sebenarnya sudah cukup baik dalam memberikan jawaban sesuai dengan konteks pertanyaan meski memang masih ada perbedaan dalam pemilihan kata atau frase yang menyebabkan tingkat kesamaan secara leksikal masih belum optimal. Hal yang kurang lebih sama juga terjadi pada metrik `ROUGE-L` . Hasil dari metrik tersebut menunjukkan bahwa model *chatbot* RASA memiliki tingkat kesesuaian urutan kata/LCS antara jawaban *chatbot* dengan *ground truth* sebesar 24%.

Penyebab dari rendahnya kedua skor metrik tersebut sama dengan apa yang telah dibahas pada penjelasan tahapan evaluasi model *chatbot* RAG, yakni bahwa sumber pengetahuan yang disediakan di dalam model *chatbot* RASA yang digunakan berasal dari tujuh dokumen SOP. Sementara sebagian jawaban *ground truth* yang diperoleh dari proses wawancara yang dicantumkan pada Lampiran 1 ternyata tidak terdapat pada dokumen SOP sumber. Belum lagi beberapa pertanyaan yang diperoleh tidak terdapat jawabannya jika hanya bergantung pada dokumen sumber SOP yang diberikan, misalnya pada pertanyaan "Berapa jumlah maksimal lembar legalisir yang dapat mintakan untuk ijazah dan atau transkrip ?" yang dalam dokumen SOP legalisir ijazah dan transkrip tidak dicantumkan informasi mengenai hal tersebut. Hal ini mengakibatkan ketika model *chatbot* berusaha

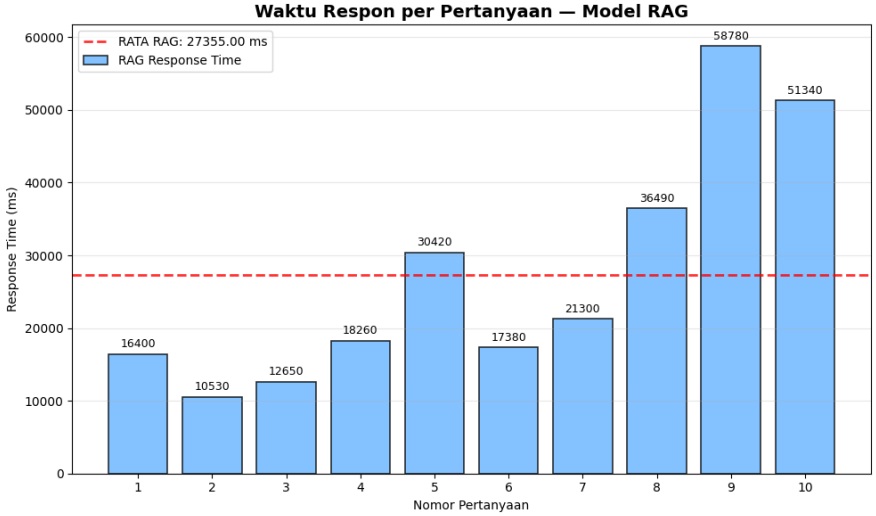
mencari jawaban yang sesuai dengan dokumen SOP sumber, model tidak menemukan dan tidak dapat menjawab dengan baik sebagian pertanyaan yang diujikan.

Sementara itu pada metrik BERTScore, model *chatbot* RASA masih menghasilkan skor rata-rata yang cukup baik yaitu diangka 70% meskipun model ini belum dapat menjawab beberapa pertanyaan. Dari sisi waktu menjawab, model RASA membutuhkan rata-rata waktu sebesar 41 milidetik untuk menghasilkan jawaban terhadap setiap pertanyaan yang diajukan. Waktu itu mencakup seluruh proses dari mulai *prediction* hingga *dialogue management* dan *result generation*.

Secara keseluruhan, dapat dikatakan bahwa model *chatbot* RASA sudah menunjukkan performa yang cukup baik dalam menjawab set pertanyaan yang diajukan. Permasalahan mengenai rendahnya skor metrik murni berasal dari adanya ketidaklengkapan data sumber yang digunakan dalam membangun *chatbot*. Dengan demikian, model ini juga dapat diajukan sebagai salah satu kandidat model yang akan digunakan sebagai *chatbot* pendukung layanan akademik dan kemahasiswaan di FSM Undip. Hasil evaluasi ini juga akan dibawa lebih lanjut pada pembahasan bagian selanjutnya dalam komparasi dengan model RAG.

4.3. Perbandingan Performa Kedua Model

Pada tahapan ini akan dilakukan komparasi dan analisis terhadap performa kedua model secara lebih mendalam satu sama lain. Pertama-tama, analisis akan dilakukan dengan metrik waktu respon dari masing-masing model *chatbot*. Waktu respon model RAG terhadap kesepuluh pertanyaan divisualisasikan pada Gambar 4.6.



Gambar 4.6. Waktu Respon Model RAG pada Setiap Pertanyaan

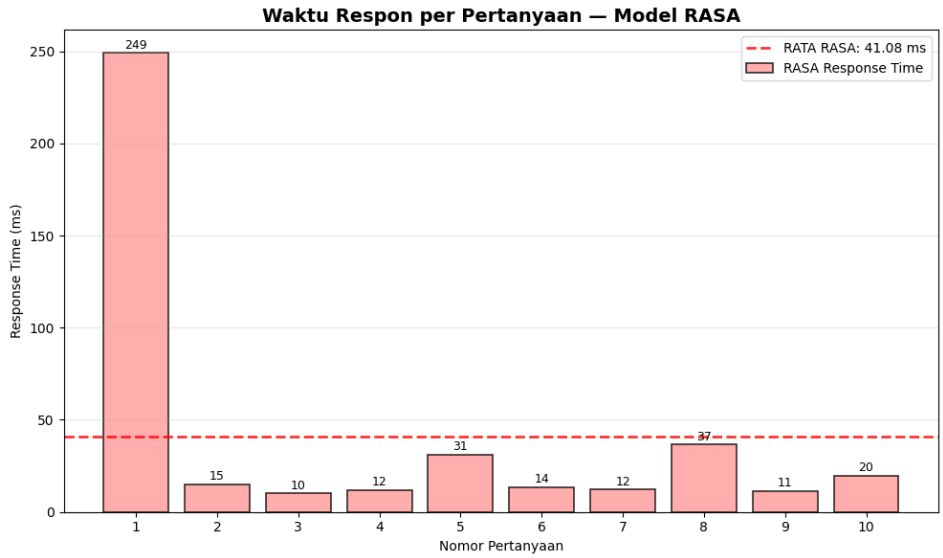
Dari grafik tersebut dapat dilihat bahwa model RAG mampu menjawab cukup cepat pada tujuh pertanyaan awal. Sementara itu pada pertanyaan kedelapan hingga pertanyaan kesepuluh, waktu respon model RAG sangat tinggi. Waktu terlama yang dibutuhkan oleh model RAG untuk memberikan respon adalah 58.78 detik yaitu dalam menjawab pertanyaan nomor 9 dan waktu tercepat adalah 10.53 detik yaitu dalam menjawab pertanyaan nomor 2. Tingginya waktu respon model RAG pada beberapa pertanyaan tersebut tidak dipengaruhi oleh panjang jawaban yang dihasilkan. Hal ini dapat dilihat pada perbandingan pertanyaan nomor 1 dan 10 dengan jawabannya yang ditampilkan pada Tabel 4.13.

Tabel 4.13. Perbandingan Waktu Respon RAG pada Pertanyaan Nomor 1 dan 10

Pertanyaan Ke-	Kalimat Pertanyaan	Jawaban	Waktu Respon (Sec)
1	Bagaimana proses pengajuan cuti akademik ?	Berikut proses pengajuan cuti akademik (maksimal diproses 3 hari kerja): 1. Unduh, isi, dan tandatangani Form Cuti Akademik serta unggah/siapkan persyaratan. Mahasiswa juga wajib mengisi pengajuan cuti secara online melalui SIAP. - Dokumen yang dibutuhkan: Form Cuti Akademik, Transkrip Akademik, bukti bayar SPP/UKT terakhir, fotokopi KTM, dan dokumen pendukung lain. - Link form: https://drive.google.com/file/d/1miPHNvRj6XO6LEhFCiC_QXcMtynCPWR2/edit 2. Minta persetujuan dan tanda tangan Ketua Program Studi pada Form Cuti Akademik. 3. Serahkan Form Cuti Akademik yang telah ditandatangani Ketua Program Studi ke Dekan (akan didisposisi ke Subbag Akademik dan Kemahasiswaan). 4. Subbag Akademik dan Kemahasiswaan memeriksa kelengkapan. Jika lengkap, form diparap dan diproses lanjut. 5. Dekan menandatangani Surat Izin Dekan (Output: Surat Izin Dekan). 6. Subbag Sumber Daya memberi nomor surat dan meneruskan ke Subbag Akademik dan Kemahasiswaan dengan tembusan ke Dosen Wali dan Ketua Program Studi (Output: Surat Izin Dekan). 7. Subbag Akademik dan Kemahasiswaan mengupdate status mahasiswa di Sistem Akademik, mengarsipkan, dan mengirimkan Surat Izin Dekan kepada mahasiswa (Output: Surat Izin Dekan). 8. Mahasiswa mengambil Surat Izin Dekan di loket akademik (Output: Surat Izin Dekan).	16.4
10	Saya sempat cuti, lalu bagaimana	Berikut prosedur untuk aktif kembali setelah cuti:	51.34

Pertanyaan Ke-	Kalimat Pertanyaan	Jawaban	Waktu Respon (Sec)
	prosedurnya untuk saya dapat aktif kembali dalam akademik ?	- Bawa Surat Izin Cuti Akademik dan fotokopi KTM, lalu lapor kepada Ketua Program Studi dan Subbag Akademik dan Kemahasiswaan (±5 menit). - Lakukan registrasi online melalui SSO Anda (±5 menit). - Setelah itu, status Anda akan terdaftar sebagai Peserta Kuliah/Mahasiswa Aktif di fakultas (±5 menit). Catatan: Setelah aktif, jika sedang masa pengisian IRS, silakan: - Membayar UKT, her-registrasi melalui SIAP, mengisi/mencetak IRS, konsultasi dengan Pembimbing Akademik, dan minta persetujuan IRS secara online di SIAP.	

Dari tabel tersebut dapat dilihat bahwa pertanyaan nomor 1 memiliki hasil jawaban respon yang lebih panjang dibandingkan pertanyaan nomor 10. Sehingga dapat disimpulkan bahwa hal yang cenderung lebih mempengaruhi tingginya waktu respon model RAG terhadap pertanyaan adalah kecepatan proses pemanggilan API model OpenAI gpt-5. Keterbatasan performa sumber daya *open source* pemanggilan LLM yang digunakan sangat berpengaruh terhadap tingginya waktu respon model RAG secara keseluruhan. Sementara itu, respon model RASA terhadap kesepuluh pertanyaan divisualisasikan pada Gambar 4.7.



Gambar 4.7. Waktu Respon Model RASA pada Setiap Pertanyaan

Sementara itu pada model RASA, waktu respon yang diperlukan oleh model RASA untuk menjawab setiap pertanyaan dari kesepuluh pertanyaan sangat rendah. Perbedaan tinggi waktu respon yang sangat drastis hanya terjadi pada pertanyaan nomor 1. Perbedaan yang drastis tersebut juga tidak dipengaruhi oleh tersebut tidak dipengaruhi oleh panjang

jawaban yang dihasilkan. Hal ini dapat dilihat misalnya pada pertanyaan nomor 1 dan 2 beserta jawabannya.

Tabel 4.14. Perbandingan Waktu Respon RASA pada Pertanyaan Nomor 1 dan 2

Pertanyaan Ke-	Kalimat Pertanyaan	Jawaban	Waktu Respon (ms)
1	Bagaimana proses pengajuan cuti akademik ?	Prosedur Cuti Akademik (Maksimal 3 hari kerja): 1. Unduh, isi, dan tandatangani Form Cuti Akademik, lampirkan persyaratan dan isi online via SIAP 2. Minta persetujuan dan tanda tangan Ketua Program Studi 3. Serahkan form yang sudah ditandatangani ke Dekan untuk disposisi ke Subbag Akademik dan Kemahasiswaan 4. Subbag Akademik memeriksa kelengkapan dan memproses 5. Dekan menandatangani Surat Izin Dekan 6. Subbag Sumber Daya memberi nomor surat dan mengirimkannya 7. Mahasiswa dapat mengambil Surat Izin Dekan di loket akademik	249.12
2	Berapa lama waktu yang dibutuhkan untuk dapat melakukan legalisir ijazah dan atau transkrip ?	Prosedur Legalisir Ijazah dan Transkrip (Maksimal 3 hari kerja): 1. Serahkan fotokopi ijazah/transkrip/sertifikat beserta aslinya ke Subbag Akademik 2. Petugas memeriksa keabsahan dan memberikan cap 3. Supervisor memeriksa dan memberi paraf 4. Dekan/Wakil Dekan menandatangani 5. Petugas memberikan stempel fakultas 6. Ambil legalisir dengan membawa aslinya dan tanda tangan di buku pengambilan	15.01

Dari tabel tersebut dapat dilihat juga bahwa perbedaan yang drastis waktu respon model RASA pada pertanyaan nomor 1 tidak dipengaruhi oleh panjang jawaban. Perbedaan waktu tersebut mungkin terjadi karena adanya perbedaan ketersediaan sumber daya pada perangkat keras saat kedua pertanyaan diajukan.

Hasil analisis terhadap lama waktu respon kedua model menghasilkan sebuah kesimpulan, yaitu bahwa model RASA mampu memberikan respon yang lebih cepat dengan perbandingan waktu yang cukup jauh dibandingkan dengan model RAG. Model RASA mampu menjawab setiap pertanyaan yang diberikan oleh *user* dengan rata-rata waktu respon sebesar 41 milidetik. Maka dapat dikatakan bahwa model RASA yang berhasil dikembangkan terbukti lebih cepat sekitar 658 kali lipat dibandingkan dengan model RAG yang untuk menjawab setiap pertanyaan membutuhkan waktu rata-rata 27.3 detik. Dari metrik waktu, model RASA lebih unggul dibandingkan model RAG. Perbandingan waktu respon antara kedua model pada masing-masing pertanyaan dapat dilihat pada Tabel 4.15.

Tabel 4.15. Perbandingan Waktu Respon RASA dan RAG

Pertanyaan Ke-	Waktu Respon RAG (ms)	Waktu Respon RASA (ms)
1	16400	249.12
2	10530	15.01
3	12650	10.12
4	18260	11.89
5	30420	31.27
6	170380	13.54
7	2130	12.07
8	36490	36.69
9	58780	11.26
10	51340	19.85

Berikutnya analisis dilakukan dengan metrik *coverage*. Model RAG mampu menjawab semua pertanyaan yang ada. Sementara itu model RASA mampu menjawab 8 dari 10 pertanyaan yang ada. Nilai *coverage* kedua model pada setiap pertanyaan dapat dilihat pada Tabel 4.16.

Tabel 4.16. Perbandingan Kemampuan *Coverage* Kedua Model *Chatbot*

Pertanyaan Ke-	*Coverage*	
	***Chatbot* RAG**	***Chatbot* RASA**
1	1	1
2	1	1
3	1	1
4	1	1
5	1	1
6	1	1
7	1	1
8	1	0
9	1	1
10	1	0

Nilai *coverage* 1 berarti model dinilai mampu menjawab pertanyaan yang diberikan oleh pengguna, sementara nilai *coverage* 0 berarti model belum mampu menjawab pertanyaan dari pengguna. Tabel tersebut menunjukkan bahwa model RAG lebih unggul dibanding model RASA. Namun hal ini perlu dikaji lebih jauh dengan melihat jawaban dari kedua model. Pembahasan pertama akan dilakukan terhadap jawaban model RAG, misalnya terhadap pertanyaan nomor 3 dan 4 yang ditampilkan pada Tabel 4.17.

Tabel 4.17. Perbandingan *Coverage* RAG pada Pertanyaan 3 dan 4

Pertanyaan Ke-	Jawaban	Ground Truth	Coverage
3	Maaf, pada SOP yang tersedia tidak disebutkan batas maksimal jumlah lembar legalisir ijazah dan/atau transkrip. Untuk kepastian, silakan menghubungi Subbag Akademik dan Kemahasiswaan FSM.	Masing-masing maksimal sebanyak 10 lembar untuk ijazah dan atau transkrip.	1

Pertanyaan Ke-	Jawaban	Ground Truth	Coverage
4	Dalam SOP yang tersedia, tidak dijelaskan persyaratan jika pengambilan legalisir ijazah/transkrip diwakilkan. SOP hanya menyebutkan alumni mengambil sendiri dengan membawa dokumen asli dan menandatangani buku pengambilan. Untuk pengambilan oleh pihak lain, silakan konfirmasi langsung ke Subbag Akademik dan Kemahasiswaan FSM (loket akademik) untuk memastikan persyaratan dan prosedur terbaru.	Pihak perwakilan wajib membawa surat kuasa dari pihak yang bersangkutan. Namun surat kuasa tidak perlu diberi materai.	1

Dari contoh pada tabel, dapat diketahui bahwa nilai *coverage* sebesar 1 tidak menjamin sepenuhnya bahwa model RAG dapat memberikan jawaban yang sesuai. Model RAG masih terdeteksi mampu menjawab semua pertanyaan karena skor similaritas dari pertanyaan yang diajukan dengan sumber pengetahuan yang ada pada dokumen sumber ternyata masih di bawah 0.3 yang berarti sebenarnya konteks masih relevan namun tidak terdapat jawaban yang diperlukan oleh *user*. Pertanyaan nomor 3 dan 4 menjadi contoh bahwa pertanyaan yang diuji cobakan masih berada dalam konteks dokumen SOP legalisir ijazah dan transkrip. Sehingga, ketika informasi seputar kedua pertanyaan tersebut dicari maka *information retrieval* dalam sistem RAG akan mengembalikan dokumen SOP legalisir ijazah dan transkrip. Namun, dalam dokumen tersebut tidak terdapat jawaban dari pertanyaan nomor 3 dan nomor 4. Sehingga akhirnya meskipun tetap terbaca dengan nilai *coverage* 1 (berhasil terjawab), jawaban yang diberikan oleh *chatbot* adalah sekedar permintaan maaf karena jawaban yang diinginkan oleh *user* tidak dapat ditemukan di dalam dokumen yang diberikan oleh *retrieval system* kepada LLM.

Pembahasan selanjutnya akan dilakukan terhadap jawaban model RASA. Hal yang sama juga masih terjadi pada *chatbot* RASA, meskipun *chatbot* RASA terbukti sedikit lebih "jujur" pada pertanyaan nomor 8 dan 10 yang memang tidak dapat dijawab dan memiliki nilai *coverage* 0. Contoh perbandingan antara jawaban dan nilai coverage yang diberikan untuk pertanyaan nomor 3 dan 8 ditampilkan pada Tabel 4.18.

Tabel 4.18. Perbandingan *Coverage* RASA pada Pertanyaan 3 dan 8

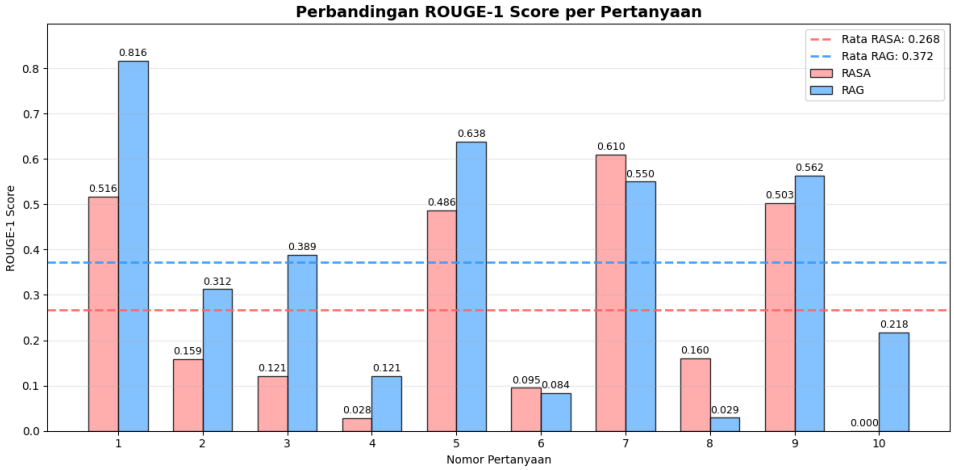
Pertanyaan Ke-	Jawaban	Ground Truth	Coverage
3	Prosedur Legalisir Ijazah dan Transkrip (Maksimal 3 hari kerja):	Masing-masing maksimal sebanyak 10 lembar untuk ijazah dan atau transkrip.	1

Pertanyaan Ke-	Jawaban	Ground Truth	Coverage
	1. Serahkan fotokopi ijazah/transkrip/sertifikat beserta aslinya ke Subbag Akademik 2. Petugas memeriksa keabsahan dan memberikan cap 3. Supervisor memeriksa dan memberi paraf 4. Dekan/Wakil Dekan menandatangani 5. Petugas memberikan stempel fakultas 6. Ambil legalisir dengan membawa aslinya dan tanda tangan di buku pengambilan		
8	Mohon maaf, saya belum dapat menjawab pertanyaan anda. Silakan hubungi Subbag Akademik langsung.	Untuk dapat memperoleh bantuan lebih lanjut, bisa hubungi operator prodi masing-masing.	0

Dari kedua pertanyaan yang ditampilkan pada tabel, dapat dilihat bahwa jawaban RASA yang diberikan pada pertanyaan dengan nilai *coverage* 1 juga masih ada yang mengalami halusinasi. Halusinasi tersebut sebenarnya juga diakibatkan karena adanya kerelevanan konteks yang ditanyakan oleh pengguna namun tidak ada jawabannya dalam informasi yang coba dikembalikan oleh model, akibatnya model mengembalikan respon halu.

Cukup berbeda dengan RAG, meskipun terdeteksi memiliki *coverage* sebesar 1, model RAG tetap memberikan jawaban *fallback* yang mengatakan bahwa dia tidak memiliki informasi mengenai hal yang ditanyakan dari dokumen yang dimiliki model sebagai sumber pengetahuan. Jadi dengan metrik *coverage*, model RAG dapat dikatakan lebih unggul untuk memastikan bahwa pertanyaan yang tidak terdapat jawabannya di dalam sumber pengetahuan *chatbot* benar-benar tidak akan dijawab dengan informasi yang menyesatkan.

Berikutnya, analisis dilakukan dengan metrik performa ROUGE-1. Kedua *chatbot* memiliki perbedaan nilai hasil evaluasi yang tidak terlalu jauh. Perbandingan performa kedua model dari metrik ROUGE-1 dapat dilihat pada Gambar 4.8.



Gambar 4.8. Perbandingan Metrik ROUGE-1 Kedua Model

Chatbot RAG memiliki nilai rata-rata ROUGE-1 sebesar 0.37, sementara *chatbot* RASA memiliki nilai rata-rata 0.27. Jika diukur dari kesamaan unigram keduanya tidak jauh berbeda. Hal ini dapat dilihat lebih jauh pada skor di jawaban masing-masing pertanyaan. Skor ROUGE-1 pada pertanyaan yang dapat terjawab oleh kedua model dan sesuai dengan isian *ground truth* ditampilkan pada Tabel 4.19.

Tabel 4.19. Perbandingan Skor ROUGE-1 Kedua Model pada Jawaban Benar

Pertanyaan Ke-	**ROUGE-1**	
	***Chatbot* RAG**	***Chatbot* RASA**
1	0.8161	0.5164
5	0.6379	0.4865
7	0.5497	0.6102
9	0.5625	0.5029

Untuk pertanyaan yang benar-benar dapat terjawab yaitu pada pertanyaan nomor 1, 5, 7, dan 9 kedua model menunjukkan performa yang cukup baik. Skor dari kedua model untuk kedua pertanyaan tersebut berada di rata-rata 0.50 hingga 0.60, bahkan untuk model RAG berhasil menyentuh skor 0.81 pada pertanyaan nomor 1. Skor tersebut menggambarkan bahwa jawaban yang dihasilkan oleh *chatbot* memperoleh 50% kesamaan dengan jawaban *ground truth*. Lebih lanjut jika dilihat pada teks respon jawaban, jawaban dari kedua *chatbot* sesuai dengan jawaban *ground truth* hanya saja disampaikan dengan kata atau frase yang berbeda baik secara urutan maupun jenis kata. Sehingga skor sekitar 0.50 ini dapat dikatakan sebagai batas bahwa suatu *chatbot* memberikan jawaban yang benar atau tidak. Misalnya pada pertanyaan nomor 7 jawaban model RAG, RASA dan *ground truth*-nya dapat dilihat pada Tabel 4.20.

Tabel 4.20. Jawaban Nomor 7 model RAG dan RASA terhadap *Ground Truth*

Pertanyaan Ke-	Jawaban Model RAG	Jawaban Model RASA	Ground Truth
7	Berikut prosedur pengajuan rekomendasi beasiswa sesuai SOP FSM Undip: - Unduh dan isi Formulir Surat Rekomendasi Pengajuan Beasiswa, siapkan KHS yang sudah dilegalisir. Link formulir: https://drive.google.com/file/d/18f_nbPXHbElgRNoplDFQVNYGGDkFxBTT/view?usp=sharing - Serahkan formulir dan KHS legalisir ke Subbag Akademik dan Kemahasiswaan (BAK) Fakultas untuk diteliti dan diverifikasi (±1 hari). - BAK memberikan paraf dan nomor surat pada Surat Rekomendasi Pengajuan Beasiswa (±10 menit). - Wakil Dekan Akademik dan Kemahasiswaan menandatangani surat rekomendasi tersebut (±2 hari). - Ambil surat yang sudah selesai di BAK Fakultas dan lakukan pencatatan bukti pengambilan (±5 menit). Estimasi total waktu pemrosesan: maksimal 3 hari kerja 45 menit.	Prosedur Pengajuan Rekomendasi Beasiswa (Maksimal 3 hari 45 menit): 1. Download dan isi formulir Surat Rekomendasi Beasiswa ke Subbag Akademik 2. BAK Fakultas meneliti dan memverifikasi 3. BAK memberikan paraf dan nomor surat 4. Wakil Dekan menandatangani 5. Ambil surat di BAK Fakultas dan catat bukti pengambilan	1. Mahasiswa mendownload dan mengisi formulir/ Surat Rekomendasi Pengajuan Beasiswa ke loket Subbag Akademik dan Kemahasiswaan (BAK) Fakultas. 2. BAK Fakultas meneliti dan memverifikasi Surat Rekomendasi Pengajuan Beasiswa. 3. BAK Fakultas memberikan paraf dan nomor surat pada Surat Rekomendasi Pengajuan Beasiswa tersebut. 4. Wakil Dekan Akademik dan Kemahasiswaan memberikan tanda tangan pada Surat Rekomendasi Pengajuan Beasiswa tersebut. 5. Mahasiswa mengambil surat tersebut di BAK Fakultas dan mencatat bukti pengambilan.

Sementara itu skor ROUGE-1 pada pertanyaan lainnya yang memang tidak bisa dijawab terbukti cukup rendah. Untuk pertanyaan-pertanyaan tersebut, model RAG memiliki skor rata-rata 0.19, sementara model RASA memiliki skor rata-rata 0.09. Hal ini berarti bahwa pertanyaan yang tidak berhasil dijawab dengan benar cenderung memiliki skor mendekati 0. Skor ROUGE-1 untuk pertanyaan-pertanyaan yang tidak dapat dijawab oleh kedua model ditampilkan pada Tabel 4.21.

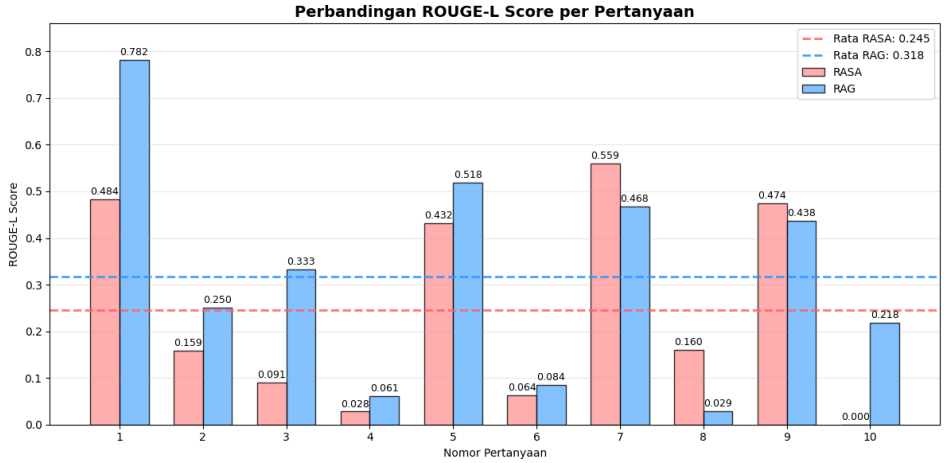
Tabel 4.21. Perbandingan Skor ROUGE-1 Model pada Pertanyaan Tak Terjawab

Pertanyaan Ke-	ROUGE-1	
	Chatbot RAG	*Chatbot* RASA
2	0.3125	0.1587
3	0.3889	0.1212
4	0.1212	0.0278

Pertanyaan Ke-	ROUGE-1	
	***Chatbot* RAG**	***Chatbot* RASA**
6	0.0842	0.0952
8	0.0290	0.16
10	0.2178	0

Jadi jika dilihat dari metrik ROUGE-1, kedua model sebenarnya tidak jauh berbeda performanya hanya saja model RAG pada keempat pertanyaan yang berhasil dijawab dengan tepat berhasil memperoleh skor yang lebih tinggi pada pertanyaan 1, 5, 9 dibanding model RASA. Hal ini berdasar pada kemampuan model RAG yang mampu mengkreasikan jawaban yang diberikan kepada *user* sehingga dapat lebih mirip dengan *ground truth* yang adalah jawaban asli menggunakan bahasa manusia. Meskipun demikian dalam memberikan jawaban atas pertanyaan yang tidak dapat dijawab, model RASA mampu memberikan penolakan yang lebih baik sehingga skor ROUGE-1 untuk pertanyaan yang tidak dapat dijawab bisa jauh lebih rendah mendekati 0. Secara keseluruhan skor ROUGE-1 kedua model dapat dikatakan masih cukup rendah. Skor yang rendah ini bukan diakibatkan oleh kegagalan model dalam memprediksi atau memberikan jawaban namun diakibatkan oleh tidak adanya jawaban yang diharapkan oleh *user* melalui pertanyaannya pada sumber dokumen yang digunakan *chatbot* untuk menjawab.

Berikutnya akan dilakukan analisis performa kedua model dengan metrik ROUGE-L. Meskipun terdengar mirip dengan ROUGE-1 dan skor yang dihasilkan pada tahap evaluasi juga tidak jauh berbeda, ROUGE-L memiliki makna yang berbeda. ROUGE-L memberikan gambaran lebih mendalam bagaimana kesamaan urutan kata yang digunakan oleh *chatbot* dalam memberikan jawaban kepada *user*. Performa kedua model pada metrik ini juga tidak jauh berbeda. Perbandingan metrik ROUGE-L antara kedua model dapat divisualisasikan seperti pada Gambar 4.9.



Gambar 4.9. Perbandingan Metrik ROUGE-L Kedua Model

Model RAG memperoleh rata-rata skor evaluasi ROUGE-L sebesar 0.31 dan model RASA memperoleh rata-rata skor ROUGE-L 0.24. Skor tersebut juga dipengaruhi oleh ketidakadaan jawaban yang diperlukan dari dokumen sumber yang dilatihkan pada kedua model *chatbot*. Jika ditelisik lebih dalam pada bagian pertanyaan yang dapat dijawab, hal yang sama juga terjadi disini sama seperti ROUGE-1. Pertanyaan-pertanyaan yang dapat dijawab memperoleh skor rata-rata ROUGE-L 0.55 pada model RAG dan 0.49 pada model RASA. Skor ROUGE-L untuk kedua model pada masing-masing pertanyaan yang dapat dijawab ditunjukkan pada Tabel 4.22.

Tabel 4.22. Perbandingan Skor ROUGE-L Kedua Model pada Jawaban Benar

Pertanyaan Ke-	**ROUGE-L**	
	***Chatbot* RAG**	***Chatbot* RASA**
1	0.7816	0.4836
5	0.5183	0.4324
7	0.4678	0.5593
9	0.4375	0.4737

Skor tersebut sebenarnya menggambarkan bahwa jawaban yang diberikan oleh kedua model sebenarnya sudah sesuai hanya saja beda urutan frasa atau kata yang digunakan. Pada pertanyaan yang memang tidak dapat dijawab, hasil ROUGE-L sangat rendah yaitu sebesar 0.16 pada model RAG dan 0.083 pada model RASA. Skor ROUGE-L untuk kedua model pada masing-masing pertanyaan yang tidak dapat dijawab ditunjukkan pada Tabel 4.23.

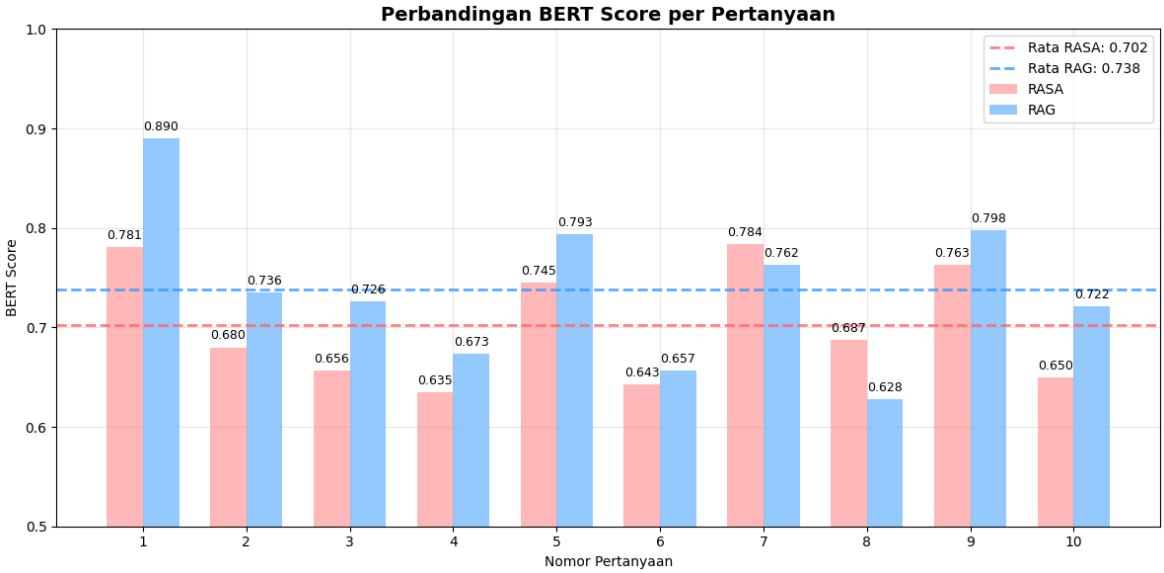
Tabel 4.23. Perbandingan Skor ROUGE-L Model pada Pertanyaan Tak Terjawab

Pertanyaan Ke-	**ROUGE-L**	
	***Chatbot* RAG**	***Chatbot* RASA**
2	0.25	0.1587
3	0.3333	0.0909

Pertanyaan Ke-	ROUGE-L	
	Chatbot RAG	*Chatbot* RASA
4	0.0606	0.0278
6	0.0842	0.0635
8	0.0290	0.16
10	0.2178	0

Hasil dari skor metrik ROUGE-L tersebut menunjukkan bahwa kedua model sebenarnya memiliki performa yang juga tidak jauh berbeda dalam penyusunan urutan kata untuk menjawab pertanyaan yang diberikan oleh *user*. Hanya saja memang model RAG terlihat lebih handal dalam menjawab pertanyaan-pertanyaan yang memang terdapat jawabannya pada dokumen sumber, sementara model RASA mampu memberikan skor yang lebih rendah untuk pertanyaan yang tidak dapat dijawab.

Selanjutnya pada metrik BERTScore, kedua model memiliki selisih skor evaluasi terhadap 10 pertanyaan sebesar 3%. Perbandingan performa kedua model pada metrik BERTScore dapat dilihat pada Gambar 4.10.



Gambar 4.10. Perbandingan Metrik BERTScore Kedua Model

Dapat dilihat pada grafik tersebut bahwa skor pada masing-masing jawaban kedua *chatbot* pada kesepuluh pertanyaan tidak terlalu jauh berbeda. Model RAG mampu memperoleh rata-rata BERTScore untuk kesepuluh pertanyaan sebesar 0.73. Hal tersebut menandakan bahwa jawaban yang diberikan oleh model RAG secara semantik 73% mirip dengan jawaban *ground truth* . Sementara itu jawaban yang dihasilkan oleh model RASA memiliki rata-rata kesamaan semantik sebesar 70% terhadap jawaban *ground truth*.

Hal yang menarik dari perolehan BERTScore kedua model pada masing-masing pertanyaan. Dari kesepuluh pertanyaan tersebut model RAG memiliki BERTScore yang lebih tinggi daripada model RASA pada 8 pertanyaan. Namun pada pertanyaan nomor 7 dan 8 terlihat bahwa model RASA mampu mengungguli perolehan BERTScore RAG. Jawaban dari model RAG, RASA dan *ground truth*-nya pada kedua pertanyaan tersebut ditampilkan pada Tabel 4.24.

Tabel 4.24. Jawaban model RAG dan RASA terhadap *Ground Truth*

Pertanyaan Ke-	**Jawaban Model RAG**	**Jawaban Model RASA**	**Ground Truth**
7	Berikut prosedur pengajuan rekomendasi beasiswa sesuai SOP FSM Undip: - Unduh dan isi Formulir Surat Rekomendasi Pengajuan Beasiswa, siapkan KHS yang sudah dilegalisir. Link formulir: https://drive.google.com/file/d/18f_nbPXHbElgRNoplDFQVNYGGDkFxBTT/view?usp=sharing - Serahkan formulir dan KHS legalisir ke Subbag Akademik dan Kemahasiswaan (BAK) Fakultas untuk diteliti dan diverifikasi (±1 hari). - BAK memberikan paraf dan nomor surat pada Surat Rekomendasi Pengajuan Beasiswa (±10 menit). - Wakil Dekan Akademik dan Kemahasiswaan menandatangani surat rekomendasi tersebut (±2 hari). - Ambil surat yang sudah selesai di BAK Fakultas dan lakukan pencatatan bukti pengambilan (±5 menit). Estimasi total waktu pemrosesan: maksimal 3 hari kerja 45 menit.	Prosedur Pengajuan Rekomendasi Beasiswa (Maksimal 3 hari 45 menit): 1. Download dan isi formulir Surat Rekomendasi Beasiswa ke Subbag Akademik 2. BAK Fakultas meneliti dan memverifikasi 3. BAK memberikan paraf dan nomor surat 4. Wakil Dekan menandatangani 5. Ambil surat di BAK Fakultas dan catat bukti pengambilan	1. Mahasiswa mendownload dan mengisi formulir/ Surat Rekomendasi Pengajuan Beasiswa ke loket Subbag Akademik dan Kemahasiswaan (BAK) Fakultas. 2. BAK Fakultas meneliti dan memverifikasi Surat Rekomendasi Pengajuan Beasiswa. 3. BAK Fakultas memberikan paraf dan nomor surat pada Surat Rekomendasi Pengajuan Beasiswa tersebut. 4. Wakil Dekan Akademik dan Kemahasiswaan memberikan tanda tangan pada Surat Rekomendasi Pengajuan Beasiswa tersebut. 5. Mahasiswa mengambil surat tersebut di BAK Fakultas dan mencatat bukti pengambilan.
8	Berdasarkan SOP IRS: - Pastikan Anda sudah membayar SPP/UKT dan melakukan her-registrasi di SIAP, lalu coba isi IRS sementara. - Konsultasikan rencana mata kuliah dengan Pembimbing Akademik.	Mohon maaf, saya belum dapat menjawab pertanyaan anda. Silakan hubungi Subbag Akademik langsung.	Untuk dapat memperoleh bantuan lebih lanjut, bisa hubungi operator prodi masing-masing.

Pertanyaan Ke-	Jawaban Model RAG	Jawaban Model RASA	Ground Truth
	SOP tidak memuat langkah khusus jika mata kuliah belum muncul di SIAP. Jika hal ini terjadi, segera berkonsultasi dengan Pembimbing Akademik untuk arahan, lalu cek kembali IRS Anda di SIAP.		

Terlihat jelas bahwa pada kedua pertanyaan tersebut, memang jawaban model RASA terlihat lebih mirip dengan jawaban *ground truth*. Meskipun pada pertanyaan nomor 8, model RASA tidak benar-benar berhasil menjawab pertanyaan dengan jawaban yang sesuai. Skor yang masih tinggi meskipun tidak benar-benar berhasil terjawab ini menjadi hal yang juga muncul pada pertanyaan-pertanyaan lain yang juga tidak dapat terjawab. Nilai BERTScore tersebut muncul karena setiap kata yang ada pada *ground truth* maupun jawaban yang diberikan *chatbot* akan direpresentasikan menjadi vektor *embedding* menggunakan model BERT. Akibatnya skor menjadi tetap tinggi karena terdapat kata yang memiliki kesamaan makna meskipun sebenarnya jawaban tersebut tidak sesuai dengan jawaban asli.

Namun meskipun pertanyaan-pertanyaan yang tidak dapat dijawab memiliki skor yang cukup baik, pertanyaan-pertanyaan yang dapat dijawab tetap memiliki skor yang jauh lebih baik. Pada pertanyaan-pertanyaan yang dapat dijawab yaitu pertanyaan 1, 5, 7, dan 9, BERTScore yang dihasilkan oleh kedua model sama-sama mendekati angka 0.80. Nilai BERTScore masing-masing model pada pertanyaan tersebut dapat dilihat pada Tabel 4.25.

Tabel 4.25. Perbandingan Skor BERTScore Kedua Model pada Jawaban Benar

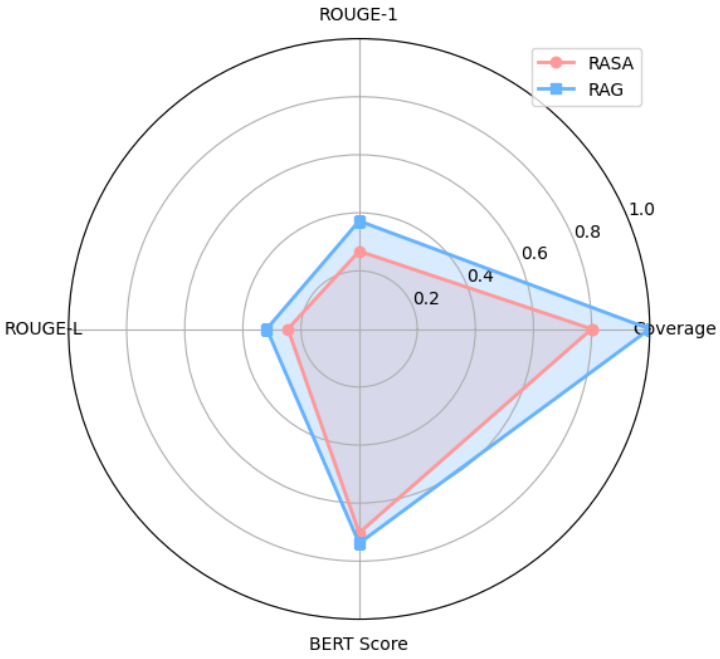
Pertanyaan Ke-	BERTScore	
	Chatbot RAG	*Chatbot* RASA
1	0.8896	0.7809
5	0.7934	0.7454
7	0.7625	0.7842
9	0.7978	0.7628

Hingga saat ini memang belum ada tolak ukur yang pasti mengenai berapa nilai BERTScore yang dapat dikatakan sebagai baik, namun dari adanya pengembangan *chatbot* dalam penelitian ini dapat dikatakan bahwa perolehan BERTScore yang mendekati 0.80 dalam konteks jawaban bidang pendidikan sudah cukup baik. Hal ini didasari dengan bukti nyata bahwa sebenarnya jawaban yang diberikan *chatbot* untuk pertanyaan 1, 5, 7, dan 9

sudah memberikan maksud yang sama dengan jawaban aslinya hanya berbeda dari segi tatanan bahasa atau jenis kata yang digunakan.

Dengan perolehan rata-rata BERTScore kedua model yang berada di angka 0.70, dapat dikatakan model sudah cukup baik meskipun masih tergolong rendah. Rendahnya skor diakibatkan oleh adanya pertanyaan-pertanyaan yang tidak jawab karena ketidakadaan jawaban pada sumber pengetahuan yang disediakan untuk *chatbot*. Dari segi BERTScore, model RAG lebih unggul sedikit dibanding model RASA.

Jadi dari keseluruhan metrik performa yang digunakan untuk membandingkan kedua model, dapat dikatakan bahwa kedua model yaitu RASA dan RAG memiliki performa yang tidak jauh berbeda dari keempat metrik yaitu *coverage*, ROUGE-1, ROUGE-L, dan BERTScore. Hal ini dapat digambarkan dengan sebuah visualisasi pada Gambar 4.11.



Gambar 4.11. Visualisasi Akhir Perbandingan Kedua Model

Dari adanya *radar chart* tersebut dapat semakin dilihat bahwa performa kedua model berbeda tipis dan model RAG terlihat unggul cukup banyak pada metrik *coverage* meskipun sejatinya tidak murni demikian seperti yang telah dijelaskan sebelumnya pada pemaparan metrik *coverage*. Terdapat satu metrik lagi yang memang tidak dimasukkan ke dalam *radar chart* yaitu *response time* karena adanya perbedaan rentang yang cukup jauh. Meskipun dari keempat metrik pada *radar chart* dapat dikatakan bahwa model RAG unggul. Namun dari metrik *response time*, model RASA jauh lebih unggul dari model RAG yang mampu

menghasilkan jawaban bagi *user* 658 kali lebih cepat dibanding model RAG. Oleh karena itu dapat diberikan suatu kesimpulan bahwa untuk dapat memberikan jawaban yang cepat bagi pengguna, pengembangan *chatbot* untuk layanan akademik dan kemahasiswaan di lingkungan FSM Undip dapat dilakukan dengan menggunakan model RASA.

Pertimbangan dalam pemilihan framework chatbot juga perlu dilihat dari kebutuhan pengembangan sistem. Pada framework RASA, pengembang harus mendefinisikan secara manual komponen intent, rule, stories, dan domain yang diperlukan agar model DIET dapat melakukan proses pembelajaran. Apabila jumlah dokumen yang digunakan sebagai sumber pengetahuan semakin besar, proses ini berpotensi memerlukan waktu dan tenaga yang signifikan.

Sebaliknya, framework RAG memungkinkan pemanfaatan sumber pengetahuan dengan cara yang lebih sederhana, yaitu dengan menyiapkan dokumen dalam format PDF dan melakukan proses ekstraksi informasi untuk disimpan ke dalam basis data. Oleh karena itu, dari sisi persiapan pengembangan, pemilihan framework dapat disesuaikan dengan ketersediaan sumber daya yang dimiliki. Apabila FSM Undip memiliki sumber daya pengembang sistem yang memadai untuk menyiapkan pengetahuan secara terstruktur, framework RASA dapat menjadi pilihan yang tepat. Namun, apabila ketersediaan sumber daya manusia terbatas, framework RAG dapat dipertimbangkan sebagai alternatif dengan dukungan sumber daya komputasi yang memadai.

Selain aspek pengembangan, pemilihan framework juga dapat ditinjau dari variasi dan fleksibilitas respons chatbot. Model RAG terbukti mampu menghasilkan jawaban yang lebih variatif dan interaktif, sedangkan model RASA cenderung menghasilkan respons yang lebih terstruktur dan kaku. Perbandingan jawaban dari kedua chatbot untuk setiap pertanyaan disajikan secara rinci pada Lampiran 4 untuk model RAG dan Lampiran 6 untuk model RASA.

Lebih lanjut, penelitian ini menunjukkan bahwa penggabungan framework RAG dan RASA berpotensi untuk diterapkan sebagai pendekatan hibrida, sehingga keunggulan masing-masing framework dapat dimanfaatkan secara optimal guna mengatasi keterbatasan yang ada.

BAB V
PENUTUP

Bab ini berisi kesimpulan dari penelitian yang telah dilakukan dan saran yang dapat digunakan sebagai bahan pertimbangan dalam penelitian yang akan dilakukan di masa yang akan datang.

5.1. Kesimpulan

Pengembangan *chatbot* berbasis *framework Retrieval-Augmented Generation* (RAG) dan RASA telah berhasil dilakukan dengan menggunakan 7 dokumen Standar Operasional Prosedur (SOP) layanan akademik dan kemahasiswaan sebagai sumber pengetahuan. Pengujian dilakukan menggunakan 10 pertanyaan yang paling sering diajukan untuk mengevaluasi kemampuan kedua model dalam mendukung layanan akademik. Kesepuluh pertanyaan tersebut diperoleh melalui wawancara dengan staf layanan akademik dan kemahasiswaan Fakultas Sains dan Matematika Universitas Diponegoro (FSM Undip).

Hasil evaluasi menunjukkan bahwa kedua model memiliki tingkat akurasi yang relatif setara. *Framework* RASA memiliki keunggulan utama pada kecepatan respon, dengan performa rata-rata 658 kali lebih cepat dibandingkan *framework* RAG. Namun, keunggulan tersebut disertai dengan kebutuhan upaya yang lebih besar dalam menyiapkan sumber pengetahuan, khususnya pada tahap pengembangan dan pelatihan komponen *Natural Language Understanding* (NLU). Keterbatasan ini dapat diatasi melalui pemanfaatan *framework* RAG yang menawarkan kemudahan dalam persiapan serta penambahan sumber pengetahuan secara langsung dari dokumen yang tersedia, meskipun memiliki kelemahan berupa kebutuhan komputasi yang relatif lebih besar.

Dalam proses konfigurasi sistem, penentuan ukuran *chunk* sebesar 1700 digunakan sebagai keputusan akhir dalam pemecahan dokumen SOP. Pemilihan ukuran ini dilakukan untuk menjaga keutuhan konteks informasi, mengingat setiap dokumen SOP perlu dipertahankan sebagai satu kesatuan alur prosedural agar makna dan keterkaitan antar langkah tetap terjaga.

5.2. Saran

Berdasarkan seluruh proses yang telah dilalui dalam tahapan pengembangan *chatbot*, berikut ini adalah beberapa saran yang dapat diterapkan pada penelitian selanjutnya di bidang atau studi kasus yang serupa.

1. Sebelum memulai pengembangan *chatbot* berbasis *external knowledge*, pengembang harus benar-benar memastikan bahwa dokumen yang digunakan harus mencakup semua pengetahuan yang memang dibutuhkan pada bidang terkait. Dalam studi kasus ini, pengetahuan yang dimaksud adalah informasi mengenai prosedur layanan akademik dan kemahasiswaan.
2. Menyediakan perangkat dengan sumber daya komputasi yang tinggi untuk memperoleh hasil performa *chatbot* yang lebih maksimal. Hal ini didasarkan pada fakta bahwa beberapa *framework* terbaik yang dapat digunakan dalam pengembangan *chatbot* memerlukan sumber daya komputasi tinggi untuk dapat digunakan.

DAFTAR PUSTAKA

Allen, J. F. (2016). Natural Language Understanding. *Artificial Intelligence*. https://api.semanticscholar.org/CorpusID:46208633

Church, K. W., Sun, J., Yue, R., Vickers, P., Saba, W., & Chandrasekar, R. (2024). Emerging trends: a gentle introduction to RAG. *Natural Language Engineering*, *30*(4), 870–881. https://doi.org/10.1017/S1351324924000044

Dadure, P., Pakray, P., & Bandyopadhyay, S. (2022). A formula embedding approach for semantic similarity and relatedness between formulas. *Concurrency and Computation: Practice and Experience*, *34*(22). https://doi.org/10.1002/cpe.7146

Deutsch, D., & Roth, D. (2021). Understanding the Extent to which Content Quality Metrics Measure the Information Quality of Summaries. *CoNLL 2021 - 25th Conference on Computational Natural Language Learning, Proceedings*, 300–309. https://doi.org/10.18653/V1/2021.CONLL-1.24

Ehsan, T., Khalid, J., Ambreen, S., Mustafa, A., & Hussain, S. (2022). Improving Phrase Chunking by using Contextualized Word Embeddings for a Morphologically Rich Language. *Arabian Journal for Science and Engineering*, *47*(8), 9781–9799. https://doi.org/10.1007/s13369-021-06343-7

Elsayed, A. A., Hafez, A. I., Ceprián, R., Martínez, G., Granados, A., Soriano, B., Llorens, C., & Sempere, J. M. (2024). Practical Design and Implementation of Virtual Chatbot Assistants for Bioinformatics Based on a NLU Open Framework. *Big Data and Cognitive Computing*, *8*(11). https://doi.org/10.3390/bdcc8110163

Fergencs, T., & Meier, F. (2021). Engagement and Usability of Conversational Search – A Study of a Medical Resource Center Chatbot. *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, *12645 LNCS*, 328–345. https://doi.org/10.1007/978-3-030-71292-1_26

Giabelli, A. (2022). A Unified Framework for Intrinsic Evaluation of Word-Embedding Algorithms. *IJCAI International Joint Conference on Artificial Intelligence*, 5853–5854. https://doi.org/10.24963/ijcai.2022/829

Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.

Hao, X., Zhang, G., & Ma, S. (2016). Deep Learning. *International Journal of Semantic Computing*, *10*(3), 417–439. https://doi.org/10.1142/S1793351X16500045

Jain, A. M., & Jain, A. (2025). Scaling LLM Inference Architectures: A Performance Analysis for Chatbot Applications. *2025 6th International Conference on Artificial*

Intelligence, Robotics, and Control, AIRC 2025, 8–16. https://doi.org/10.1109/AIRC64931.2025.11077484

Jeong, T. (2024). 4bit-Quantization in Vector-Embedding for RAG. *Proceedings - 2024 International Conference on Machine Learning and Applications, ICMLA 2024*, 1037–1042. https://doi.org/10.1109/ICMLA61862.2024.00156

Jiang, S., Cormier, S., Angarita, R., & Rousseaux, F. (2023). Improving text mining in plant health domain with GAN and/or pre-trained language model. *Frontiers in Artificial Intelligence*, *6*. https://doi.org/10.3389/frai.2023.1072329

Joshi, D., Tekade, S., Zanzane, S., Sakharwade, D., Tripathi, A., & Tiwadi, S. (2024). Generative AI-Driven Chatbot for Personalized University Information and Assistance. *2024 International Conference on Artificial Intelligence and Quantum Computation-Based Sensor Applications, ICAIQSA 2024 - Proceedings*. https://doi.org/10.1109/ICAIQSA64000.2024.10882421

Jurafsky, D., & Martin, J. H. (2026). *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models* (3rd ed.). https://web.stanford.edu/~jurafsky/slp3/

Kang, G., Zhang, X., Ge, Z., Wang, L., Hu, J., & Zhan, J. (2025). BigVectorBench: Heterogeneous Data Embedding and Compound Queries are Essential in Evaluating Vector Databases. *Proceedings of the VLDB Endowment*, *18*(5), 1536–1550. https://doi.org/10.14778/3718057.3718078

Kaster, M., Zhao, W., & Eger, S. (2021). Global Explainability of BERT-Based Evaluation Metrics by Disentangling along Linguistic Factors. *EMNLP 2021 - 2021 Conference on Empirical Methods in Natural Language Processing, Proceedings*, 8912–8925. https://doi.org/10.18653/V1/2021.EMNLP-MAIN.701

Kim, S., Moon, J., Oh, J., Choi, I., & Yang, J. S. (2025). Survey and Evaluation of Converging Architecture in LLMs Based on Footsteps of Operations. *IEEE Open Journal of the Computer Society*. https://doi.org/10.1109/OJCS.2025.3587005

Kong, X., & Wang, G. (2021). *Conversational AI with Rasa : build, automate, and deploy AI-powered text and voice-based assistants and chatbots*. Packt Publishing Ltd.

Lee, J., Kim, S., Jo, W., & Yoo, H. J. (2023). An Energy-Efficient Heterogeneous Fourier Transform-Based Transformer Accelerator with Frequency-Wise Dynamic Bit-Precision. *2023 IEEE Asian Solid-State Circuits Conference, A-SSCC 2023*. https://doi.org/10.1109/A-SSCC58667.2023.10347933

Liu, L., & Xu, X. (2023). Self-attention Mechanism at the Token Level: Gradient Analysis and Algorithm Optimization. *Knowledge-Based Systems*, *277*, 110784. https://doi.org/10.1016/J.KNOSYS.2023.110784

Luise, R. S. A. De, Arevalillo-Herraez, M., & Arnau, D. (2023). On Using Conversational Frameworks to Support Natural Language Interaction in Intelligent Tutoring Systems. *IEEE Transactions on Learning Technologies*, *16*(5), 722–735. https://doi.org/10.1109/TLT.2023.3245121

Maheswari, B., & Nagarajan, R. G. (2023). Educational Chatbot for Adopting Effective Teaching Learning Process using Long Short-Term Memory with Feature Extraction. *Proceedings of the 2023 2nd International Conference on Augmented Intelligence and Sustainable Systems, ICAISS 2023*, 1749–1757. https://doi.org/10.1109/ICAISS58487.2023.10250704

Mungai, P. K., & Huang, R. (2017). Chunking mechanisms for a self improving associative memory model. *2017 IEEE Symposium Series on Computational Intelligence, SSCI 2017 - Proceedings*, *2018-January*, 1–6. https://doi.org/10.1109/SSCI.2017.8285215

NGO, T. T. A., BUI, C. T., CHAU, H. K. L., TRAN, N. P. N., NGUYEN, P. P. T., & TRAN, N. K. T. (2025). Influence of university service quality on student experiences, academic performance and institutional loyalty: A case study in Vietnam. *Acta Psychologica*, *260*, 105599. https://doi.org/10.1016/J.ACTPSY.2025.105599

Oliveira, P. F., & Matos, P. (2023). Introducing a Chatbot to the Web Portal of a Higher Education Institution to Enhance Student Interaction †. *Engineering Proceedings*, *56*(1). https://doi.org/10.3390/ASEC2023-16621

Osman, Z., Yusoof, F., … A. A.-Int. J. Acad. Res. P., & 2024, undefined. (2024). Enhancing student retention through academic support and course design: Satisfaction as mediator. *kwpublications.comZ Osman, F Yusoof, A Ali, RC AzizInt. J. Acad. Res. Progress. Educ. Dev, 2024•kwpublications.com*, *13*(4). https://doi.org/10.6007/IJARPED/v13-i4/23679

Peña-Lang, M. B., Barrutia, J. M., & Echebarria, C. (2023). Service quality and students' academic achievement. *Quality Assurance in Education*, *31*(2), 247–262. https://doi.org/10.1108/QAE-02-2022-0039

Plazas, R. A., & Avella Rodríguez, E. J. (2024). Chatbots: The Future of Education? *International Journal of Engineering Pedagogy*, *14*(8), 107–119. https://doi.org/10.3991/ijep.v14i8.49715

Purba, C. N. Y., Parulian, P. V. A., Iswara, R. W., Suhandana, A. A., & Sari, R. (2024). The Implementation of Natural Language Processing on Chatbot as Academic Information System: A Case Study of IC2IE 2024. *Proceedings 7th IC2IE 2024 - 2024 International Conference of Computer and Informatics Engineering: Generative AI in Democratizing Access to Knowledge and Skills*. https://doi.org/10.1109/IC2IE63342.2024.10748156

Rogers, S., Girolami, M., Rish, I., Grabarnik, G., Liu, X., Datta, A., Lim, E.-P., Suykens, J. A. K., Signoretto, M., Argyriou, A., Marsland, S., Programming, B., Bessiere, P.,

Mazer, E., Ahuactzin, J. M., Mekhnacha, K., Lu, H., Plataniotis, K. N., Venetsanopoulos, A., … Lappin, S. (2022). Transformers for Machine Learning : A Deep Dive. *Transformers for Machine Learning*. https://doi.org/10.1201/9781003170082

Rothman, Denis. (2024). *RAG-Driven Generative AI : Build Custom Retrieval Augmented Generation Pipelines with LlamaIndex, Deep Lake, and Pinecone*. Packt Publishing Ltd.

S., K., Kailath, B. J., & B.S., R. (2024). CiMComp: An Energy Efficient Compute-in-Memory Based Comparator for Convolutional Neural Networks. *2024 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, 1–2. https://doi.org/https://doi.org/10.23919/date58400.2024.10546864

Salloum, S. A., Shalan, K., Basiouni, A., Salloum, A., & Alfaisal, R. (2024). Building and Evaluating a Chatbot Using a University FAQs Dataset. *Communications in Computer and Information Science*, *2162 CCIS*, 207–218. https://doi.org/10.1007/978-3-031-65996-6_18

Sellam, T., Das, D., & Parikh, A. P. (2020). BLEURT: Learning robust metrics for text generation. *Proceedings of the Annual Meeting of the Association for Computational Linguistics*. https://doi.org/https://doi.org/10.18653/v1/2020.acl-main.704

Shenoy, A., Y., S. R., Sharma, A., Rajan, A., & Gv, A. (2019). NLP models behind RASA stack. *International Journal of Engineering and Advanced Technology*, *8(5s)*, 62–66. https://doi.org/https://doi.org/10.35940/ijeat.e1013.0585s19

Sonkar, A., Singh, S. P., Sahu, K., Sahu, A., & Mishra, S. (2025). Dynamic Query Handling with RAG Fusion for PDF-Based Knowledge Retrieval Systems. *2025 4th OPJU International Technology Conference on Smart Computing for Innovation and Advancement in Industry 5.0, OTCON 2025*. https://doi.org/10.1109/OTCON65728.2025.11070378

Szmurlo, H., & Akhtar, Z. (2024). Digital Sentinels and Antagonists: The Dual Nature of Chatbots in Cybersecurity. *Information (Switzerland)*, *15*(8). https://doi.org/10.3390/info15080443

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, *2017-December*, 5999–6009.

Yang, J., Cai, Q., & Tian, X. (2020). How do we segment text? Two-stage chunking operation in reading. *eNeuro*, *7*(3), 1–14. https://doi.org/10.1523/ENEURO.0425-19.2020

Yin, B., Li, D., Xu, Z., & Hu, N. (2025). A Semantic-Structural Synergistic Chunking Method for Enhancing Domain-Specific LLM Fine-Tuning. *Lecture Notes in Computer Science*, 507–518. https://doi.org/10.1007/978-981-95-0020-8_43

Zhang, H., & Shafiq, M. O. (2024). Survey of transformers and towards ensemble learning using transformers for natural language processing. *Journal of Big Data*, *11*(1). https://doi.org/10.1186/s40537-023-00842-0

Zhang, T., Kishore, V., Wu, F., Weinberger, K. Q., & Artzi, Y. (2020). Bertscore: Evaluating Text Generation With Bert. *8th International Conference on Learning Representations, ICLR 2020*.

Zhang, T., Patterson, L., Beiting-Parrish, M., Webb, B., Abeysinghe, B., Bailey, P., & Sikali, E. (2024). Ask NAEP: A Generative AI Assistant for Querying Assessment Information. *Journal of Measurement and Evaluation in Education and Psychology*, *15*, 378–394. https://doi.org/10.21031/epod.1548128

Zhou, Q., Liu, C., Duan, Y., Sun, K., Li, Y., Kan, H., Gu, Z., Shu, J., & Hu, J. (2024). GastroBot: a Chinese gastrointestinal disease chatbot based on the retrieval-augmented generation. *frontiersin.org*, *11*, 1392555. https://doi.org/10.3389/FMED.2024.1392555/FULL

Zhu, J., Yan, L., Shi, H., Yin, D., & Sha, L. (2024). ATM: Adversarial Tuning Multi-agent System Makes a Robust Retrieval-Augmented GENERATOR. *EMNLP 2024 - 2024 Conference on Empirical Methods in Natural Language Processing, Proceedings of the Conference*, 10902–10919. https://doi.org/10.18653/v1/2024.emnlp-main.610

Zhu, Y., Yang, X., Wu, Y., Zhu, M., & Zhang, W. (2023). Differentiable N-gram objective on abstractive summarization. *Expert Systems with Applications*, *215*, 119367. https://doi.org/10.1016/J.ESWA.2022.119367

LAMPIRAN - LAMPIRAN

LAMPIRAN 1. Hasil Wawancara dengan Staf Akademik

Hasil Wawancara

Hari/tanggal : Selasa, 4 November 2025

Tempat : Kantor Layanan Akademik FSM

Pertanyaan : "Pertanyaan apa yang sering ditanyakan mahasiswa saat mendatangi layanan akademik dan kemahasiswaan ? Lalu apa jawaban yang diberikan oleh petugas ?"

Jawaban :

No	Pertanyaan	Sumber Dokumen	Jawaban
1	Bagaimana proses pengajuan cuti akademik ?	SOP Izin Cuti Akademik	1. Mahasiswa mengunduh, mengisi, dan menandatangani Form Cuti Akademik dengan melampirkan persyaratan. Selain itu, mahasiswa juga wajib mengisi secara online melalui SIAP. Dokumen yang dibutuhkan: Form Cuti Akademik, Transkrip Akademik, Bukti bayar SPP/UKT terakhir, Fotokopi KTM, dan dokumen pendukung lain Link unduh form: https://drive.google.com/file/d/1miPHNvRj6XO6LEhFCiC_QXcMtynCPWR2/edit 2. Mahasiswa meminta persetujuan dan tanda tangan Ketua Program Studi. 3. Mahasiswa menyerahkan Form Cuti Akademik yang telah ditandatangani Ketua Program Studi ke Dekan. Kemudian dilakukan disposisi ke Subbag Akademik dan Kemahasiswaan. 4. Subbag Akademik dan Kemahasiswaan menerima, memeriksa, dan meneliti kelengkapan persyaratan form cuti akademik. Jika lengkap, form diberi paraf dan diproses lebih lanjut. 5. Dekan menandatangani Surat Izin Dekan. 6. Subbag Sumber Daya memberi nomor surat dan mengirimkannya ke Subbag Akademik dan Kemahasiswaan dengan tembusan ke Dosen Wali dan Ketua Program Studi. 7. Subbag Akademik dan Kemahasiswaan mengupdate status mahasiswa di Sistem Akademik, mengarsipkan Surat Izin Dekan, dan mengirimkan kepada mahasiswa. 8. Mahasiswa dapat mengambil Surat Izin Dekan di loket akademik.
2	Berapa lama waktu yang dibutuhkan untuk dapat melakukan legalisir ijazah dan atau transkrip ?	SOP Legalisir Ijazah dan Transkrip	Proses legalisir maksimal dilakukan selama 3 hari kerja.
3	Berapa jumlah maksimal lembar legalisir yang dapat mintakan untuk ijazah dan atau transkrip ?	SOP Legalisir Ijazah dan Transkrip	Masing-masing maksimal sebanyak 10 lembar untuk ijazah dan atau transkrip.
4	Persyaratan apa yang diperlukan	SOP Legalisir	Pihak perwakilan wajib membawa surat kuasa dari pihak yang bersangkutan. Namun surat kuasa tidak perlu diberi materai.

	apabila pengambilan legalisir ijazah dan atau transkrip diwakilkan oleh pihak lain ?	Ijazah dan Transkrip	
5	Bagaimana prosedur dan apa syarat yang diperlukan untuk dapat melakukan pengajuan legalisir ijazah dan atau transkrip ?	SOP Legalisir Ijazah dan Transkrip	1. Alumni menyerahkan foto copy ijazah/transkrip nilai/sertifikat akreditasi program studi beserta surat aslinya kepada petugas Subbag Akademik dan Kemahasiswaan. Dokumen yang dibutuhkan: Berkas Pengajuan Legalisir 2. Petugas Subbag Akademik dan Kemahasiswaan memeriksa keabsahan, kesesuaian foto copy ijazah/transkrip nilai/sertifikat akreditasi program studi dengan surat aslinya dan memberikan cap. 3. Supervisor Akademik dan Kemahasiswaan memeriksa dan memberi paraf di samping kanan nama Dekan. 4. Dekan/Wakil Dekan Akademik dan Kemahasiswaan menandatangani pada "Pengesahan Telah diperiksa Kebenarannya dan Sesuai dengan Aslinya". 5. Petugas Subbag Akademik dan Kemahasiswaan mengambil kembali Legalisir yang sudah ditanda tangani Dekan / Wakil Dekan Akademik dan Kemahasiswaan dan memberikan stempel Fakultas Sains dan Matematika. 6. Alumni mengambil Legalisir di Subbag Akademik dan Kemahasiswaan dengan membawa aslinya dan menulis di buku pengambilan dan di tanda tangani ybs.
6	Apa syarat mengajukan proposal kegiatan yang terkait dengan orientasi mahasiswa baru ?	SOP Pengajuan Proposal Kegiatan Organisasi Mahasiswa	Semua kegiatan yang berkaitan dengan mahasiswa baru diwajibkan membuat surat pernyataan dari dosen pendamping.
7	Bagaimana prosedur yang dapat dilakukan untuk mendapatkan rekomendasi beasiswa ?	SOP Pengajuan Rekomendasi Beasiswa	1. Mahasiswa mendownload dan mengisi formulir/ Surat Rekomendasi Pengajuan Beasiswa lalu membawanya ke loket Subbag Akademik dan Kemahasiswaan (BAK) Fakultas. 2. BAK Fakultas meneliti dan memverifikasi Surat Rekomendasi Pengajuan Beasiswa. 3. BAK Fakultas memberikan paraf dan nomor surat pada Surat Rekomendasi Pengajuan Beasiswa tersebut. 4. Wakil Dekan Akademik dan Kemahasiswaan memberikan tanda tangan pada Surat Rekomendasi Pengajuan Beasiswa tersebut. 5. Mahasiswa mengambil surat tersebut di BAK Fakultas dan mencatat bukti pengambilan.
8	Bagaimana jika mata kuliah yang ingin diambil belum muncul di IRS ?	SOP Pengisian IRS	Untuk dapat memperoleh bantuan lebih lanjut, bisa hubungi operator prodi masing-masing.
9	Bagaimana jika saya terlambat melakukan pembayaran	SOP Pengisian IRS, SOP Permohonan Izin	Ajukan izin keterlambatan pembayaran UKT (maksimal 3 hari kerja) dengan prosedur : 1. Mahasiswa mengunduh, mengisi, dan menandatangani Form Permohonan Keterlambatan Pembayaran UKT serta melampirkan persyaratan.

	UKT dan pengisian IRS?	Keterlambatan Pembayaran UKT	2. Mahasiswa meminta tanda tangan dosen wali serta persetujuan dan tanda tangan Ketua Program Studi. 3. Mahasiswa menyerahkan Form Permohonan Keterlambatan UKT yang telah ditandatangani dan disetujui kepada Supervisor Sumber Daya. 4. Supervisor Sumber Daya menerima, memeriksa, dan meneliti kelengkapan persyaratan form, serta memberikan paraf. 5. Wakil Dekan Sumber Daya memproses Surat Permohonan Izin Keterlambatan Pembayaran UKT. 6. Wakil Dekan Sumber Daya menandatangani Surat Permohonan Izin Keterlambatan Pembayaran UKT. 7. Subbag Sumber Daya memproses Surat Permohonan Keterlambatan Pembayaran UKT. 8. Mahasiswa membawa Surat Permohonan Keterlambatan Pembayaran UKT kepada Wakil Rektor II melalui Manajer Akademik. 9. Wakil Rektor II mendisposisikan ke Direktorat Keuangan untuk memproses pembayaran UKT. 10. Bendahara Penerimaan UKT membuka sistem pembayaran mahasiswa. 11. Mahasiswa membayar biaya UKT ke bank. Setelah sistem terbuka dan UKT berhasil dibayar, segera lakukan pengisian IRS (maksimal 3 hari kerja) dengan prosedur : 1. Mahasiswa melakukan her-registrasi online melalui SIAP. 2. Mahasiswa menemui Pembimbing Akademik untuk berkonsultasi mengenai mata kuliah yang akan diambil. Pembimbing memberikan arahan dengan mempertimbangkan ketentuan jumlah SKS maksimal pada semester terkait. 3. Mahasiswa melakukan pengisian, perbaikan, atau mencetak IRS sementara dengan bantuan operator masing-masing prodi secara online melalui SIAP. 4. Apabila mata kuliah dan jumlah SKS sudah sesuai, Pembimbing Akademik melakukan persetujuan secara online pada SIAP. 5. Mahasiswa dapat melakukan pengecekan IRS online di sistem SIAP bila diperlukan.
10	Saya sempat cuti, lalu bagaimana prosedurnya untuk saya dapat aktif kembali dalam akademik ?	SOP Izin Aktif Setelah Cuti	1. Mahasiswa melakukan registrasi online pada SSO masing-masing mahasiswa. 2. Mahasiswa terdaftar sebagai Peserta Kuliah / Mahasiswa Aktif di Fakultas.

Semarang, 4 November 2025

Pewawancara,

Alfonso Clement Sutantio

NIM. 24060122130080

Narasumber,

Sukarni, S.E.

NIP. 197504162007012001

LAMPIRAN 2. Isi Mentah Dokumen Sumber Pengetahuan

1. SOP pengisian IRS

NO.	KEGIATAN	PELAKSANA			MUTU BAKU				KETERANGAN
		Mahasiswa	Pembimbing Akademik	Staff Administrasi Prodi	Kelengkapan	Waktu		Output	
1.	Mahasiswa membayar biaya pendidikan pada bank yang bekerjasama dengan Undip (BNI, Mandiri, BTN dan BRI).					10	menit	Bukti Pembayaran SPP/UKT	
2.	Mahasiswa melakukan her-registrasi online melalui SIAP.					10	menit		
3.	Mahasiswa melakukan pengisian/perbaikan/mencetak IRS sementara secara online pada SIAP.					10	menit		
4.	Mahasiswa menemui Pembimbing Akademik untuk berkonsultasi tentang mata kuliah yang diambil dan Pembimbing Akademik memberikan arahan dan masukan dengan mempertimbangkan ketentuan jumlah sks maksimal yang boleh diambil dalam semester terkait.		N Y			10	menit	Print out IRS / online IRS	
5.	Apabila mata kuliah dan jumlah sks sudah sesuai, maka Pembimbing Akademik melakukan persetujuan secara online pada pada SIAP					10	menit	Persetujuan Pembimbing Akademik melalui SIAP	
6.	Cek IRS online di system SIAP.bila diperlukan					10	menit	File IRS SIAP	
	Total Pelayanan (Maximal)					3	hari		

2. SOP permohonan izin aktif kuliah setelah cuti

NO.	KEGIATAN	PELAKSANA			MUTU BAKU				KETERANGAN
		Mahasiswa	Ketua Program Studi	Subbag Akademik dan Kmhs	Kelengkapan	Waktu		Output	
1.	Mahasiswa membawa surat izin cuti akademik semester melapor kepada Ketua Program Studi dan Subbag Akademik dan Kemahasiswaan.					5	menit	- Surat Izin Cuti. - Fotokopi Kartu Tanda Mahasiswa (KTM).	
2.	Mahasiswa melakukan registrasi online pada SSO masing-masing mahasiswa.					5	menit		
3.	Mahasiswa terdaftar sebagai Peserta Kuliah / Mahasiswa Aktif di Fakuktas					5	menit		

3. SOP permohonan izin cuti akademik

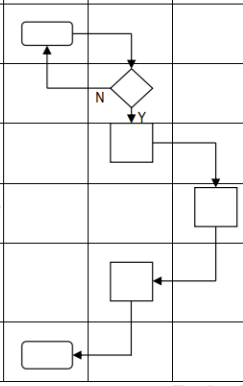
NO.	KEGIATAN	PELAKSANA					MUTU BAKU			KETERANGAN
		Mahasiswa	Ketua Program Studi	Subbag Akademik dan Kmhs	Subbag Sumber Daya	Dekan	Kelengkapan	Waktu	Output	
1.	Mahasiswa mengunduh, mengisi dan menandatangani form cuti akademik dengan melampirkan persyaratan. Selain itu, mahasiswa mengisi secara online pada SIAP.								- Form Cuti Akademik - Transkrip akademik. - Bukti bayar SPP / UKT terakhir. - fotokopi KTM. - Dokumen pendukung.	Unduh form cuti akademik: Download
2.	Mahasiswa meminta persetujuan dan tanda tangan Ketua Program Studi.		N / Y							
3.	Mahasiswa menyerahkan form cuti akademik yang telah ditandatangani dan disetujui oleh Ketua Program Studi ke Dekan. Kemudian melakukan disposisi ke Subbag Akademik dan Kemahasiswaan.									
4.	Supervisor Akademik dan Kemahasiswaan menerima, memeriksa, meneliti kelengkapan persyaratan form cuti akademik, memberikan paraf dan memprosesnya.			N / Y						
5.	Dekan menandatangani Surat Izin Dekan.								Surat Izin Dekan	
6.	Subbag Sumber Daya memberi nomor surat, mengirim ke Subbag Akademik dan Kemahasiswaan dengan tembusan Surat Izin Dekan ke Dosen Wali dan Ketua Program Studi.								Surat Izin Dekan	

4. SOP permohonan izin keterlambatan pembayaran UKT

NO.	KEGIATAN	PELAKSANA							MUTU BAKU			KETERA-NGAN
		Mahasiswa	Ketua Program Studi	Subbag Akademik dan Kmhs	Subbag Sumber Daya	Wakil Dekan Akademik dan Kmhs	Direktorat Akademik	Direktorat Keuangan, Akuntansi dan Perpajakan	Kelengkapan	Waktu	Output	
1.	Mahasiswa mengunduh, mengisi dan menandatangani form permohonan keterlambatan UKT dengan melampirkan persyaratan. https://drive.google.com/file/d/1o7hogihjZsFTB_V27JU1OfNGBuI2Sr55/											
2.	Mahasiswa meminta tanda tangan dosen wali, persetujuan dan tanda tangan Ketua Program Studi.		N / Y									
3.	Mahasiswa menyerahkan form permohonan keterlambatan UKT yang telah ditandatangani dan disetujui oleh Ketua Program Studi ke Supervisor sumber daya											
4.	Supervisor Sunber Daya menerima, memeriksa, meneliti kelengkapan persyaratan form permohonan keterlambatan UKT dan memberikan paraf.			N / Y								
5.	Wakil Dekan Sumber Daya memproses surat permohonan izin keterlambatan pembayaran UKT.											
6.	Wakil Dekan Sumber Daya menandatangani surat permohonan izin keterlambatan pembayaran UKT.											

5. SOP legalisir ijazah dan transkrip

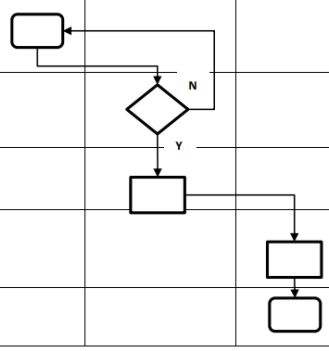
NO.	KEGIATAN	PELAKSANA			MUTU BAKU				KET
		Alumni	Subbag Akademik dan Kmhs	Wakil Dekan Akademik dan Kmhs	Kelengkapan	Waktu		Output	
1.	Alumni menyerahkan foto copy ijazah/transkrip nilai/sertifikat akreditasi program studi beserta surat aslinya kepada petugas Subbag Akademik dan Kemahasiswaan.					5	menit	Berkas Pengajuan Legalisir	
2.	Petugas Subbag Akademik dan Kemahasiswaan memeriksa keabsahan, kesesuaian foto copy ijazah/transkrip nilai/sertifikat akreditasi program studi dengan surat aslinya dan memberikan cap					10	menit	Persetujuan Legalisir	
3.	Supervisor Akademik dan Kemahasiswaan memeriksa dan memberi paraf di samping kanan nama Dekan					15	menit	Paraf Supervisor Akademik dan Kemahasiswaan	
4.	Dekan/Wakil Dekan Akademik dan Kemahasiswaan menandatangani pada "***Pengesahan Telah diperiksa Kebenarannya dan Sesuai dengan Aslinya***".					1	hari	TTD Dekan/Wakil Dekan Akademik dan Kemahasiswaan	
5.	Petugas Subbag Akademik dan Kemahasiswaan mengambil kembali Legalisir yang sudah ditanda tangani Dekan / Wakil Dekan Akademik dan Kemahasiswaan dan memberikan stempel Fakultas Sains dan Matematika.					5	menit	Stempel Fakultas Sains dan Matematika	
6.	Alumni mengambil Legalisir di Subbag Akademik dan Kemahasiswaan dengan membawa aslinya dan menulis di buku pengambilan dan di tanda tangani ybs.					15	menit	Legalisir Ijazah/transkrip nilai/sertifikat akreditasi	



Total waktu (maksimal): 3 hari kerja

6. SOP pengajuan rekomendasi beasiswa

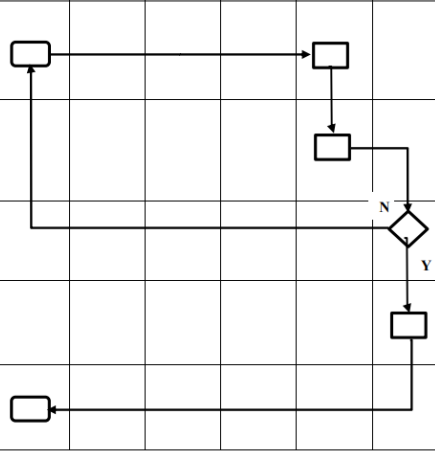
No	Kegiatan	PELAKSANA			MUTU BAKU				Keterangan
		Mahasiswa	Subbag Akademik dan Kemahasiswaan (BAK) Fakultas	Wakil Dekan Akademik dan Kemahassiswaan	Kelengkapan	Waktu		Output	
1	Mahasiswa mendownload dan mengisi formulir/ Surat Rekomendasi Pengajuan Beasiswa ke Subbag Akademik dan Kemahasiswaan (BAK) Fakultas.				Surat Rekomendasi Beasiswa dan KHS yang dilegalisir	30	menit	Surat Rekomendasi Beasiswa dan KHS yang dilegalisir	Link Download: https://fsm.undip.ac.id/download-area/
2	BAK Fakultas meneliti dan memverifikasi Surat Rekomendasi Pengajuan Beasiswa.					1	hari		
3	BAK Fakultas memberikan paraf dan nomor surat pada Surat Rekomendasi Pengajuan Beasiswa tersebut					10	menit		
4	Wakil Dekan Akademik dan Kemahasiswaan memberikan tanda tangan pada Surat Rekomendasi Pengajuan Beasiswa tersebut					2	hari		
5	Mahasiswa mengambil surat tersebut di BAK Fakultas dan mencatat bukti pengambilan					5	menit	Surat Rekomendasi Beasiswa	



Total Pelayanan (Maksima): 3 hari 45 menit

7. SOP pengajuan proposal kegiatan organisasi mahasiswa

NO.	KEGIATAN	PELAKSANA						MUTU BAKU				KETERANG-AN
		Mahasiswa	Pembim-bing	Ketua Departe-men	BEM dan Senat Mahasiswa	Subbag Akademik dan Kmhs	Wakil Dekan Akademik dan Kmhs	Kelengkapan	Waktu		Output	
1.	Mahasiswa menyerahkan proposal ke petugas kemahasiswaan dan Supervisor Akademik untuk registrasi dan alokasi ruang (jika kegiatan di FSM).											
2.	Supervisor Akademik dan Kemahasiswaan meneliti dan memverifikasi proposal kegiatan yang diajukan agar sesuai peraturan dan memberikan paraf.										Paraf Supervisor Akademik dan Kemahasiswaan	
3.	Wakil Dekan Akademik dan Kemahasiswaan mengevaluasi substansi proposal kegiatan.										Persetujuan Wakil Dekan Akademik dan Kemahasiswaan	
4.	Pengesahan Wakil Dekan Akademik dan Kemahasiswaan.											
5.	Mahasiswa mengambil proposal yang telah disetujui.										Proposal kegiatan organisasi	
Total Waktu Pelayanan (Maximal)									3	hari		



LAMPIRAN 3. Isi Dokumen Hasil Penyesuaian Manual

1. SOP pengisian IRS

Standar Operasional Prosedur (SOP) Pengisian Isian Rencana Studi (IRS)

Dokumen ini berisi langkah-langkah yang dapat diikuti untuk dapat mengisi Isian Rencana Studi (IRS) dengan total maksimal waktu pemrosesan maksimal 3 (tiga) hari kerja.

1. Mahasiswa membayar biaya pendidikan pada bank yang bekerja sama dengan Undip (BNI, Mandiri, BTN, dan BRI).
   Waktu: ±10 menit
   Output: Bukti Pembayaran SPP/UKT
2. Mahasiswa melakukan her-registrasi online melalui SIAP.
   Waktu: ±10 menit
3. Mahasiswa melakukan pengisian, perbaikan, atau mencetak IRS sementara secara online melalui SIAP.
   Waktu: ±10 menit
4. Mahasiswa menemui Pembimbing Akademik untuk berkonsultasi mengenai mata kuliah yang akan diambil. Pembimbing memberikan arahan dengan mempertimbangkan ketentuan jumlah SKS maksimal pada semester terkait.
   Waktu: ±10 menit
   Output: Print out IRS / Online IRS
5. Apabila mata kuliah dan jumlah SKS sudah sesuai, Pembimbing Akademik melakukan persetujuan secara online pada SIAP.
   Waktu: ±10 menit
   Output: Persetujuan Pembimbing Akademik melalui SIAP
6. Mahasiswa dapat melakukan pengecekan IRS online di sistem SIAP bila diperlukan.
   Waktu: ±10 menit
   Output: File IRS SIAP

2. SOP permohonan izin aktif kuliah setelah cuti

Standar Operasional Prosedur (SOP) Permohonan Izin Aktif Setelah Cuti

Dokumen ini berisi langkah-langkah yang dapat diikuti untuk dapat memperoleh izin aktif setelah cuti.

1. Mahasiswa membawa surat izin cuti akademik semester melapor kepada Ketua Program Studi dan Subbag Akademik dan Kemahasiswaan.
   Waktu: ±5 menit
   Dokumen yang dibutuhkan: Surat Izin Cuti dan Fotokopi Kartu Tanda Mahasiswa (KTM)
2. Mahasiswa melakukan registrasi online pada SSO masing-masing mahasiswa.
   Waktu: ±5 menit
3. Mahasiswa terdaftar sebagai Peserta Kuliah / Mahasiswa Aktif di Fakuktas.
   Waktu: ±5 menit

3. SOP permohonan izin cuti akademik

Standar Operasional Prosedur (SOP) Permohonan Izin Cuti Akademik

Dokumen ini berisi langkah-langkah yang dapat diikuti untuk memperoleh izin cuti akademik dengan total maksimal waktu pemrosesan maksimal 3 (tiga) hari kerja.

1. Mahasiswa mengunduh, mengisi, dan menandatangani Form Cuti Akademik dengan melampirkan persyaratan. Selain itu, mahasiswa juga wajib mengisi secara online melalui SIAP.
   Dokumen yang dibutuhkan: Form Cuti Akademik, Transkrip Akademik, Bukti bayar SPP/UKT terakhir, Fotokopi KTM, dan dokumen pendukung lain
   Link unduh form: https://drive.google.com/file/d/1miPHNvRj6XO6LEhFCiC_QXcMtynCPWR2/edit
2. Mahasiswa meminta persetujuan dan tanda tangan Ketua Program Studi.
3. Mahasiswa menyerahkan Form Cuti Akademik yang telah ditandatangani Ketua Program Studi ke Dekan. Kemudian dilakukan disposisi ke Subbag Akademik dan Kemahasiswaan.
4. Subbag Akademik dan Kemahasiswaan menerima, memeriksa, dan meneliti kelengkapan persyaratan form cuti akademik. Jika lengkap, form diberi paraf dan diproses lebih lanjut.
5. Dekan menandatangani Surat Izin Dekan.
   Output: Surat Izin Dekan
6. Subbag Sumber Daya memberi nomor surat dan mengirimkannya ke Subbag Akademik dan Kemahasiswaan dengan tembusan ke Dosen Wali dan Ketua Program Studi.
   Output: Surat Izin Dekan
7. Subbag Akademik dan Kemahasiswaan mengupdate status mahasiswa di Sistem Akademik, mengarsipkan Surat Izin Dekan, dan mengirimkan kepada mahasiswa.
   Output: Surat Izin Dekan
8. Mahasiswa dapat mengambil Surat Izin Dekan di loket akademik.
   Output: Surat Izin Dekan

4. SOP permohonan izin keterlambatan pembayaran UKT

Standar Operasional Prosedur (SOP) Permohonan Izin Keterlambatan Pembayaran UKT

Dokumen ini berisi langkah-langkah yang dapat diikuti untuk memperoleh izin keterlambatan pembayaran UKT dengan total maksimal waktu pemrosesan maksimal 3 (tiga) hari kerja.

1. Mahasiswa mengunduh, mengisi, dan menandatangani Form Permohonan Keterlambatan Pembayaran UKT serta melampirkan persyaratan.
   Link unduh form: https://drive.google.com/file/d/1o7hogihjZsFTB_V27JU1OfNGBuI2Sr55/
2. Mahasiswa meminta tanda tangan dosen wali serta persetujuan dan tanda tangan Ketua Program Studi.
3. Mahasiswa menyerahkan Form Permohonan Keterlambatan UKT yang telah ditandatangani dan disetujui kepada Supervisor Sumber Daya.
4. Supervisor Sumber Daya menerima, memeriksa, dan meneliti kelengkapan persyaratan form, serta memberikan paraf.
5. Wakil Dekan Sumber Daya memproses Surat Permohonan Izin Keterlambatan Pembayaran UKT.
6. Wakil Dekan Sumber Daya menandatangani Surat Permohonan Izin Keterlambatan Pembayaran UKT.
7. Subbag Sumber Daya memproses Surat Permohonan Keterlambatan Pembayaran UKT.
8. Mahasiswa membawa Surat Permohonan Keterlambatan Pembayaran UKT kepada Wakil Rektor II melalui Manajer Akademik.
9. Wakil Rektor II mendisposisikan ke Direktorat Keuangan untuk memproses pembayaran UKT.
10. Bendahara Penerimaan UKT membuka sistem pembayaran mahasiswa.
11. Mahasiswa membayar biaya UKT ke bank.

5. SOP legalisir ijazah dan transkrip

Standar Operasional Prosedur (SOP) Legalisir Ijazah Dan Transkrip

Dokumen ini berisi langkah-langkah yang dapat diikuti untuk dapat melalukan legalisir ijazah dan transkrip dengan total maksimal waktu pemrosesan maksimal 3 (tiga) hari kerja.

1. Alumni menyerahkan foto copy ijazah/transkrip nilai/sertifikat akreditasi program studi beserta surat aslinya kepada petugas Subbag Akademik dan Kemahasiswaan.
   Waktu: ±5 menit
   Dokumen yang dibutuhkan: Berkas Pengajuan Legalisir
   Output: Berkas Pengajuan Legalisir
2. Petugas Subbag Akademik dan Kemahasiswaan memeriksa keabsahan, kesesuaian foto copy ijazah/transkrip nilai/sertifikat akreditasi program studi dengan surat aslinya dan memberikan cap.
   Waktu: ±10 menit
   Output: Persetujuan Legalisir
3. Supervisor Akademik dan Kemahasiswaan memeriksa dan memberi paraf di samping kanan nama Dekan.
   Waktu: ±15 menit
   Output: Paraf Supervisor Akademik dan Kemahasiswaan
4. Dekan/Wakil Dekan Akademik dan Kemahasiswaan menandatangani pada "Pengesahan Telah diperiksa Kebenarannya dan Sesuai dengan Aslinya".
   Waktu: ±1 hari
   Output: TTD Dekan/Wakil Dekan Akademik dan Kemahasiswaan
5. Petugas Subbag Akademik dan Kemahasiswaan mengambil kembali Legalisir yang sudah ditanda tangani Dekan / Wakil Dekan Akademik dan Kemahasiswaan dan memberikan stempel Fakultas Sains dan Matematika.
   Waktu: ±5 menit
   Output: Stempel Fakultas Sains dan Matematika
6. Alumni mengambil Legalisir di Subbag Akademik dan Kemahasiswaan dengan membawa aslinya dan menulis di buku pengambilan dan di tanda tangani ybs.
   Waktu: ±15 menit
   Output: Legalisir Ijazah/transkrip nilai/sertifikat akreditasi

6. SOP pengajuan rekomendasi beasiswa

Standar Operasional Prosedur (SOP) Surat Pengajuan Rekomendasi Beasiswa

Dokumen ini berisi langkah-langkah yang dapat diikuti untuk mengajukan dan mendapatkan rekomendasi beasiswa dengan total maksimal waktu pemrosesan maksimal 3 (tiga) hari, 45 menit waktu kerja.

1. Mahasiswa mendownload dan mengisi formulir/ Surat Rekomendasi Pengajuan Beasiswa ke Subbag Akademik dan Kemahasiswaan (BAK) Fakultas.
   Waktu: ±30 menit
   Dokumen yang dibutuhkan: Surat Rekomendasi Beasiswa dan KHS yang dilegalisir
   Link unduh form: https://drive.google.com/file/d/18f_nbPXHbElgRNoplDFQVNYGGDkFxBTT/view?usp=sharing
   Output: Surat Rekomendasi Beasiswa dan KHS yang dilegalisir
2. BAK Fakultas meneliti dan memverifikasi Surat Rekomendasi Pengajuan Beasiswa.
   Waktu: ±1 hari
3. BAK Fakultas memberikan paraf dan nomor surat pada Surat Rekomendasi Pengajuan Beasiswa tersebut.
   Waktu: ±10 menit
4. Wakil Dekan Akademik dan Kemahasiswaan memberikan tanda tangan pada Surat Rekomendasi Pengajuan Beasiswa tersebut.
   Waktu: ±2 hari
5. Mahasiswa mengambil surat tersebut di BAK Fakultas dan mencatat bukti pengambilan.
   Waktu: ±5 menit

7. SOP pengajuan proposal kegiatan organisasi mahasiswa

Standar Operasional Prosedur (SOP) Pengajuan Proposal Kegiatan Organisasi Mahasiswa

Dokumen ini berisi langkah-langkah yang dapat diikuti untuk dapat melalukan pengajuan proposal kegiatan organisasi mahasiswa dengan total maksimal waktu pemrosesan maksimal 3 (tiga) hari kerja.

1. Mahasiswa menyerahkan proposal ke petugas kemahasiswaan dan Supervisor Akademik untuk registrasi dan alokasi ruang (jika kegiatan di FSM).
   Dokumen yang dibutuhkan: Proposal kegiatan organisasi
2. Supervisor Akademik dan Kemahasiswaan meneliti dan memverifikasi proposal kegiatan yang diajukan agar sesuai peraturan dan memberikan paraf.
   Output: Paraf Supervisor Akademik dan Kemahasiswaan
3. Wakil Dekan Akademik dan Kemahasiswaan mengevaluasi substansi proposal kegiatan.
   Output: Persetujuan Wakil Dekan Akademik dan Kemahasiswaan
4. Pengesahan Wakil Dekan Akademik dan Kemahasiswaan.
5. Mahasiswa mengambil proposal yang telah disetujui.
   Output: Proposal kegiatan organisasi yang telah disetujui

LAMPIRAN 4. Jawaban dan *Ground Truth* Evaluasi Model RAG

No	Pertanyaan	Jawaban	*Ground Truth*
1	Bagaimana proses pengajuan cuti akademik ?	Berikut proses pengajuan cuti akademik (maksimal diproses 3 hari kerja): 1. Unduh, isi, dan tandatangani Form Cuti Akademik serta unggah/siapkan persyaratan. Mahasiswa juga wajib mengisi pengajuan cuti secara online melalui SIAP. - Dokumen yang dibutuhkan: Form Cuti Akademik, Transkrip Akademik, bukti bayar SPP/UKT terakhir, fotokopi KTM, dan dokumen pendukung lain. - Link form: https://drive.google.com/file/d/1miPHNvRj6XO6LEhFCiC_QXcMtynCPWR2/edit 2. Minta persetujuan dan tanda tangan Ketua Program Studi pada Form Cuti Akademik. 3. Serahkan Form Cuti Akademik yang telah ditandatangani Ketua Program Studi ke Dekan (akan didisposisi ke Subbag Akademik dan Kemahasiswaan). 4. Subbag Akademik dan Kemahasiswaan memeriksa kelengkapan. Jika lengkap, form diparap dan diproses lanjut. 5. Dekan menandatangani Surat Izin Dekan (Output: Surat Izin Dekan). 6. Subbag Sumber Daya memberi nomor surat dan meneruskan ke Subbag Akademik dan Kemahasiswaan dengan tembusan ke Dosen Wali dan Ketua Program Studi (Output: Surat Izin Dekan). 7. Subbag Akademik dan Kemahasiswaan mengupdate status mahasiswa di Sistem Akademik, mengarsipkan, dan mengirimkan Surat Izin Dekan kepada mahasiswa (Output: Surat Izin Dekan). 8. Mahasiswa mengambil Surat Izin Dekan di loket akademik (Output: Surat Izin Dekan).	1. Mahasiswa mengunduh, mengisi, dan menandatangani Form Cuti Akademik dengan melampirkan persyaratan. Selain itu, mahasiswa juga wajib mengisi secara online melalui SIAP. Dokumen yang dibutuhkan: Form Cuti Akademik, Transkrip Akademik, Bukti bayar SPP/UKT terakhir, Fotokopi KTM, dan dokumen pendukung lain Link unduh form: https://drive.google.com/file/d/1miPHNvRj6XO6LEhFCiC_QXcMtynCPWR2/edit 2. Mahasiswa meminta persetujuan dan tanda tangan Ketua Program Studi. 3. Mahasiswa menyerahkan Form Cuti Akademik yang telah ditandatangani Ketua Program Studi ke Dekan. Kemudian dilakukan disposisi ke Subbag Akademik dan Kemahasiswaan. 4. Subbag Akademik dan Kemahasiswaan menerima, memeriksa, dan meneliti kelengkapan persyaratan form cuti akademik. Jika lengkap, form diberi paraf dan diproses lebih lanjut. 5. Dekan menandatangani Surat Izin Dekan. 6. Subbag Sumber Daya memberi nomor surat dan mengirimkannya ke Subbag Akademik dan Kemahasiswaan dengan tembusan ke Dosen Wali dan Ketua Program Studi. 7. Subbag Akademik dan Kemahasiswaan mengupdate status mahasiswa di Sistem Akademik, mengarsipkan Surat Izin Dekan, dan mengirimkan kepada mahasiswa. 8. Mahasiswa dapat mengambil Surat Izin Dekan di loket akademik.

No	Pertanyaan	Jawaban	*Ground Truth*
2	Berapa lama waktu yang dibutuhkan untuk dapat melakukan legalisir ijazah dan atau transkrip ?	Maksimal 3 (tiga) hari kerja sejak berkas diajukan. Proses mencakup pemeriksaan, paraf, tanda tangan Dekan/Wakil Dekan (±1 hari), stempel, lalu pengambilan oleh alumni.	Proses legalisir maksimal dilakukan selama 3 hari kerja.
3	Berapa jumlah maksimal lembar legalisir yang dapat mintakan untuk ijazah dan atau transkrip ?	Maaf, pada SOP yang tersedia tidak disebutkan batas maksimal jumlah lembar legalisir ijazah dan/atau transkrip. Untuk kepastian, silakan menghubungi Subbag Akademik dan Kemahasiswaan FSM.	Masing-masing maksimal sebanyak 10 lembar untuk ijazah dan atau transkrip.
4	Persyaratan apa yang diperlukan apabila pengambilan legalisir ijazah dan atau transkrip diwakilkan oleh pihak lain ?	Dalam SOP yang tersedia, tidak dijelaskan persyaratan jika pengambilan legalisir ijazah/transkrip diwakilkan. SOP hanya menyebutkan alumni mengambil sendiri dengan membawa dokumen asli dan menandatangani buku pengambilan. Untuk pengambilan oleh pihak lain, silakan konfirmasi langsung ke Subbag Akademik dan Kemahasiswaan FSM (loket akademik) untuk memastikan persyaratan dan prosedur terbaru.	Pihak perwakilan wajib membawa surat kuasa dari pihak yang bersangkutan. Namun surat kuasa tidak perlu diberi materai.
5	Bagaimana prosedur dan apa syarat yang diperlukan untuk dapat melakukan pengajuan legalisir ijazah dan atau transkrip ?	Berikut prosedur dan syarat pengajuan legalisir ijazah dan/atau transkrip di Fakultas Sains dan Matematika UNDIP: Syarat berkas: - Fotokopi ijazah dan/atau transkrip nilai yang akan dilegalisir. - (Opsional bila diperlukan) Fotokopi sertifikat akreditasi program studi. - Masing-masing disertai dokumen aslinya untuk dicocokkan. Alur/prosedur (maksimal 3 hari kerja): 1) Alumni menyerahkan fotokopi beserta dokumen aslinya ke petugas Subbag Akademik dan Kemahasiswaan (±5 menit). 2) Petugas memeriksa keabsahan dan kesesuaian dengan dokumen asli serta memberi cap (±10 menit). 3) Supervisor Akademik dan Kemahasiswaan memeriksa dan memberi paraf di samping kanan nama Dekan (±15 menit). 4) Dekan/Wakil Dekan Akademik dan Kemahasiswaan menandatangani pengesahan “Telah diperiksa kebenarannya dan sesuai	1. Alumni menyerahkan foto copy ijazah/transkrip nilai/sertifikat akreditasi program studi beserta surat aslinya kepada petugas Subbag Akademik dan Kemahasiswaan. Dokumen yang dibutuhkan: Berkas Pengajuan Legalisir 2. Petugas Subbag Akademik dan Kemahasiswaan memeriksa keabsahan, kesesuaian foto copy ijazah/transkrip nilai/sertifikat akreditasi program studi dengan surat aslinya dan memberikan cap. 3. Supervisor Akademik dan Kemahasiswaan memeriksa dan memberi paraf di samping kanan nama Dekan. 4. Dekan/Wakil Dekan Akademik dan Kemahasiswaan menandatangani pada “Pengesahan Telah diperiksa Kebenarannya dan Sesuai dengan Aslinya”.

No	Pertanyaan	Jawaban	*Ground Truth*
		dengan aslinya" (±1 hari). 5) Petugas Subbag mengambil kembali berkas yang sudah ditandatangani dan memberi stempel Fakultas Sains dan Matematika (±5 menit). 6) Alumni mengambil hasil legalisir di Subbag Akademik dan Kemahasiswaan dengan membawa dokumen asli, mengisi buku pengambilan, dan menandatangani (±15 menit). Catatan: Waktu pemrosesan total maksimal 3 hari kerja. Tidak ada informasi tambahan terkait biaya atau jumlah rangkap pada dokumen yang tersedia.	5. Petugas Subbag Akademik dan Kemahasiswaan mengambil kembali Legalisir yang sudah ditanda tangani Dekan / Wakil Dekan Akademik dan Kemahasiswaan dan memberikan stempel Fakultas Sains dan Matematika. 6. Alumni mengambil Legalisir di Subbag Akademik dan Kemahasiswaan dengan membawa aslinya dan menulis di buku pengambilan dan di tanda tangani ybs.
6	Apa syarat mengajukan proposal kegiatan yang terkait dengan orientasi mahasiswa baru ?	Berdasarkan SOP Pengajuan Proposal Kegiatan Organisasi Mahasiswa FSM Undip, syaratnya adalah menyiapkan dan menyerahkan Proposal Kegiatan Organisasi kepada petugas kemahasiswaan dan Supervisor Akademik (sekalian registrasi dan alokasi ruang jika kegiatan dilaksanakan di FSM). Prosesnya: - Diverifikasi dan diberi paraf oleh Supervisor Akademik dan Kemahasiswaan. - Dievaluasi dan disetujui oleh Wakil Dekan Akademik dan Kemahasiswaan. - Disahkan, lalu proposal yang telah disetujui diambil oleh mahasiswa. Waktu pemrosesan maksimal 3 hari kerja. Tidak ada ketentuan tambahan khusus untuk kegiatan orientasi mahasiswa baru dalam dokumen yang tersedia.	Semua kegiatan yang berkaitan dengan mahasiswa baru diwajibkan membuat surat pernyataan dari dosen pendamping.
7	Bagaimana prosedur yang dapat dilakukan untuk mendapatkan rekomendasi beasiswa ?	Berikut prosedur pengajuan rekomendasi beasiswa sesuai SOP FSM Undip: - Unduh dan isi Formulir Surat Rekomendasi Pengajuan Beasiswa, siapkan KHS yang sudah dilegalisir. Link formulir: https://drive.google.com/file/d/18f_nbPXHbElgRNoplDFQVNYGGDkFxBTT/view?usp=sharing - Serahkan formulir dan KHS legalisir ke Subbag Akademik dan Kemahasiswaan (BAK) Fakultas	1. Mahasiswa mendownload dan mengisi formulir/ Surat Rekomendasi Pengajuan Beasiswa ke loket Subbag Akademik dan Kemahasiswaan (BAK) Fakultas. 2. BAK Fakultas meneliti dan memverifikasi Surat Rekomendasi Pengajuan Beasiswa.

No	Pertanyaan	Jawaban	*Ground Truth*
		untuk diteliti dan diverifikasi (±1 hari). - BAK memberikan paraf dan nomor surat pada Surat Rekomendasi Pengajuan Beasiswa (±10 menit). - Wakil Dekan Akademik dan Kemahasiswaan menandatangani surat rekomendasi tersebut (±2 hari). - Ambil surat yang sudah selesai di BAK Fakultas dan lakukan pencatatan bukti pengambilan (±5 menit). Estimasi total waktu pemrosesan: maksimal 3 hari kerja 45 menit.	3. BAK Fakultas memberikan paraf dan nomor surat pada Surat Rekomendasi Pengajuan Beasiswa tersebut. 4. Wakil Dekan Akademik dan Kemahasiswaan memberikan tanda tangan pada Surat Rekomendasi Pengajuan Beasiswa tersebut. 5. Mahasiswa mengambil surat tersebut di BAK Fakultas dan mencatat bukti pengambilan.
8	Bagaimana jika mata kuliah yang ingin diambil belum muncul di IRS ?	Berdasarkan SOP IRS: - Pastikan Anda sudah membayar SPP/UKT dan melakukan her-registrasi di SIAP, lalu coba isi IRS sementara. - Konsultasikan rencana mata kuliah dengan Pembimbing Akademik. SOP tidak memuat langkah khusus jika mata kuliah belum muncul di SIAP. Jika hal ini terjadi, segera berkonsultasi dengan Pembimbing Akademik untuk arahan, lalu cek kembali IRS Anda di SIAP.	Untuk dapat memperoleh bantuan lebih lanjut, bisa hubungi operator prodi masing-masing.
9	Bagaimana jika saya terlambat melakukan pembayaran UKT dan pengisian IRS?	Berikut yang perlu Anda lakukan jika terlambat bayar UKT dan mengisi IRS: 1) Ajukan izin keterlambatan pembayaran UKT (maksimal proses 3 hari kerja) - Unduh, isi, dan tandatangani Form Permohonan Keterlambatan Pembayaran UKT serta lampirkan persyaratan sesuai yang tercantum pada form: https://drive.google.com/file/d/1o7hogihjZsFTB_V27JU1OfNGBuI2Sr55/ - Minta tanda tangan Dosen Wali dan persetujuan Ketua Program Studi. - Serahkan form ke Supervisor Sumber Daya. - Menunggu pemeriksaan dan paraf Supervisor Sumber Daya. - Wakil Dekan Sumber Daya memproses dan menandatangani Surat Izin Keterlambatan. - Subbag Sumber Daya memproses surat tersebut.	Ajukan izin keterlambatan pembayaran UKT (maksimal 3 hari kerja) dengan prosedur : 1. Mahasiswa mengunduh, mengisi, dan menandatangani Form Permohonan Keterlambatan Pembayaran UKT serta melampirkan persyaratan. 2. Mahasiswa meminta tanda tangan dosen wali serta persetujuan dan tanda tangan Ketua Program Studi. 3. Mahasiswa menyerahkan Form Permohonan Keterlambatan UKT yang telah ditandatangani dan disetujui kepada Supervisor Sumber Daya. 4. Supervisor Sumber Daya menerima, memeriksa, dan meneliti kelengkapan persyaratan form, serta memberikan paraf. 5. Wakil Dekan Sumber Daya memproses Surat Permohonan Izin Keterlambatan Pembayaran UKT. 6. Wakil Dekan Sumber Daya menandatangani Surat Permohonan Izin Keterlambatan Pembayaran UKT. 7. Subbag Sumber Daya memproses Surat Permohonan Keterlambatan Pembayaran UKT. 8. Pihak fakultas membawa Surat Permohonan Keterlambatan

No	Pertanyaan	Jawaban	*Ground Truth*
		- Bawa surat ke Wakil Rektor II melalui Manajer Akademik. - WR II mendisposisikan ke Direktorat Keuangan. - Bendahara Penerimaan UKT membuka sistem pembayaran. - Lakukan pembayaran UKT di bank mitra. 2) Setelah sistem terbuka dan UKT terbayar, segera proses IRS (maksimal proses 3 hari kerja) - Her-registrasi online melalui SIAP (SSO). - Isi/perbaiki/cetak IRS sementara di SIAP. - Konsultasi dengan Pembimbing Akademik terkait mata kuliah/SKS. - Pembimbing Akademik menyetujui IRS secara online di SIAP. - Cek kembali IRS Anda di SIAP bila diperlukan. Catatan: - SOP khusus mengenai prosedur jika melewati batas waktu pengisian IRS tidak tercantum dalam dokumen yang tersedia. Jika Anda sudah lewat tenggat IRS, segera hubungi Pembimbing Akademik dan Ketua Program Studi/Subbag Akademik untuk arahan lebih lanjut.	Pembayaran UKT kepada Wakil Rektor II melalui Manajer Akademik. 9. Wakil Rektor II mendisposisikan ke Direktorat Keuangan untuk memproses pembayaran UKT. 10. Bendahara Penerimaan UKT membuka sistem pembayaran mahasiswa. 11. Mahasiswa membayar biaya UKT ke bank. Setelah sistem terbuka dan UKT berhasil dibayar, segera lakukan pengisian IRS (maksimal 3 hari kerja) dengan prosedur : 1. Mahasiswa melakukan her-registrasi online melalui SIAP. 2. Mahasiswa menemui Pembimbing Akademik untuk berkonsultasi mengenai mata kuliah yang akan diambil. Pembimbing memberikan arahan dengan mempertimbangkan ketentuan jumlah SKS maksimal pada semester terkait. 3. Mahasiswa melakukan pengisian, perbaikan, atau mencetak IRS sementara dengan bantuan operator masing-masing prodi secara online melalui SIAP. 4. Apabila mata kuliah dan jumlah SKS sudah sesuai, Pembimbing Akademik melakukan persetujuan secara online pada SIAP. 5. Mahasiswa dapat melakukan pengecekan IRS online di sistem SIAP bila diperlukan.
10	Saya sempat cuti, lalu bagaimana prosedurnya untuk saya dapat aktif kembali dalam akademik ?	Berikut prosedur untuk aktif kembali setelah cuti: - Bawa Surat Izin Cuti Akademik dan fotokopi KTM, lalu lapor kepada Ketua Program Studi dan Subbag Akademik dan Kemahasiswaan (±5 menit). - Lakukan registrasi online melalui SSO Anda (±5 menit). - Setelah itu, status Anda akan terdaftar sebagai Peserta Kuliah/Mahasiswa Aktif di fakultas (±5 menit). Catatan: Setelah aktif, jika sedang masa pengisian IRS, silakan: - Membayar UKT, her-registrasi melalui SIAP, mengisi/mencetak IRS, konsultasi dengan Pembimbing Akademik, dan minta persetujuan IRS secara online di SIAP.	1. Mahasiswa melakukan registrasi online pada SSO masing-masing mahasiswa. 2. Mahasiswa terdaftar sebagai Peserta Kuliah / Mahasiswa Aktif di Fakultas.

LAMPIRAN 5. Daftar Komponen yang Digunakan Pada RASA

1. Komponen NLU

No	Nama NLU	Tipe NLU	Examples
1	pembuka	*intent*	- hai - halo - yow - selamat pagi - selamat siang - selamat malam - test - hai, apa kabar? - halo, ada yang bisa dibantu? - saya mau tanya dong - permisi
2	penutup	*intent*	- sampai jumpa - bye - dadah - sampai nanti - oke - cukup
3	terima_kasih	*intent*	- terima kasih - makasih - thanks - thank you - terima kasih banyak - terima kasih banyak - makasih ya - makasih, bantuannya sudah cukup - makasih, sudah cukup - makasih, itu saja dulu - makasih, itu saja - okedeh, makasih
4	tanya_cuti_akademik	*intent*	- bagaimana cara mengajukan cuti akademik? - jelaskan bagaimana prosedur cuti akademik! - gimana cara saya dapat cuti kuliah? - bagaimana izin cuti kuliah? - bagaimaan cara cuti semester? - bagaimana SOP cuti akademik? - cara cuti kuliah - prosedur cuti
5	tanya_legalisir	*intent*	- bagaimana cara legalisir ijazah? - jelaskan prosedur legalisir transkrip! - bagaimana legalisir dokumen akademik? - bagaimana cara legalisir nilai? - bagaimana SOP legalisir ijazah dan transkrip? - cara legalisir dokumen - prosedur pengesahan ijazah
6	tanya_proposal_organisasi	*intent*	- bagaimana cara mengajukan proposal kegiatan organisasi? - jelaskan SOP pengajuan proposal organisasi mahasiswa! - bagaimana prosedur proposal kegiatan kemahasiswaan? - cara ajukan proposal organisasi - prosedur pengajuan acara kemahasiswaan
7	tanya_rekomendasi_beasiswa	*intent*	- bagaimana cara mengajukan rekomendasi beasiswa? - jelaskan SOP mengajukan rekomendasi beasiswa! - bagaimana SOP rekomendasi beasiswa?

No	Nama NLU	Tipe NLU	Examples
			- bagaimana prosedur pengajuan beasiswa? - cara minta surat rekomendasi beasiswa - prosedur pengajuan beasiswa
8	tanya_irs	*intent*	- bagaimana cara mengisi IRS? - jelaskan prosedur pengisian IRS! - bagaimana SOP pengisian isian rencana studi? - bagaimana mengisi KRS? - apa tahapan pengisian rencana studi semester? - cara isi KRS - prosedur pengisian rencana studi
9	tanya_izin_aktif_setelah_cuti	*intent*	- bagaimana cara aktif kuliah setelah cuti? - jelaskan prosedur izin aktif setelah cuti! - bagaimana kembali aktif setelah cuti? - bagaimana SOP izin aktif setelah cuti? - bagaimana registrasi setelah cuti akademik? - cara aktif kuliah lagi setelah cuti - prosedur kembali aktif
10	tanya_izin_terlambat_bayar_ukt	*intent*	- bagaimana cara mengajukan izin keterlambatan bayar UKT? - jelaskan prosedur keterlambatan pembayaran UKT! - bagaimana SOP izin keterlambatan UKT? - bagaimana mengajukan penundaan bayar ukt - cara minta izin telat bayar ukt - prosedur keterlambatan bayar uang kuliah
11	tanya_link_form	*intent*	- dimana link download form cuti akademik - dimana link download form rekomendasi beasiswa? - dimana link download form keterlambatan UKT? - dimana link form pengajuan? - dimana link download formulir? - link form cuti - download form dimana
12	tanya_waktu_proses	*intent*	- berapa lama prosesnya? - waktu pemrosesan - kapan selesainya? - berapa hari proses pengajuan? - maksimal waktu proses - berapa lama selesai? - cepat tidak prosesnya?
13	tanya_layanan_akademik	*intent*	- layanan apa saja yang tersedia? - apa saja yang bisa dibantu? - fitur apa yang ada? - bantuan apa yang disediakan? - menu layanan akademik - apa saja layanan yang ada?
14	tanya_kontak	*intent*	- nomor telepon akademik - kontak layanan mahasiswa - email akademik fsm - telepon subbag akademik - kontak bantuan akademik - nomor hubungi akademik
15	tanya_jadwal	*intent*	- jam berapa buka layanan? - jadwal operasional akademik - kapan buka layanan? - jam kerja subbag akademik - hari apa saja buka? - jadwal pelayanan
16	nlu_fallback	*intent*	- apa kabar?

No	Nama NLU	Tipe NLU	Examples
			- bagaimana cuaca hari ini? - siapa nama kamu? - kamu siapa? - ceritakan tentang dirimu - apa hobimu? - kamu tinggal dimana? - umur berapa? - kamu robot atau manusia? - bisa bahasa inggris? - ceritakan lelucon - apa makanan favoritmu? - siapa presiden indonesia? - berita hari ini - rekomendasi film - cara masak nasi goreng - tips belajar - olahraga apa yang bagus? - rekomendasi tempat wisata - bagaimana cara investasi?
17	cuti	*synonym*	- cuti akademik - izin cuti - cuti semester
18	legalisir	*synonym*	- legalisir ijazah - legalisir transkrip - pengesahan dokumen
19	proposal	*synonym*	- proposal kegiatan - proposal organisasi - pengajuan acara
20	irs	*synonym*	- isian rencana studi - krs - rencana studi - kartu rencana studi
21	ukt	*synonym*	- uang kuliah tunggal - biaya pendidikan - pembayaran ukt

2. Komponen *Rules*

No	***Rule***	***Steps***
1	Sambut salam	steps: - intent: pembuka - action: utter_pembuka
2	Ucapkan sampai jumpa	steps: - intent: penutup - action: utter_penutup
3	Ucapkan terima kasih	steps: - intent: terima_kasih - action: utter_terima_kasih
4	Respon pertanyaan cuti akademik	steps: - intent: tanya_cuti_akademik - action: utter_jawab_cuti_akademik
5	Respon pertanyaan legalisir	steps: - intent: tanya_legalisir - action: utter_jawab_legalisir
6	Respon pertanyaan proposal organisasi	steps: - intent: tanya_proposal_organisasi

No	*Rule*	*Steps*
		- action: utter_jawab_proposal_organisasi
7	Respon pertanyaan rekomendasi beasiswa	steps: - intent: tanya_rekomendasi_beasiswa - action: utter_jawab_rekomendasi_beasiswa
8	Respon pertanyaan IRS	steps: - intent: tanya_irs - action: utter_jawab_irs
9	Respon pertanyaan izin aktif setelah cuti	steps: - intent: tanya_izin_aktif_setelah_cuti - action: utter_jawab_izin_aktif_kuliah_setelah_cuti
10	Respon pertanyaan izin terlambat membayar UKT	steps: - intent: tanya_izin_terlambat_bayar_ukt - action: utter_jawab_izin_terlambat_bayar_ukt
11	Respon pertanyaan link formulir	steps: - intent: tanya_link_form - action: utter_jawab_link_form
12	Respon pertanyaan lama waktu proses	steps: - intent: tanya_waktu_proses - action: utter_jawab_waktu_proses
13	Respon pertanyaan layanan akademik	steps: - intent: tanya_layanan_akademik - action: utter_layanan_akademik
14	Respon pertanyaan kontak	steps: - intent: tanya_kontak - action: utter_kontak
15	Respon pertanyaan jadwal	steps: - intent: tanya_jadwal - action: utter_jadwal
16	Respon pertanyaan di luar scope	steps: - intent: nlu_fallback - action: utter_diluar_scope

3. Komponen *Stories*

No	*Story*	*Steps*
1	Cuti akademik process	steps: - intent: tanya_cuti_akademik - action: utter_jawab_cuti_akademik - intent: tanya_link_form - action: utter_jawab_link_form
2	Legalisir process	steps: - intent: tanya_legalisir - action: utter_jawab_legalisir - intent: tanya_waktu_proses - action: utter_jawab_waktu_proses
3	Proposal organisasi process	steps: - intent: tanya_proposal_organisasi - action: utter_jawab_proposal_organisasi
4	Beasiswa process	steps: - intent: tanya_rekomendasi_beasiswa - action: utter_jawab_rekomendasi_beasiswa - intent: tanya_link_form - action: utter_jawab_link_form
5	IRS process	steps: - intent: tanya_irs - action: utter_jawab_irs
6	Aktif setelah cuti process	steps:

No	*Story*	*Steps*
		- intent: tanya_izin_aktif_setelah_cuti - action: utter_jawab_izin_aktif_kuliah_setelah_cuti
7	Keterlambatan UKT process	steps: - intent: tanya_izin_terlambat_bayar_ukt - action: utter_jawab_izin_terlambat_bayar_ukt - intent: tanya_link_form - action: utter_jawab_link_form
8	Greeting and asking about services	steps: - intent: pembuka - action: utter_pembuka - intent: tanya_layanan_akademik - action: utter_layanan_akademik - intent: tanya_cuti_akademik - action: utter_jawab_cuti_akademik
9	Multiple questions about academic services	steps: - intent: tanya_irs - action: utter_jawab_irs - intent: tanya_legalisir - action: utter_jawab_legalisir - intent: terima_kasih - action: utter_terima_kasih
10	Information about contact and schedule	steps: - intent: tanya_kontak - action: utter_kontak - intent: tanya_jadwal - action: utter_jadwal
11	Complete service inquiry	steps: - intent: pembuka - action: utter_pembuka - intent: tanya_layanan_akademik - action: utter_layanan_akademik - intent: tanya_cuti_akademik - action: utter_jawab_cuti_akademik - intent: tanya_waktu_proses - action: utter_jawab_waktu_proses - intent: terima_kasih - action: utter_terima_kasih - intent: penutup - action: utter_penutup

4. Komponen Domain

No	Jenis Sub Komponen	*Values*
1	intents	- pembuka - penutup - terima_kasih - tanya_cuti_akademik - tanya_legalisir - tanya_proposal_organisasi - tanya_rekomendasi_beasiswa - tanya_irs - tanya_izin_aktif_setelah_cuti - tanya_izin_terlambat_bayar_ukt - tanya_link_form - tanya_waktu_proses - tanya_layanan_akademik - tanya_kontak

		- tanya_jadwal - nlu_fallback
2	entities	- layanan - dokumen
3	slots	layanan: type: text influence_conversation: true mappings: - type: from_entity entity: layanan
4	responses	utter_pembuka: - text: "Halo! Saya chatbot layanan akademik FSM. Ada yang bisa saya bantu hari ini?" - text: "Selamat datang! Saya siap membantu Anda dengan informasi layanan akademik." - text: "Hai! Silakan tanyakan tentang layanan akademik yang Anda butuhkan." utter_penutup: - text: "Sampai jumpa! Semoga informasi yang diberikan membantu studi Anda." - text: "Terima kasih! Jangan ragu untuk bertanya lagi jika butuh bantuan." - text: "Sampai ketemu lagi! Semoga sukses dengan urusan akademiknya." utter_terima_kasih: - text: "Sama-sama! Senang bisa membantu. Ada lagi yang bisa saya bantu?" - text: "Dengan senang hati! Jika ada pertanyaan lain tentang layanan akademik, silakan tanyakan." - text: "Terima kasih kembali! Jangan sungkan bertanya jika butuh informasi lain." utter_jawab_cuti_akademik: - text: | Prosedur Cuti Akademik (Maksimal 3 hari kerja): 1. Unduh, isi, dan tandatangani Form Cuti Akademik, lampirkan persyaratan dan isi online via SIAP 2. Minta persetujuan dan tanda tangan Ketua Program Studi 3. Serahkan form yang sudah ditandatangani ke Dekan untuk disposisi ke Subbag Akademik dan Kemahasiswaan 4. Subbag Akademik memeriksa kelengkapan dan memproses 5. Dekan menandatangani Surat Izin Dekan 6. Subbag Sumber Daya memberi nomor surat dan mengirimkannya 7. Mahasiswa dapat mengambil Surat Izin Dekan di loket akademik utter_jawab_legalisir: - text: | Prosedur Legalisir Ijazah dan Transkrip (Maksimal 3 hari kerja): 1. Serahkan fotokopi ijazah/transkrip/sertifikat beserta aslinya ke Subbag Akademik 2. Petugas memeriksa keabsahan dan memberikan cap 3. Supervisor memeriksa dan memberi paraf 4. Dekan/Wakil Dekan menandatangani 5. Petugas memberikan stempel fakultas

		6. Ambil legalisir dengan membawa aslinya dan tanda tangan di buku pengambilan utter_jawab_proposal_organisasi: - text: | Prosedur Pengajuan Proposal Kegiatan Organisasi Mahasiswa (Maksimal 3 hari kerja): 1. Serahkan proposal ke petugas kemahasiswaan dan Supervisor Akademik untuk registrasi dan alokasi ruang 2. Supervisor meneliti dan memverifikasi proposal, memberikan paraf 3. Wakil Dekan mengevaluasi substansi proposal 4. Pengesahan oleh Wakil Dekan 5. Ambil proposal yang telah disetujui utter_jawab_rekomendasi_beasiswa: - text: | Prosedur Pengajuan Rekomendasi Beasiswa (Maksimal 3 hari 45 menit): 1. Download dan isi formulir Surat Rekomendasi Beasiswa ke Subbag Akademik 2. BAK Fakultas meneliti dan memverifikasi 3. BAK memberikan paraf dan nomor surat 4. Wakil Dekan menandatangani 5. Ambil surat di BAK Fakultas dan catat bukti pengambilan utter_jawab_irs: - text: | Prosedur Pengisian IRS (Isian Rencana Studi): 1. Bayar biaya pendidikan di bank yang bekerja sama (BNI, Mandiri, BTN, BRI) 2. Lakukan her-registrasi online melalui SIAP 3. Isi, perbaiki, atau cetak IRS sementara via SIAP 4. Temui Pembimbing Akademik untuk konsultasi mata kuliah 5. Pembimbing Akademik melakukan persetujuan online di SIAP 6. Cek IRS online di sistem SIAP bila diperlukan utter_jawab_izin_aktif_kuliah_setelah_cuti: - text: | Prosedur Izin Aktif Setelah Cuti: 1. Bawa surat izin cuti akademik semester ke Ketua Program Studi dan Subbag Akademik 2. Lakukan registrasi online pada SSO masing-masing 3. Mahasiswa terdaftar sebagai Peserta Kuliah/Mahasiswa Aktif utter_jawab_izin_terlambat_bayar_ukt: - text: | Prosedur Izin Keterlambatan Pembayaran UKT (Maksimal 3 hari kerja): 1. Unduh, isi, dan tandatangani Form Permohonan Keterlambatan UKT dengan lampiran 2. Minta tanda tangan dosen wali dan persetujuan Ketua Program Studi 3. Serahkan form yang sudah ditandatangani ke Supervisor Sumber Daya 4. Supervisor memeriksa kelengkapan dan memberikan paraf 5. Wakil Dekan Sumber Daya memproses surat permohonan 6. Wakil Dekan Sumber Daya menandatangani surat 7. Subbag Sumber Daya memproses surat 8. Bawa surat ke Wakil Rektor II melalui Manajer Akademik

		9. Wakil Rektor II mendisposisikan ke Direktorat Keuangan 10. Bendahara membuat sistem pembayaran 11. Bayar UKT ke bank utter_jawab_link_form: - text: | Link Download Formulir: Form Cuti Akademik: https://drive.google.com/file/d/1miPHNvRj6XO6LEhFCiC_QXcMtynCPWR2/edit Form Rekomendasi Beasiswa: https://drive.google.com/file/d/18f_nbpXHbElgRNoplDFQVNYGGDkFxBTT/view Form Keterlambatan Pembayaran UKT: https://drive.google.com/file/d/1o7hogihjZsFTB_V27JU1OfNGBuI2Sr55/ utter_jawab_waktu_proses: - text: | Waktu Pemrosesan Layanan Akademik: Cuti Akademik: Maksimal 3 hari kerja Legalisir: Maksimal 3 hari kerja Proposal Organisasi: Maksimal 3 hari kerja Rekomendasi Beasiswa: Maksimal 3 hari 45 menit Keterlambatan UKT: Maksimal 3 hari kerja Pengisian IRS: Maksimal 3 hari kerja Aktif Setelah Cuti: ±15 menit utter_layanan_akademik: - text: | Layanan Akademik yang Tersedia: Cuti Akademik Legalisir Ijazah & Transkrip Proposal Kegiatan Organisasi Rekomendasi Beasiswa Pengisian IRS (Isian Rencana Studi) Izin Aktif Setelah Cuti Izin Keterlambatan Bayar UKT Silakan pilih layanan yang ingin Anda tanyakan! utter_kontak: - text: | Kontak Layanan Akademik FSM: Subbag Akademik dan Kemahasiswaan Telepon: (024) 747 4754 Email: akademikfsm@live.undip.ac.id utter_jadwal: - text: | Informasi Jadwal Layanan: Jam Operasional: Senin - Jumat, 08.00 - 16.00 WIB

		Lokasi: Gedung Acintya Prasada Lt. 1, Fakultas Sains dan Matematika, UNDIP Hari Libur: Mengikuti kalender akademik UNDIP utter_diluar_scope: - text: "Maaf, saya belum dapat membantu anda untuk pertanyaan tersebut." - text: "Mohon maaf, saya belum dapat menjawab pertanyaan anda. Silakan hubungi Subbag Akademik langsung." - text: "Saya hanya terbatas pada informasi yang disediakan. Untuk pertanyaan lain, silakan hubungi Subbag Akademik langsung."
5	session_config	session_expiration_time: 180 carry_over_slots_to_new_session: true

LAMPIRAN 6. Jawaban dan *Ground Truth* Evaluasi Model RASA

No	Pertanyaan	Jawaban	*Ground Truth*
1	Bagaimana proses pengajuan cuti akademik ?	Prosedur Cuti Akademik (Maksimal 3 hari kerja): 1. Unduh, isi, dan tandatangani Form Cuti Akademik, lampirkan persyaratan dan isi online via SIAP 2. Minta persetujuan dan tanda tangan Ketua Program Studi 3. Serahkan form yang sudah ditandatangani ke Dekan untuk disposisi ke Subbag Akademik dan Kemahasiswaan 4. Subbag Akademik memeriksa kelengkapan dan memproses 5. Dekan menandatangani Surat Izin Dekan 6. Subbag Sumber Daya memberi nomor surat dan mengirimkannya 7. Mahasiswa dapat mengambil Surat Izin Dekan di loket akademik	1. Mahasiswa mengunduh, mengisi, dan menandatangani Form Cuti Akademik dengan melampirkan persyaratan. Selain itu, mahasiswa juga wajib mengisi secara online melalui SIAP. Dokumen yang dibutuhkan: Form Cuti Akademik, Transkrip Akademik, Bukti bayar SPP/UKT terakhir, Fotokopi KTM, dan dokumen pendukung lain Link unduh form: https://drive.google.com/file/d/1miPHNvRj6XO6LEhFCiC_QXcMtynCPWR2/edit 2. Mahasiswa meminta persetujuan dan tanda tangan Ketua Program Studi. 3. Mahasiswa menyerahkan Form Cuti Akademik yang telah ditandatangani Ketua Program Studi ke Dekan. Kemudian dilakukan disposisi ke Subbag Akademik dan Kemahasiswaan. 4. Subbag Akademik dan Kemahasiswaan menerima, memeriksa, dan meneliti kelengkapan persyaratan form cuti akademik. Jika lengkap, form diberi paraf dan diproses lebih lanjut. 5. Dekan menandatangani Surat Izin Dekan. 6. Subbag Sumber Daya memberi nomor surat dan mengirimkannya ke Subbag Akademik dan Kemahasiswaan dengan tembusan ke Dosen Wali dan Ketua Program Studi. 7. Subbag Akademik dan Kemahasiswaan mengupdate status mahasiswa di Sistem Akademik, mengarsipkan Surat Izin Dekan, dan mengirimkan kepada mahasiswa. 8. Mahasiswa dapat mengambil Surat Izin Dekan di loket akademik.
2	Berapa lama waktu yang dibutuhkan	Prosedur Legalisir Ijazah dan Transkrip (Maksimal 3 hari kerja):	Proses legalisir maksimal dilakukan selama 3 hari kerja.

No	Pertanyaan	Jawaban	*Ground Truth*
	untuk dapat melakukan legalisir ijazah dan atau transkrip ?	1. Serahkan fotokopi ijazah/transkrip/sertifikat beserta aslinya ke Subbag Akademik 2. Petugas memeriksa keabsahan dan memberikan cap 3. Supervisor memeriksa dan memberi paraf 4. Dekan/Wakil Dekan menandatangani 5. Petugas memberikan stempel fakultas 6. Ambil legalisir dengan membawa aslinya dan tanda tangan di buku pengambilan	
3	Berapa jumlah maksimal lembar legalisir yang dapat mintakan untuk ijazah dan atau transkrip ?	Prosedur Legalisir Ijazah dan Transkrip (Maksimal 3 hari kerja): 1. Serahkan fotokopi ijazah/transkrip/sertifikat beserta aslinya ke Subbag Akademik 2. Petugas memeriksa keabsahan dan memberikan cap 3. Supervisor memeriksa dan memberi paraf 4. Dekan/Wakil Dekan menandatangani 5. Petugas memberikan stempel fakultas 6. Ambil legalisir dengan membawa aslinya dan tanda tangan di buku pengambilan	Masing-masing maksimal sebanyak 10 lembar untuk ijazah dan atau transkrip.
4	Persyaratan apa yang diperlukan apabila pengambilan legalisir ijazah dan atau transkrip diwakilkan oleh pihak lain ?	Prosedur Legalisir Ijazah dan Transkrip (Maksimal 3 hari kerja): 1. Serahkan fotokopi ijazah/transkrip/sertifikat beserta aslinya ke Subbag Akademik 2. Petugas memeriksa keabsahan dan memberikan cap 3. Supervisor memeriksa dan memberi paraf 4. Dekan/Wakil Dekan menandatangani 5. Petugas memberikan stempel fakultas 6. Ambil legalisir dengan membawa aslinya dan tanda tangan di buku pengambilan	Pihak perwakilan wajib membawa surat kuasa dari pihak yang bersangkutan. Namun surat kuasa tidak perlu diberi materai.
5	Bagaimana prosedur dan apa syarat yang diperlukan untuk dapat melakukan pengajuan legalisir ijazah dan atau transkrip ?	Prosedur Legalisir Ijazah dan Transkrip (Maksimal 3 hari kerja): 1. Serahkan fotokopi ijazah/transkrip/sertifikat beserta aslinya ke Subbag Akademik 2. Petugas memeriksa keabsahan dan memberikan cap	1. Alumni menyerahkan foto copy ijazah/transkrip nilai/sertifikat akreditasi program studi beserta surat aslinya kepada petugas Subbag Akademik dan Kemahasiswaan.

No	Pertanyaan	Jawaban	*Ground Truth*
		3. Supervisor memeriksa dan memberi paraf 4. Dekan/Wakil Dekan menandatangani 5. Petugas memberikan stempel fakultas 6. Ambil legalisir dengan membawa aslinya dan tanda tangan di buku pengambilan	Dokumen yang dibutuhkan: Berkas Pengajuan Legalisir 2. Petugas Subbag Akademik dan Kemahasiswaan memeriksa keabsahan, kesesuaian foto copy ijazah/transkrip nilai/sertifikat akreditasi program studi dengan surat aslinya dan memberikan cap. 3. Supervisor Akademik dan Kemahasiswaan memeriksa dan memberi paraf di samping kanan nama Dekan. 4. Dekan/Wakil Dekan Akademik dan Kemahasiswaan menandatangani pada "Pengesahan Telah diperiksa Kebenarannya dan Sesuai dengan Aslinya". 5. Petugas Subbag Akademik dan Kemahasiswaan mengambil kembali Legalisir yang sudah ditanda tangani Dekan / Wakil Dekan Akademik dan Kemahasiswaan dan memberikan stempel Fakultas Sains dan Matematika. 6. Alumni mengambil Legalisir di Subbag Akademik dan Kemahasiswaan dengan membawa aslinya dan menulis di buku pengambilan dan di tanda tangani ybs.
6	Apa syarat mengajukan proposal kegiatan yang terkait dengan orientasi mahasiswa baru ?	Prosedur Pengajuan Proposal Kegiatan Organisasi Mahasiswa (Maksimal 3 hari kerja): 1. Serahkan proposal ke petugas kemahasiswaan dan Supervisor Akademik untuk registrasi dan alokasi ruang 2. Supervisor meneliti dan memverifikasi proposal, memberikan paraf 3. Wakil Dekan mengevaluasi substansi proposal 4. Pengesahan oleh Wakil Dekan 5. Ambil proposal yang telah disetujui	Semua kegiatan yang berkaitan dengan mahasiswa baru diwajibkan membuat surat pernyataan dari dosen pendamping.
7	Bagaimana prosedur yang dapat dilakukan untuk mendapatkan rekomendasi beasiswa ?	Prosedur Pengajuan Rekomendasi Beasiswa (Maksimal 3 hari 45 menit): 1. Download dan isi formulir Surat Rekomendasi Beasiswa ke Subbag Akademik	1. Mahasiswa mendownload dan mengisi formulir/ Surat Rekomendasi Pengajuan Beasiswa ke loket Subbag Akademik dan Kemahasiswaan (BAK) Fakultas.

No	Pertanyaan	Jawaban	*Ground Truth*
		2. BAK Fakultas meneliti dan memverifikasi 3. BAK memberikan paraf dan nomor surat 4. Wakil Dekan menandatangani 5. Ambil surat di BAK Fakultas dan catat bukti pengambilan	2. BAK Fakultas meneliti dan memverifikasi Surat Rekomendasi Pengajuan Beasiswa. 3. BAK Fakultas memberikan paraf dan nomor surat pada Surat Rekomendasi Pengajuan Beasiswa tersebut. 4. Wakil Dekan Akademik dan Kemahasiswaan memberikan tanda tangan pada Surat Rekomendasi Pengajuan Beasiswa tersebut. 5. Mahasiswa mengambil surat tersebut di BAK Fakultas dan mencatat bukti pengambilan.
8	Bagaimana jika mata kuliah yang ingin diambil belum muncul di IRS ?	Mohon maaf, saya belum dapat menjawab pertanyaan anda. Silakan hubungi Subbag Akademik langsung.	Untuk dapat memperoleh bantuan lebih lanjut, bisa hubungi operator prodi masing-masing.
9	Bagaimana jika saya terlambat melakukan pembayaran UKT dan pengisian IRS?	Prosedur Izin Keterlambatan Pembayaran UKT (Maksimal 3 hari kerja): 1. Unduh, isi, dan tandatangani Form Permohonan Keterlambatan UKT dengan lampiran 2. Minta tanda tangan dosen wali dan persetujuan Ketua Program Studi 3. Serahkan form yang sudah ditandatangani ke Supervisor Sumber Daya 4. Supervisor memeriksa kelengkapan dan memberikan paraf 5. Wakil Dekan Sumber Daya memproses surat permohonan 6. Wakil Dekan Sumber Daya menandatangani surat 7. Subbag Sumber Daya memproses surat 8. Bawa surat ke Wakil Rektor II melalui Manajer Akademik 9. Wakil Rektor II mendisposisikan ke Direktorat Keuangan 10. Bendahara membuat sistem pembayaran 11. Bayar UKT ke bank	Ajukan izin keterlambatan pembayaran UKT (maksimal 3 hari kerja) dengan prosedur : 1. Mahasiswa mengunduh, mengisi, dan menandatangani Form Permohonan Keterlambatan Pembayaran UKT serta melampirkan persyaratan. 2. Mahasiswa meminta tanda tangan dosen wali serta persetujuan dan tanda tangan Ketua Program Studi. 3. Mahasiswa menyerahkan Form Permohonan Keterlambatan UKT yang telah ditandatangani dan disetujui kepada Supervisor Sumber Daya. 4. Supervisor Sumber Daya menerima, memeriksa, dan meneliti kelengkapan persyaratan form, serta memberikan paraf. 5. Wakil Dekan Sumber Daya memproses Surat Permohonan Izin Keterlambatan Pembayaran UKT. 6. Wakil Dekan Sumber Daya menandatangani Surat Permohonan Izin Keterlambatan Pembayaran UKT. 7. Subbag Sumber Daya memproses Surat Permohonan Keterlambatan Pembayaran UKT. 8. Pihak fakultas membawa Surat Permohonan Keterlambatan Pembayaran UKT kepada Wakil Rektor II melalui Manajer Akademik. 9. Wakil Rektor II mendisposisikan ke Direktorat Keuangan untuk memproses pembayaran UKT.

No	Pertanyaan	Jawaban	*Ground Truth*
			10. Bendahara Penerimaan UKT membuka sistem pembayaran mahasiswa. 11. Mahasiswa membayar biaya UKT ke bank. Setelah sistem terbuka dan UKT berhasil dibayar, segera lakukan pengisian IRS (maksimal 3 hari kerja) dengan prosedur : 1. Mahasiswa melakukan her-registrasi online melalui SIAP. 2. Mahasiswa menemui Pembimbing Akademik untuk berkonsultasi mengenai mata kuliah yang akan diambil. Pembimbing memberikan arahan dengan mempertimbangkan ketentuan jumlah SKS maksimal pada semester terkait. 3. Mahasiswa melakukan pengisian, perbaikan, atau mencetak IRS sementara dengan bantuan operator masing-masing prodi secara online melalui SIAP. 4. Apabila mata kuliah dan jumlah SKS sudah sesuai, Pembimbing Akademik melakukan persetujuan secara online pada SIAP. 5. Mahasiswa dapat melakukan pengecekan IRS online di sistem SIAP bila diperlukan.
10	Saya sempat cuti, lalu bagaimana prosedurnya untuk saya dapat aktif kembali dalam akademik ?	Mohon maaf, saya belum dapat menjawab pertanyaan anda. Silakan hubungi Subbag Akademik langsung.	1. Mahasiswa melakukan registrasi online pada SSO masing-masing mahasiswa. 2. Mahasiswa terdaftar sebagai Peserta Kuliah / Mahasiswa Aktif di Fakultas.